

Vue.js를 활용한 프론트엔드 개발

CONTENTS

목차

CHAPTER 0

웹 작동 원리

CHAPTER 1

개발환경 설정

CHAPTER 2

HTML

CHAPTER 3

주요 HTML 엘리먼트 실습

CHAPTER 4

CSS

CHAPTER 5

Flexbox로 레이아웃 구성하기

CHAPTER 6

폼 스타일링 실습

CHAPTER 7

반응형 웹 디자인

CHAPTER 8

자바스크립트 소개

CONTENTS

목차

CHAPTER 9

변수와 자료형

CHAPTER 10

배열과 객체

CHAPTER 11

연산자 및 제어문

CHAPTER 12

함수와 파라미터

CHAPTER 13

이벤트 처리

CHAPTER 14

비동기처리

CHAPTER 15

개발 환경 추가

CONTENTS

목차

CHAPTER 16

Vue.js 개요 및 특징

CHAPTER 17

Vite로 Vue 프로젝트 시작하기

CHAPTER 18

Vue 기본 문법 -데이터 바인딩 & 이벤트 처리

CHAPTER 19

Vue Directive - 조건문과 반복문

CHAPTER 20

컴포넌트 기반 개발 - props와 emit

CHAPTER 21

Vue Router로 페이지 이동 처리하기

CHAPTER 22

Pinia를 활용한 상태 관리

CHAPTER 23

axios로 백엔드와 통신

CHAPTER 9

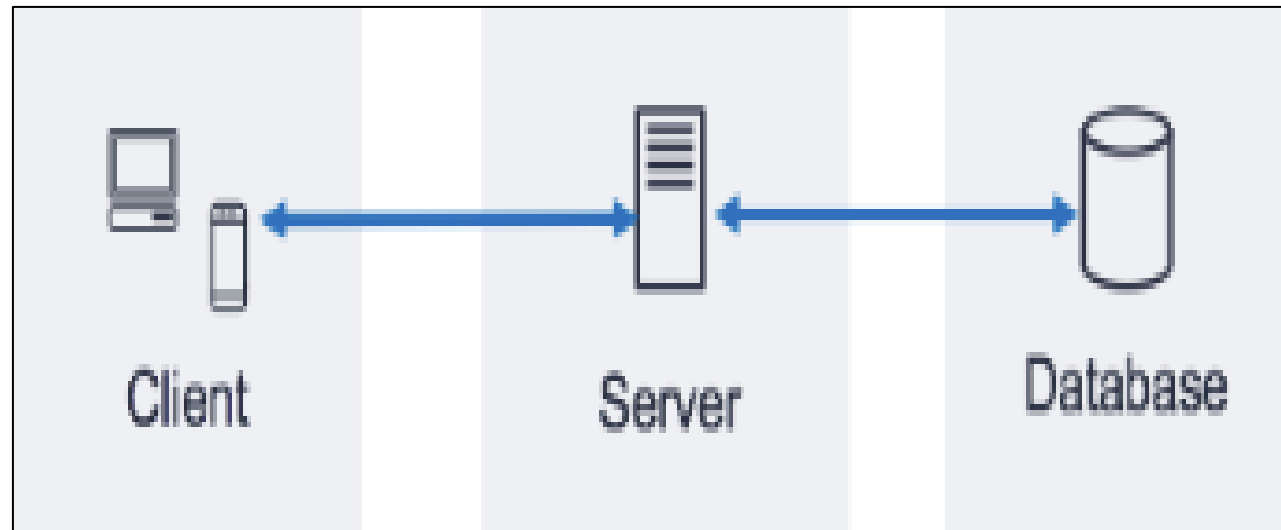
Echarts로 데이터 시각화 하기

Chapter 01

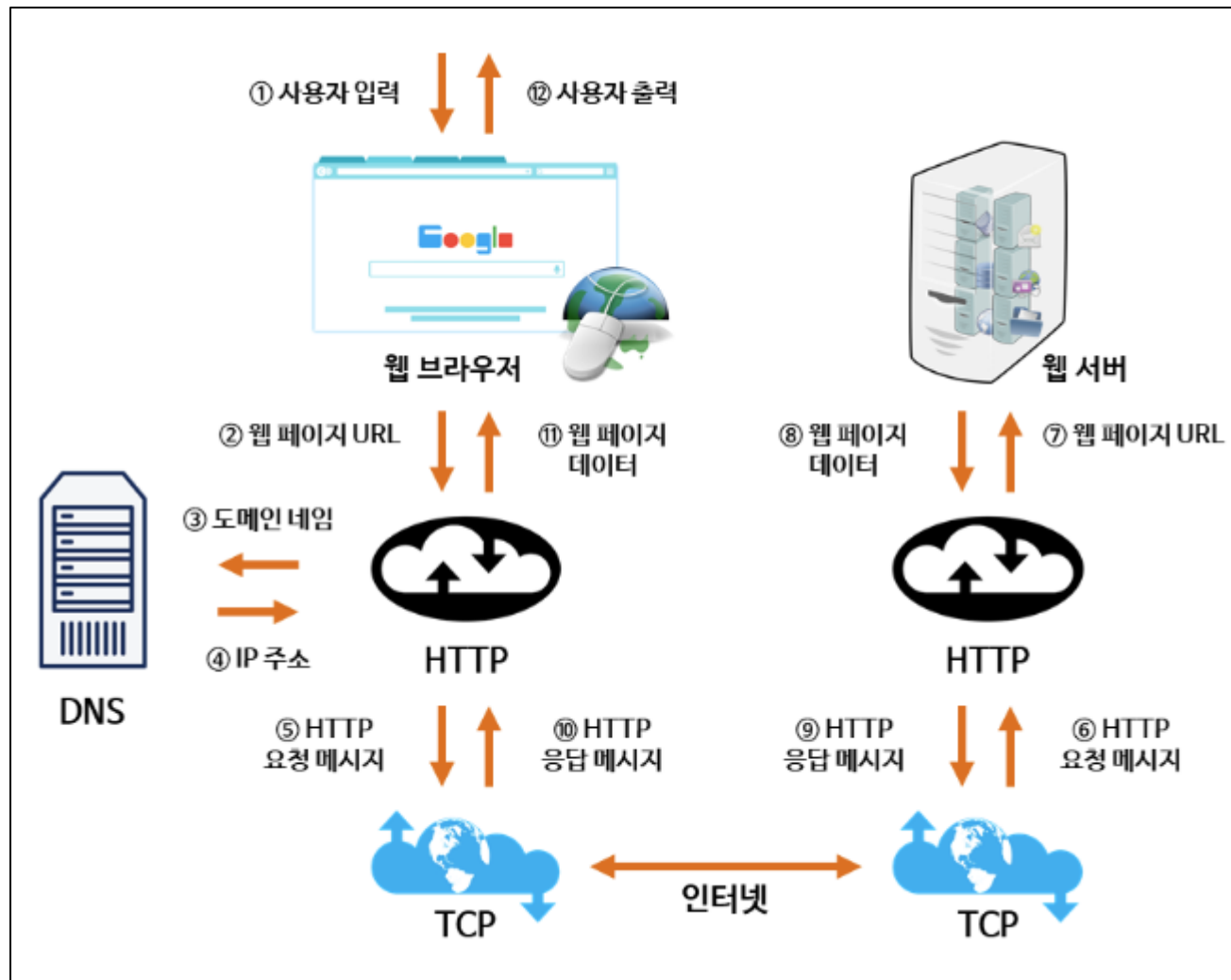
웹 작동 원리

Vue.js를 활용한 프론트 앤드 개발

웹 작동 원리



웹 작동 원리



Chapter 02

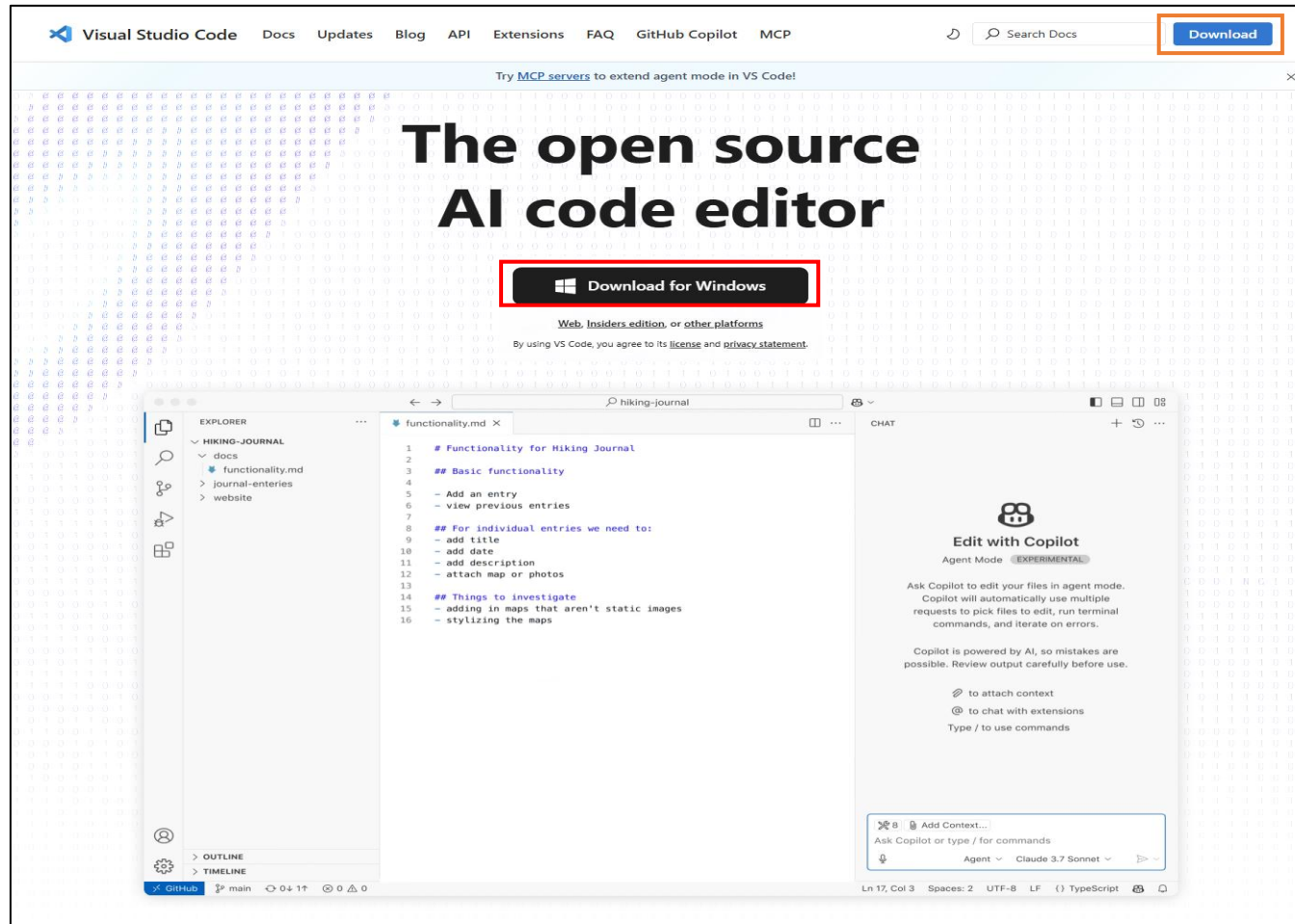
개발환경 설정

Vue.js를 활용한 프론트 앤드 개발

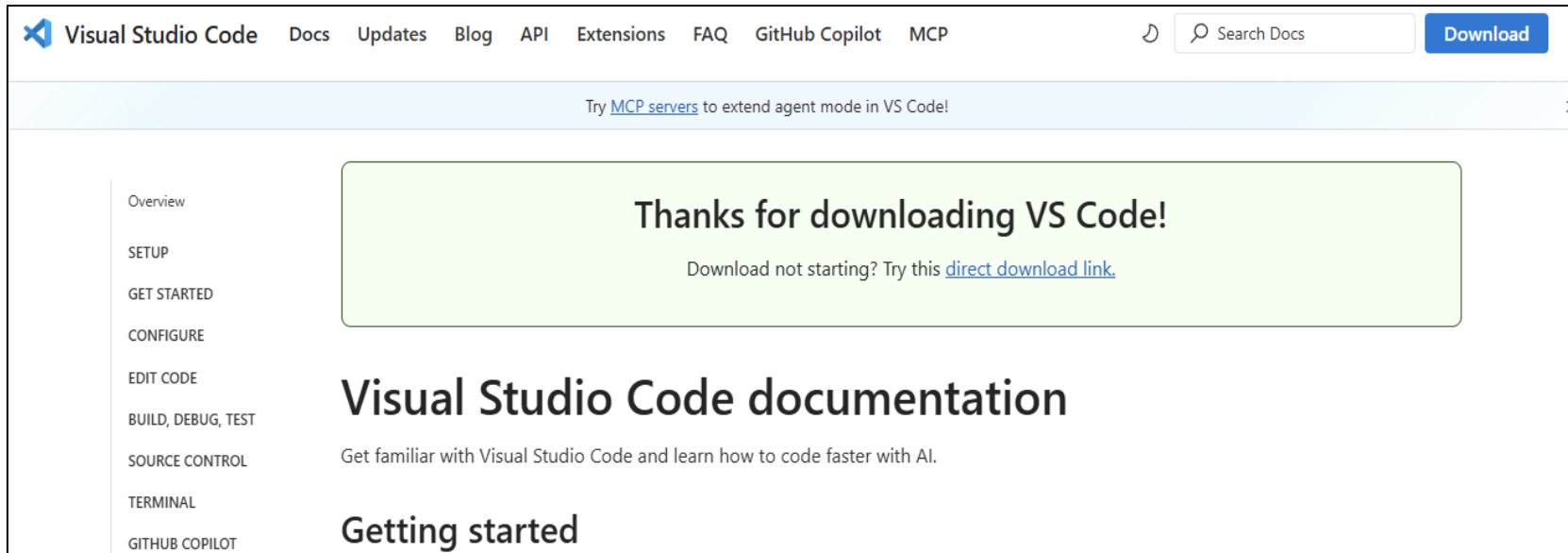
교안대로 설치하되, 화면이 다르더라도 **기본값**으로 설치 진행

개발환경 설정

- ✓ 검색엔진에 vscode 검색
- ✓ <https://code.visualstudio.com/> 방문



25.07 window기준



개발환경 설정

- ✓ 본인에게 맞는 운영체제 설치

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.




Windows
 Windows 10, 11




.deb
 Debian, Ubuntu


.rpm
 Red Hat, Fedora, SUSE




Mac
 macOS 11.0+

User Installer x64 Arm64

System Installer x64 Arm64

.zip x64 Arm64

CLI x64 Arm64

.deb x64 Arm32 Arm64

.rpm x64 Arm32 Arm64

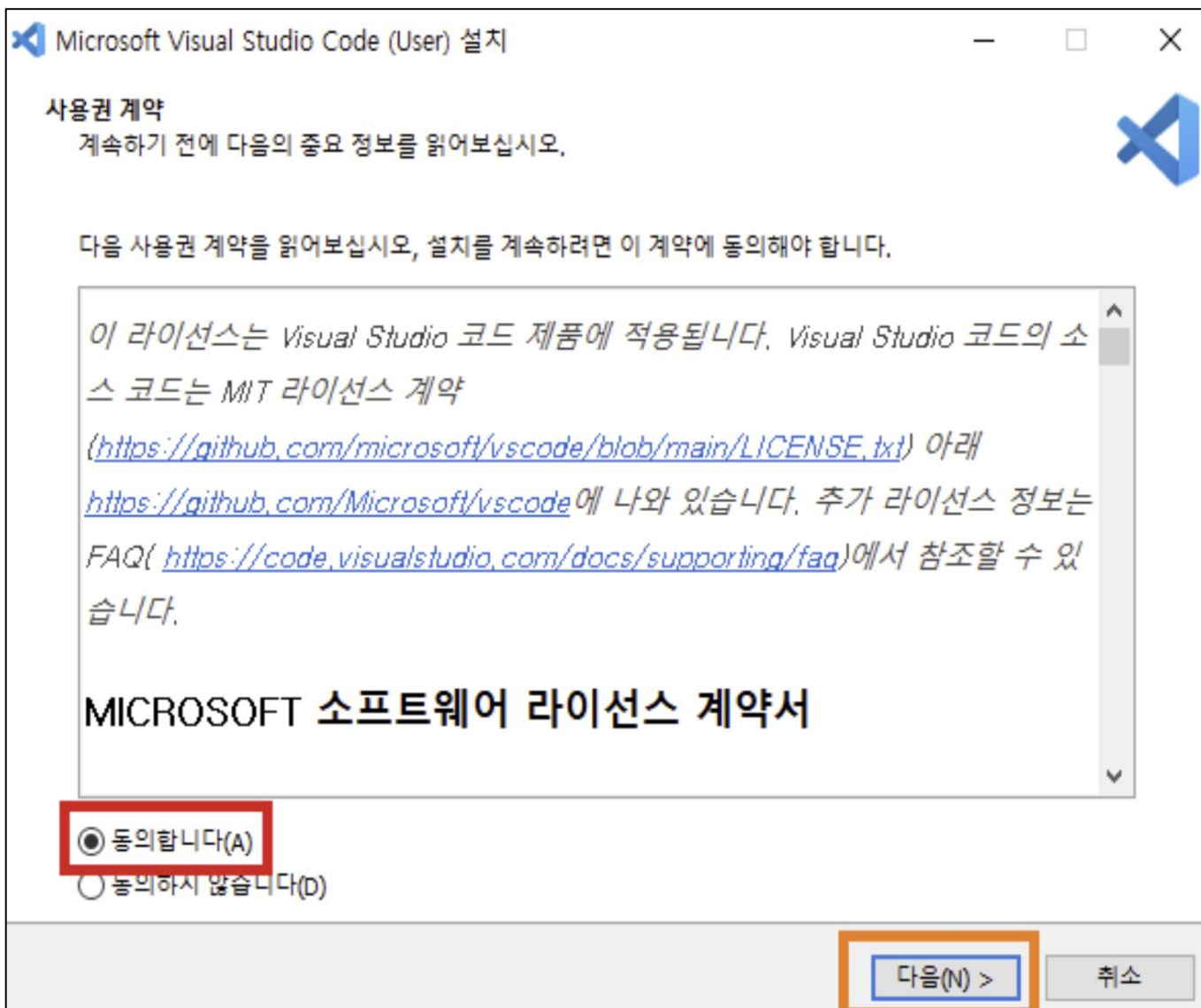
.tar.gz x64 Arm32 Arm64

Snap Snap Store

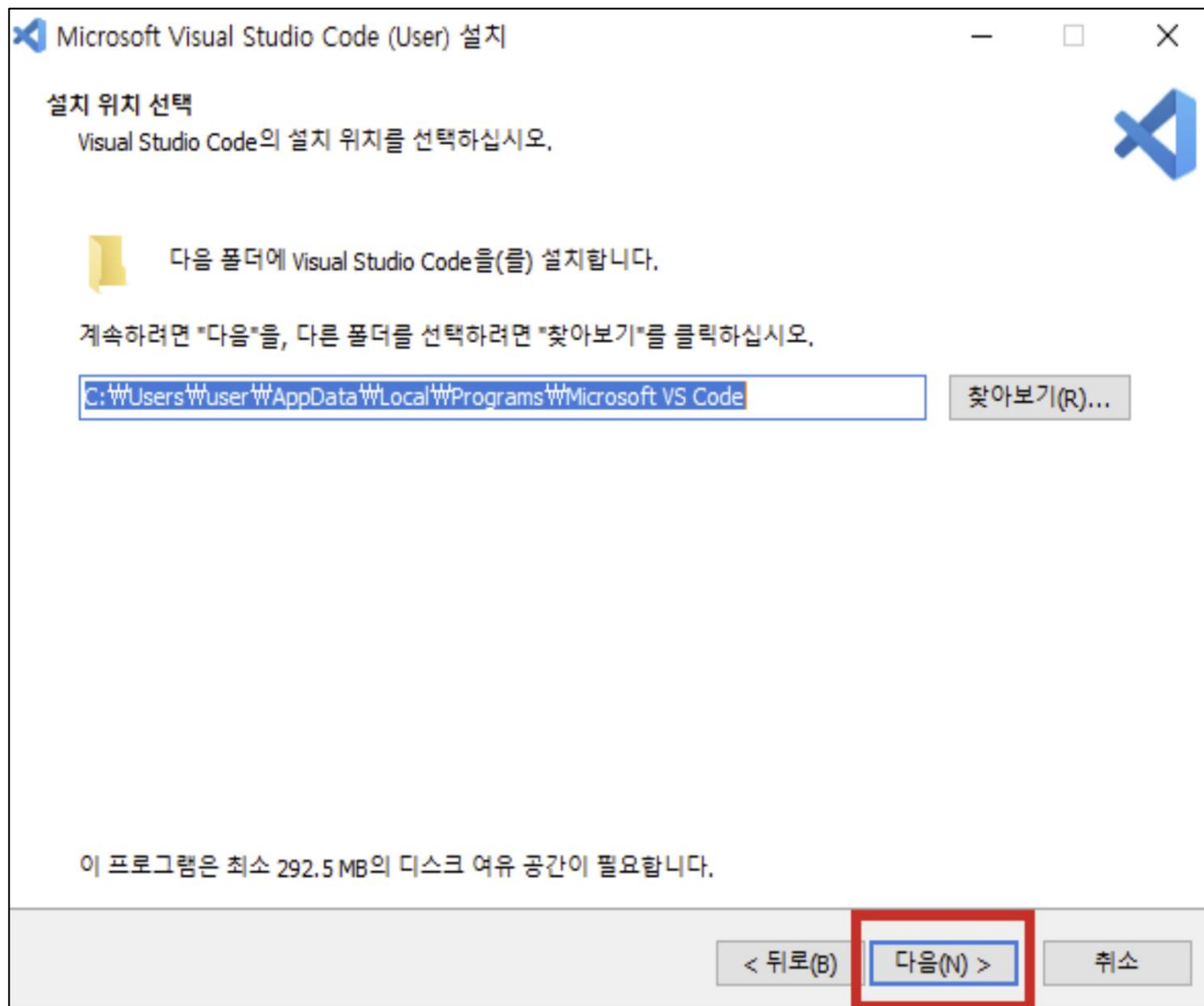
CLI x64 Arm32 Arm64

.zip Intel chip Apple silicon Universal

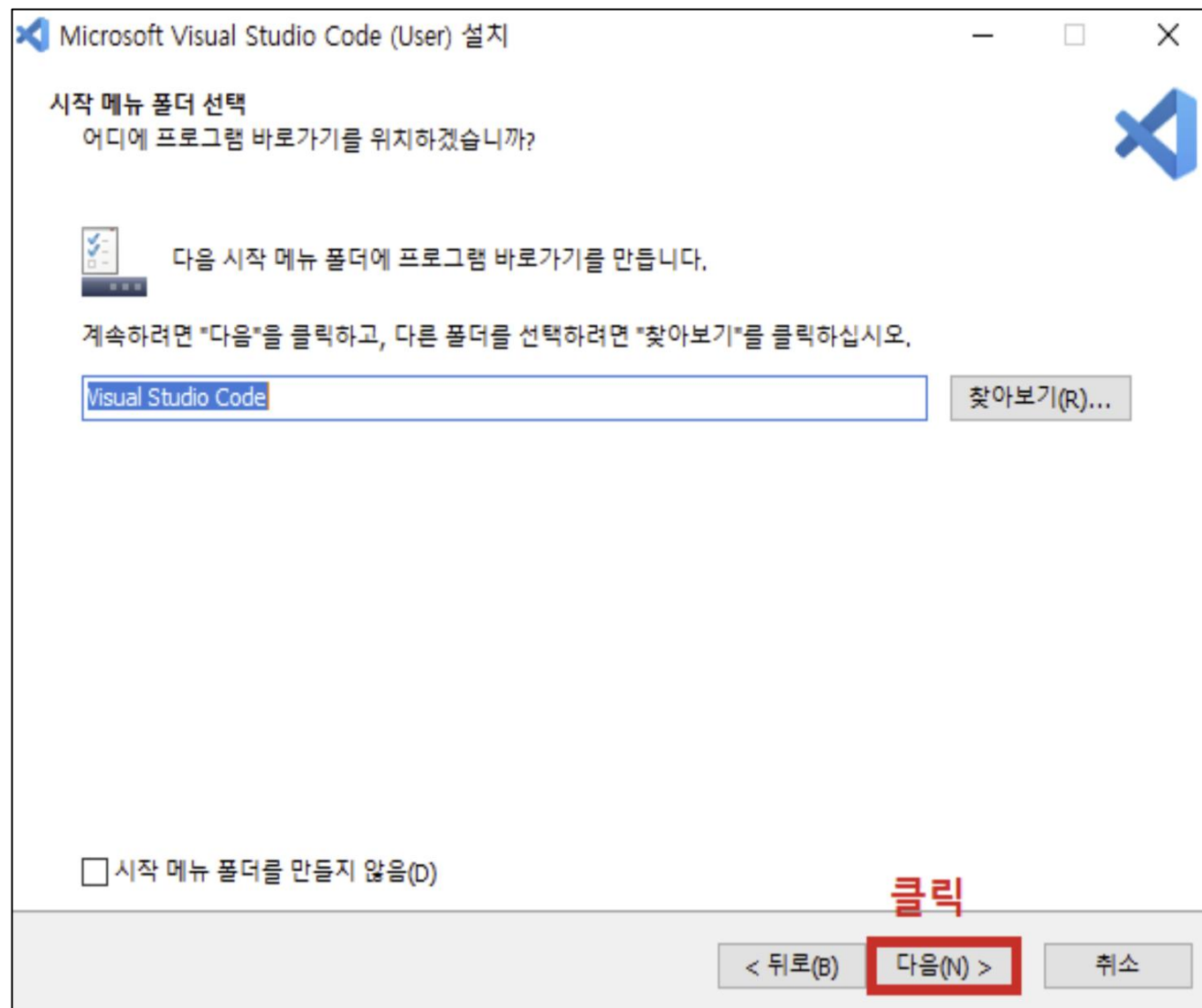
CLI Intel chip Apple silicon



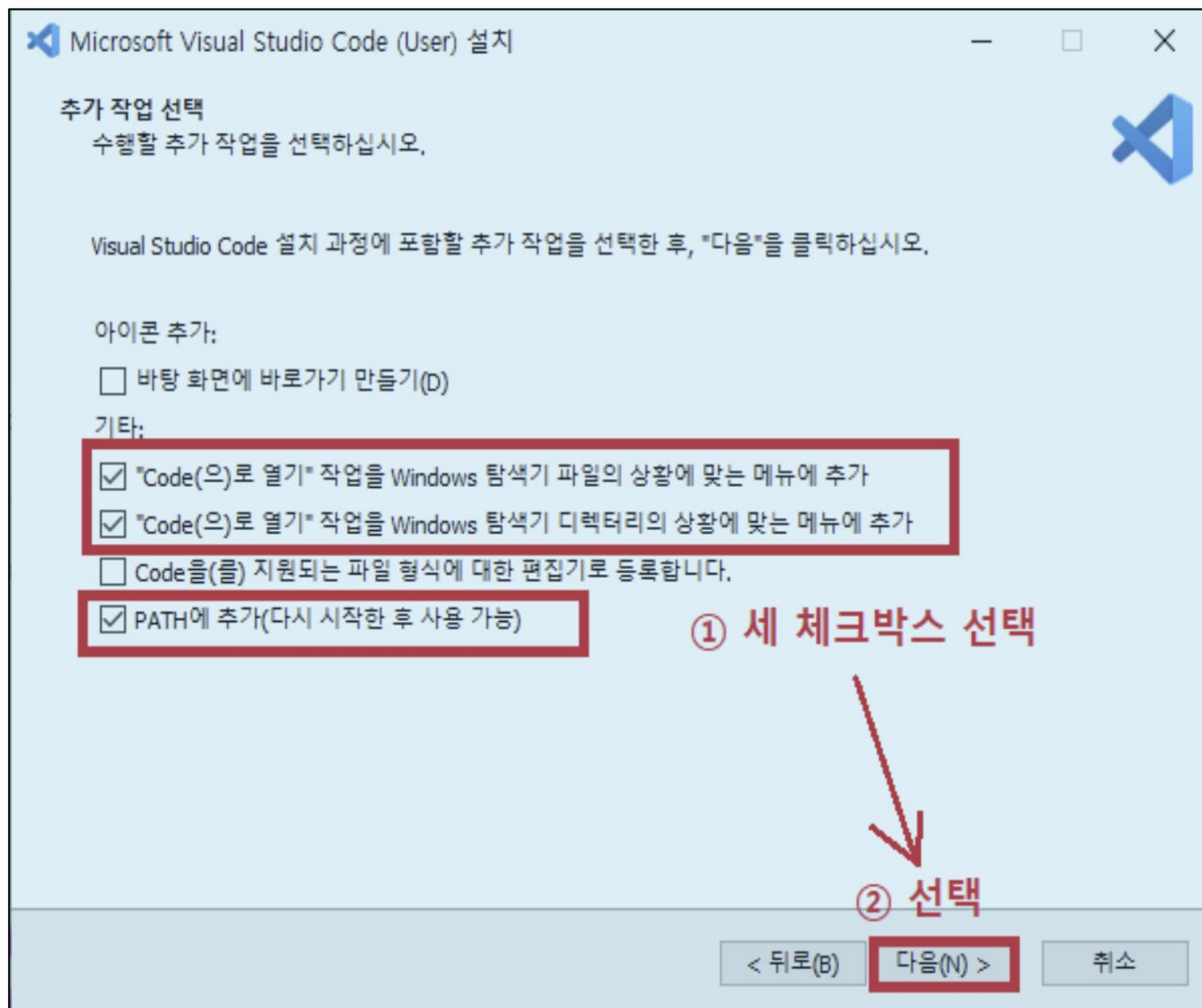
개발환경 설정



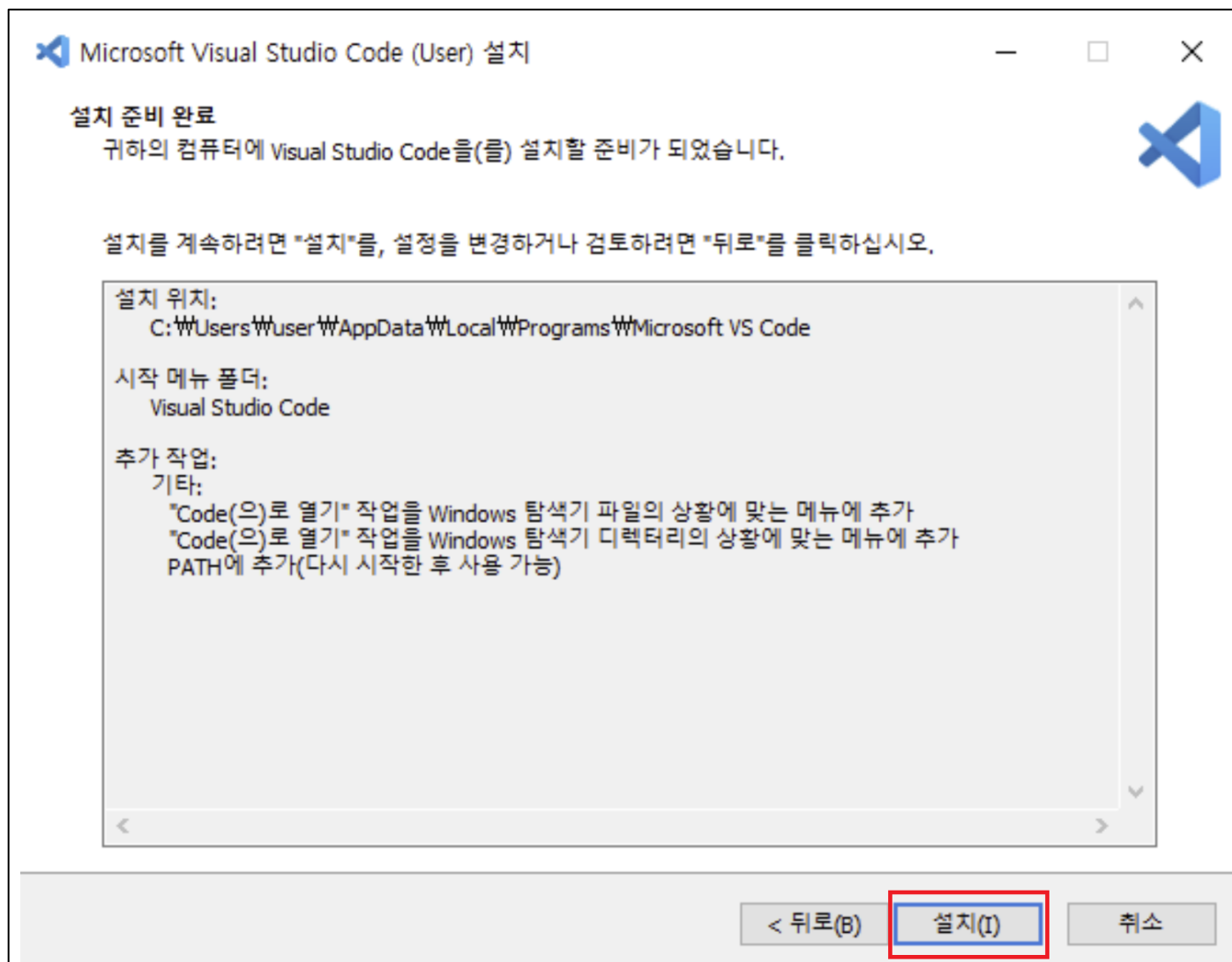
에디터 설치 & 익스텐션 설정

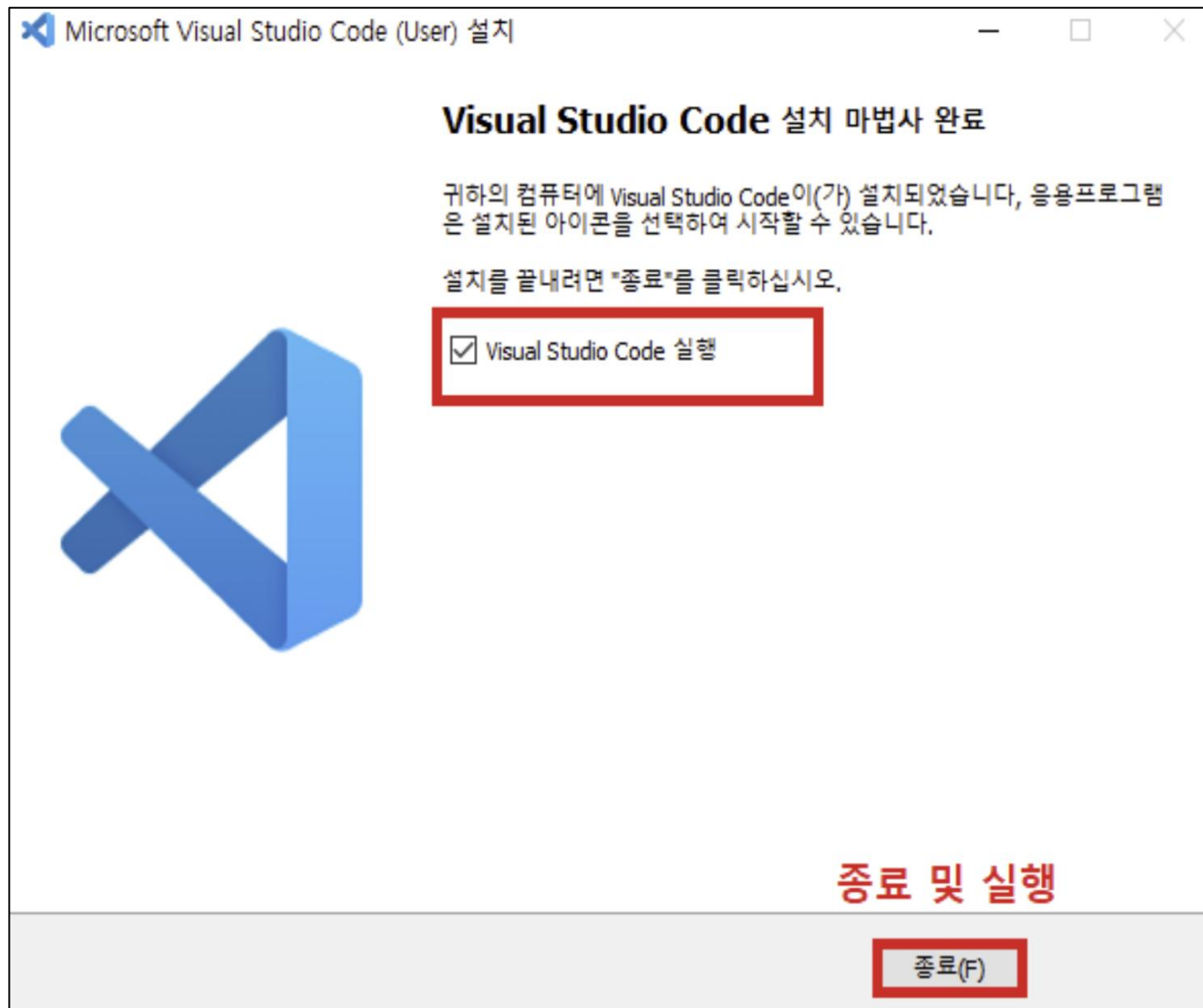


개발환경 설정

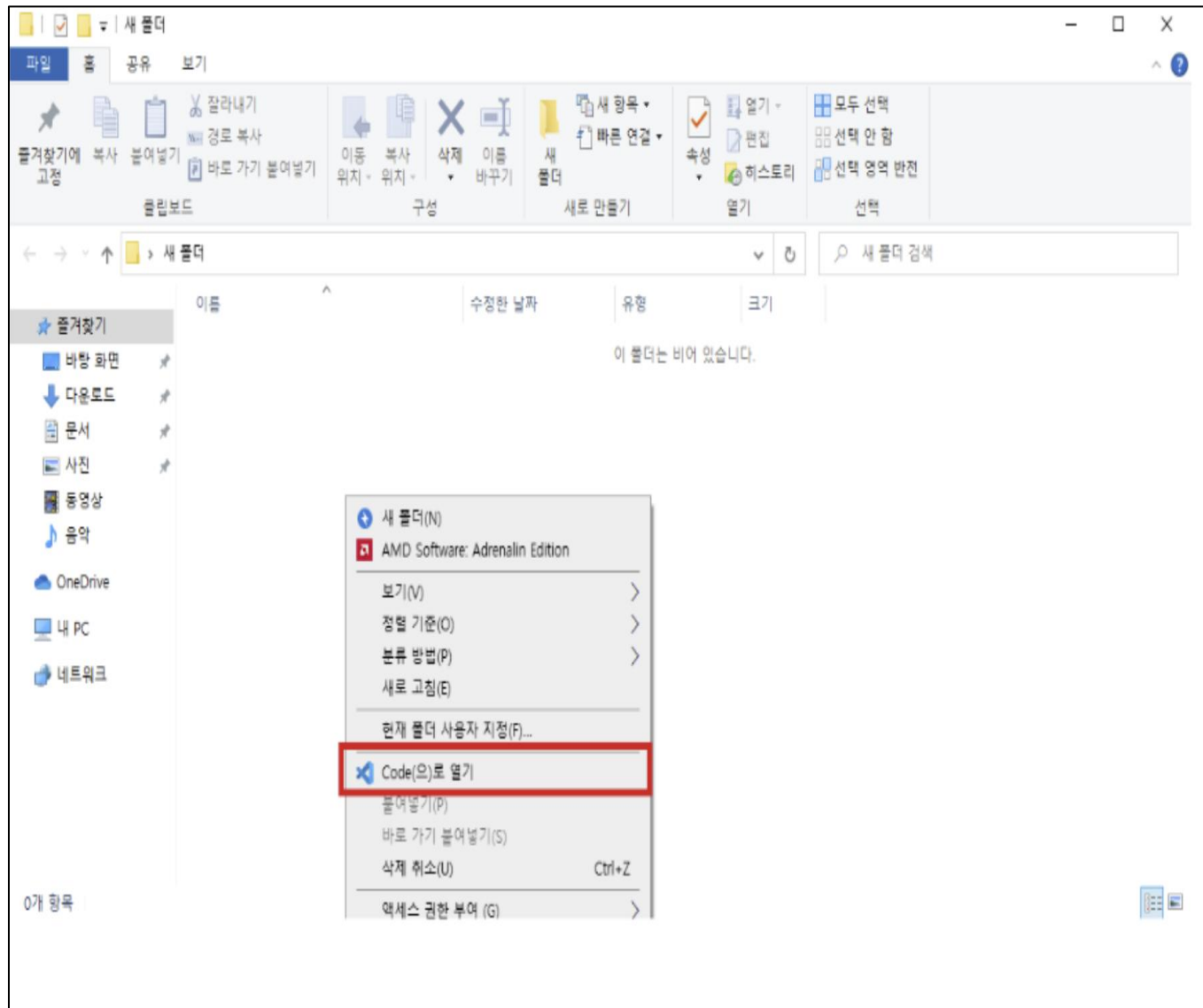


개발환경 설정





[실습] 설치 확인

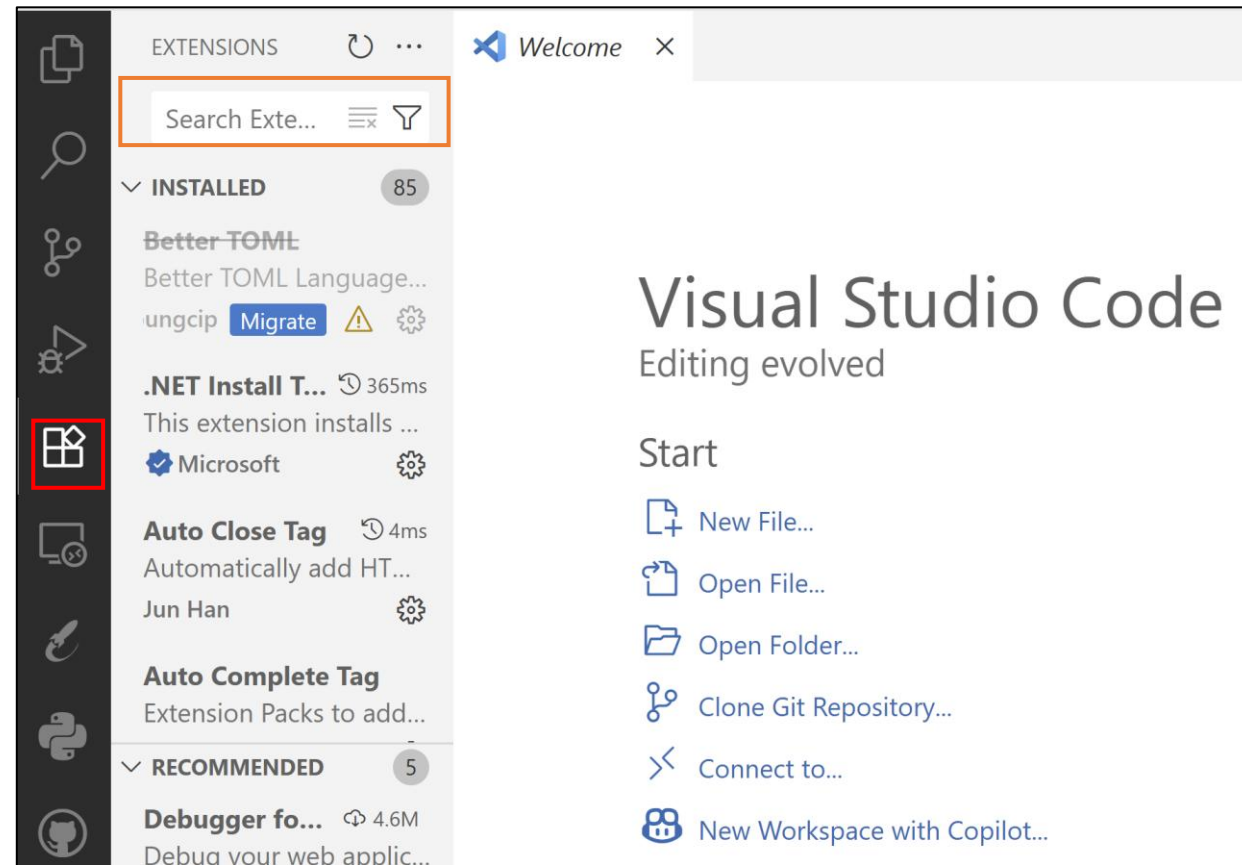


[참고] vscode 안전모드 설정

- ✓ 목적 : 악성코드 방지
- ✓ 엄격 모드
- ✓ 디버깅 사용불가
- ✓ 제약 조건 발생

확장프로그램 설치

- ✓ Auto Close Tag
- ✓ Live Server
- ✓ Auto Rename Tag
- ✓ Korean Language Pack for Visual Studio Code(선택)



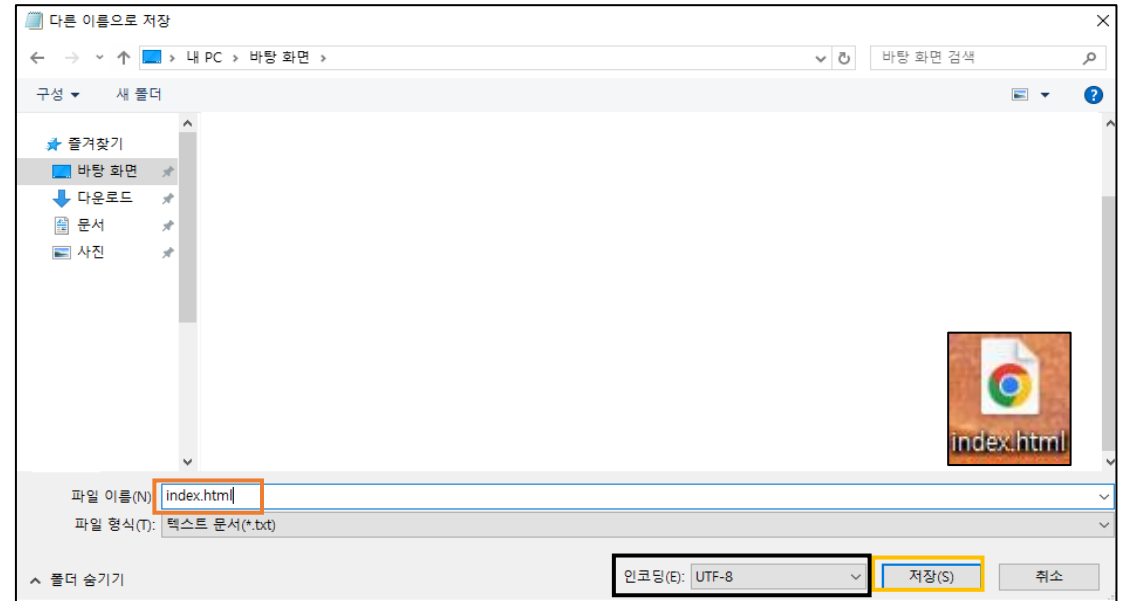
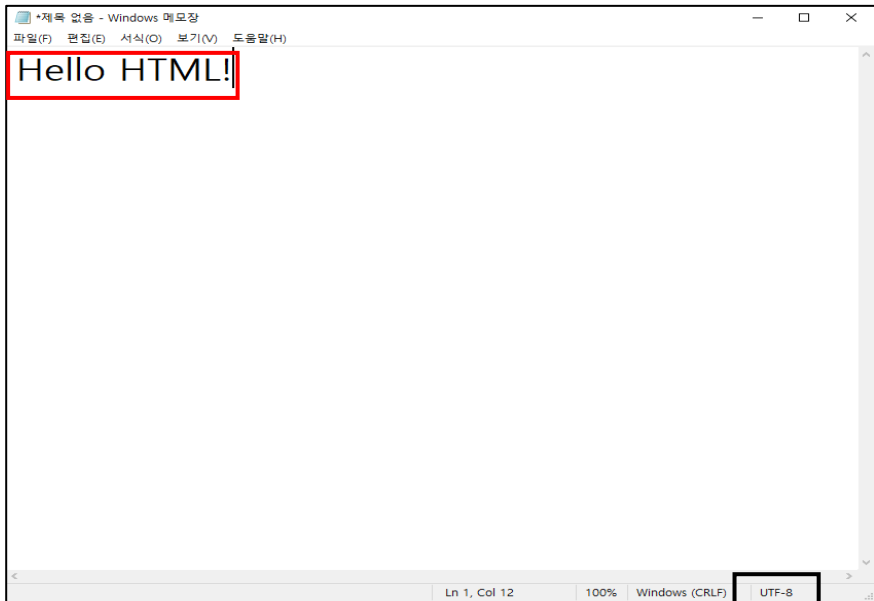
Chapter 02

HTML

Vue.js를 활용한 프론트 앤드 개발

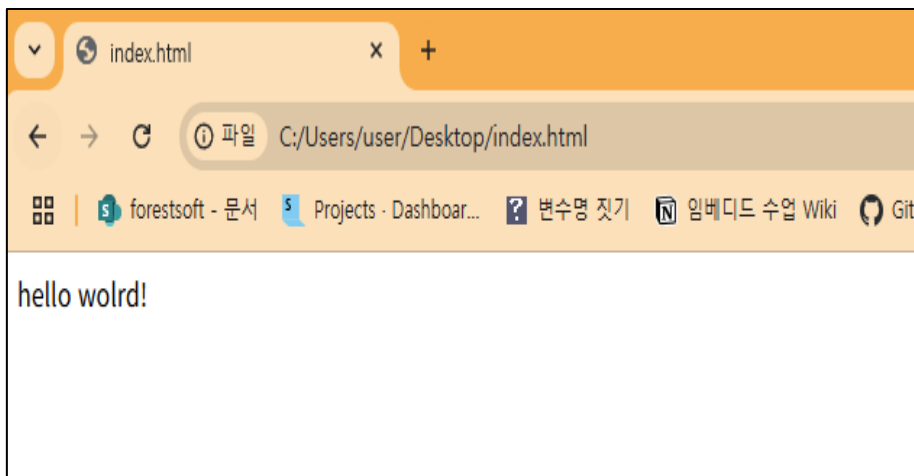
Hello HTML

✓ 메모장에 작성 후 저장하고 실행해보자.



세계 최초의 웹 페이지

✓ 첫번째 웹 페이지를 제작해보자.



http://info.cern.ch - home of the first website

From here you can:

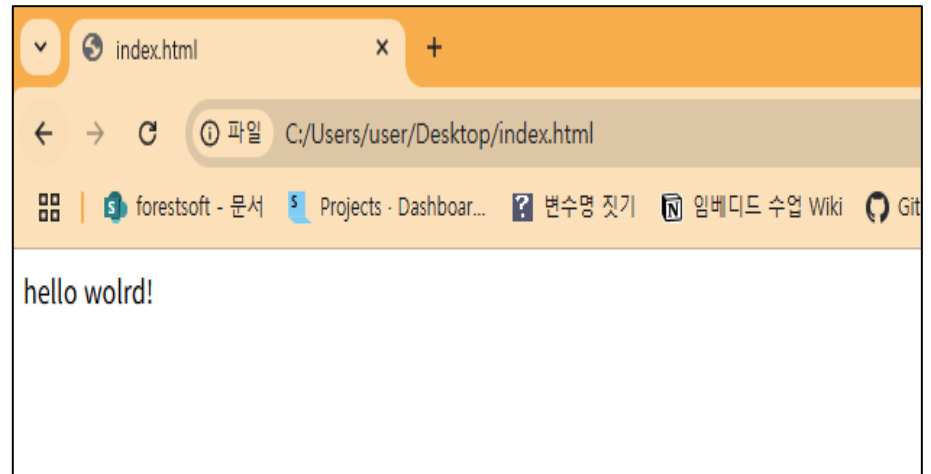
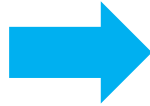
- [Browse the first website](#)
- [Browse the first website using the line-mode browser simulator](#)
- [Learn about the birth of the web](#)
- [Learn about CERN, the physics laboratory where the web was born](#)

코드 변경

✓ 코드를 변경하고, 실행해보자.

 index.html

1 hello world!

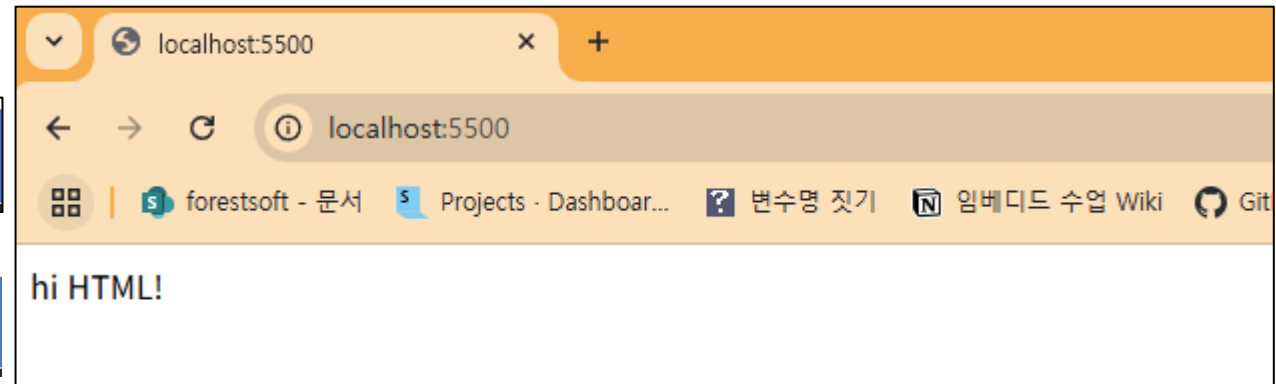


Live server 역할

✓ 저장하면, 자동 반영

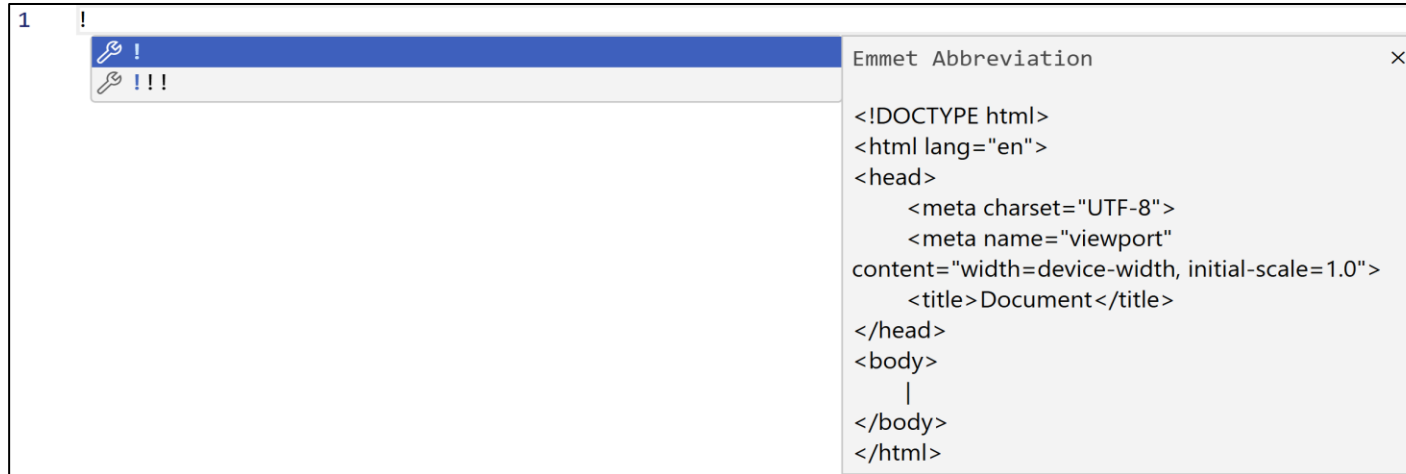
Ln 1, Col 12 Spaces: 4 UTF-8 CRLF {} HTML  Go Live  tabnine basic  Prettier 

Ln 1, Col 13 Spaces: 4 UTF-8 CRLF {} HTML  Port : 5500  tabnine basic  Prettier 



[실습] 기본 코드 사용

✓ VSCODE에 ! 또는 html:5 입력



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  |
</body>
</html>
```

[참고] HTML 기본 구조와 문서 선언

✓ <!DOCTYPE html>

<!DOCTYPE html>: HTML5 문서임을 선언

<html lang="ko"></html>: 문서의 루트, 언어 속성

<head>와 <body>: 문서 정보와 실제 콘텐츠 영역 구분

<meta>, <title>, <link>, <script>

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  |
</body>
</html>
```

HTML 소개

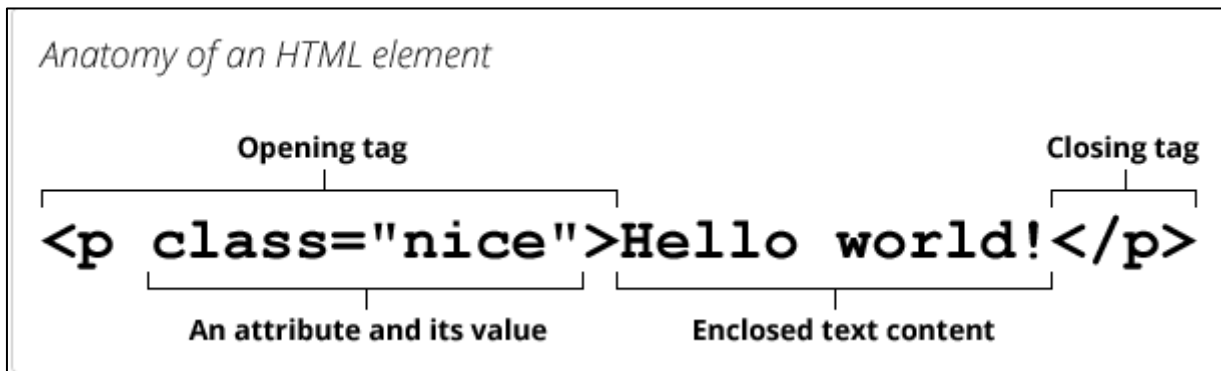
✓ Hyper Text Markup Language

HTML이란?

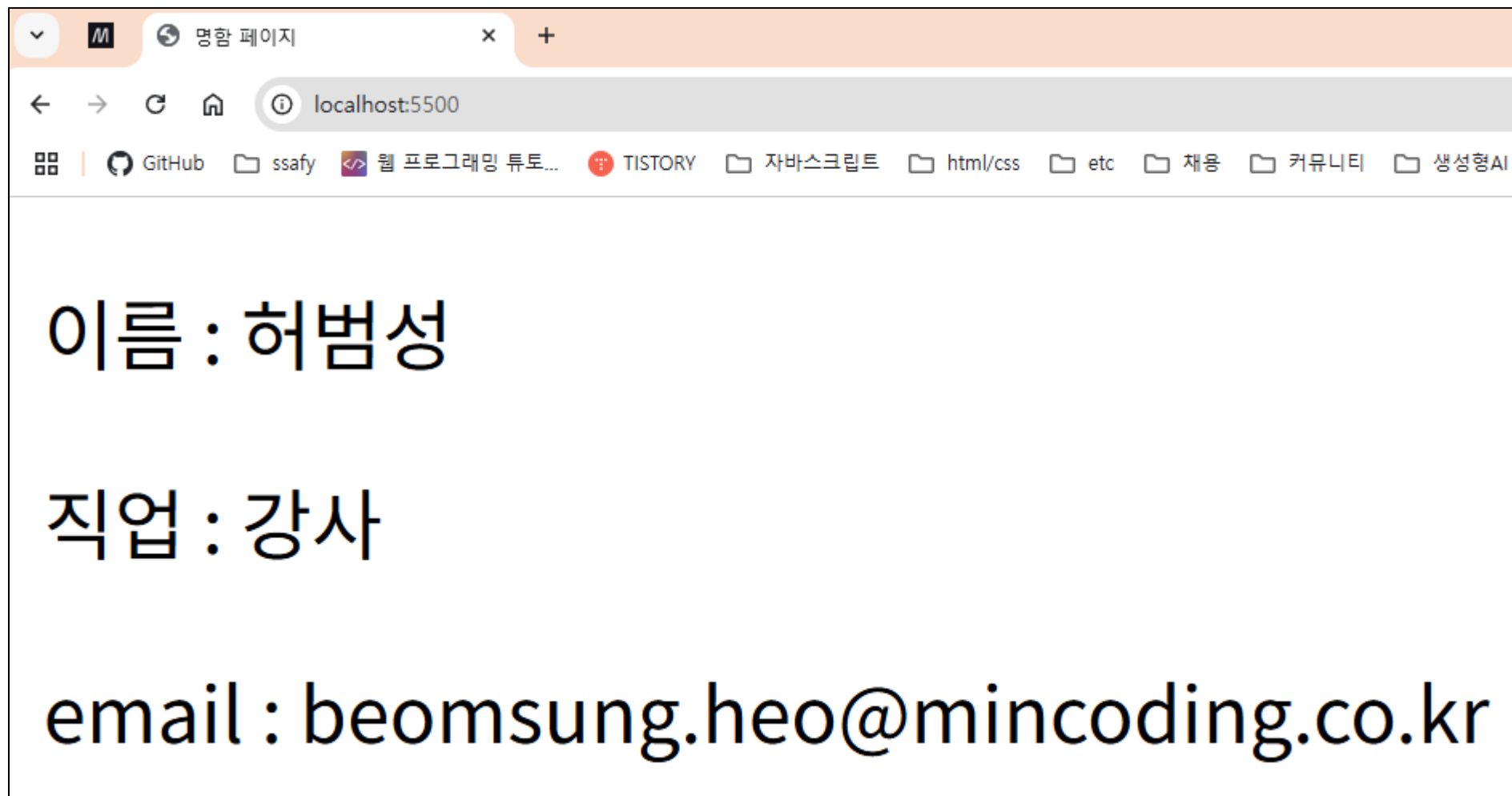
[HTML](#) (Hypertext Markup Language, 하이퍼텍스트 마크업 언어)는 프로그래밍 언어는 아니고, 우리가 보는 웹페이지가 어떻게 구조화되어 있는지 브라우저로 하여금 알 수 있도록 하는 마크업 언어입니다. 이는 개발자로 하여금 복잡하게도 간단하게도 프로그래밍 할 수 있습니다. HTML은 [elements](#)로 구성되어 있으며, 이들은 적절한 방법으로 나타내고 실행하기 위해 각 콘텐츠의 여러 부분들을 감싸고 마크업 합니다. [tags](#) 는 웹 상의 다른 페이지로 이동하게 하는 하이퍼링크 내용들을 생성하거나, 단어를 강조하는 등의 역할을 합니다. 예를들어, 다음의 내용을 봅시다.

HTML 소개

- ✓ 여는 태그
- ✓ 닫는 태그
- ✓ 내용
- ✓ 요소



[도전] 명함 만들기



Chapter 03

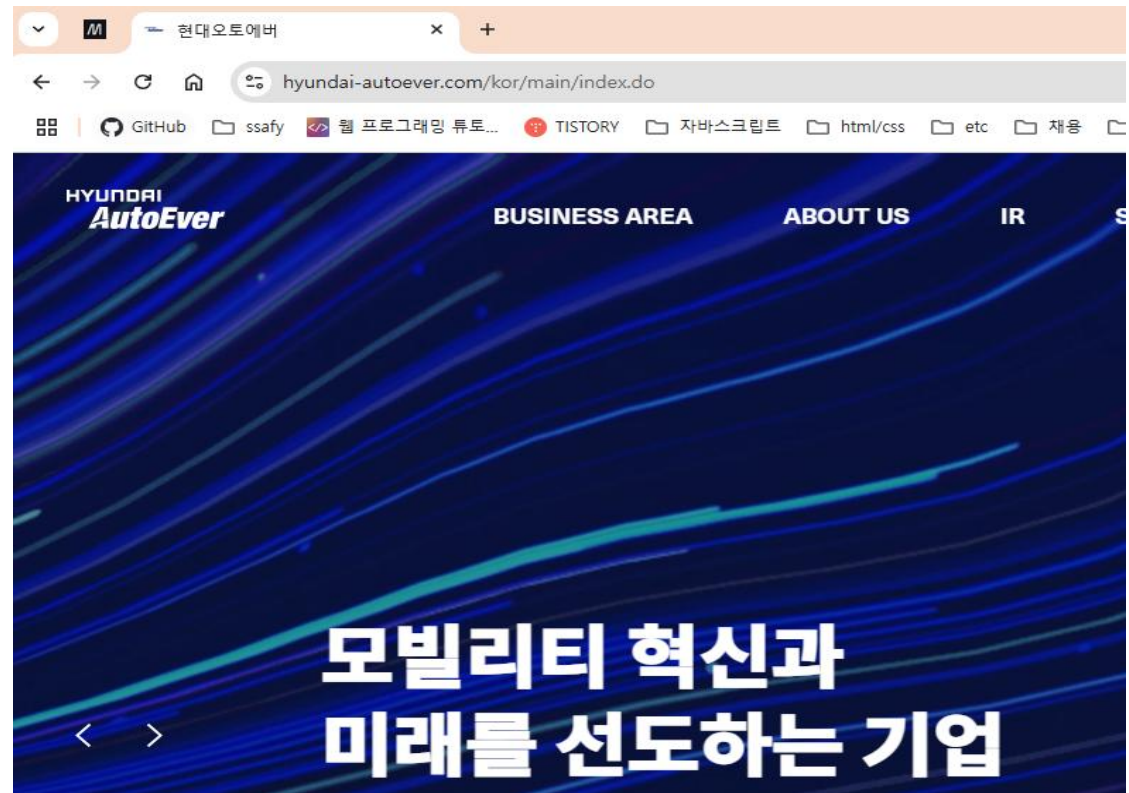
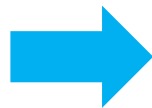
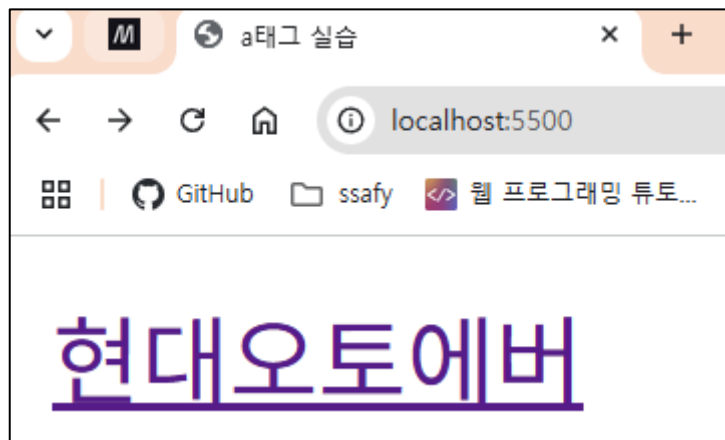
주요 element 실습

Vue.js를 활용한 프론트 앤드 개발

a태그 소개

- ✓ <a> 앵커태그
- ✓ href 속성 : 이동할 주소 작성(다른 페이지로 이동, 같은 페이지의 다른 위치 이동)
- ✓ Target : 링크 여는 창 / 탭 지정
- ✓ Title : 마우스 오버 시 툴팁 설명
- ✓ Download : 파일 다운로드 기능

[실습] a태그 사용



li태그 소개

- ✓ 리스트 태그
- ✓ 목록의 항목을 나타냄
- ✓ 반드시 정렬(ol), 비정렬(ul), 메뉴(menu)안에 위치해야함

[실습] li태그 사용

스마트팩토리 주요 기능

- 실시간 생산 모니터링
- 설비 자동화
- 데이터 분석 및 예측
- 품질 관리

스마트팩토리 점검 절차

- 설비 전원 확인
- 센서 상태 점검
- 시운전 실시
- 이상 유무 기록

스마트팩토리 대시보드 메뉴

<menu>

- 생산 현황
- 설비 관리
- 품질 분석
- 알람 내역

</menu>



스마트팩토리 주요 기능

- 실시간 생산 모니터링
- 설비 자동화
- 데이터 분석 및 예측
- 품질 관리

스마트팩토리 점검 절차

1. 설비 전원 확인
2. 센서 상태 점검
3. 시운전 실시
4. 이상 유무 기록

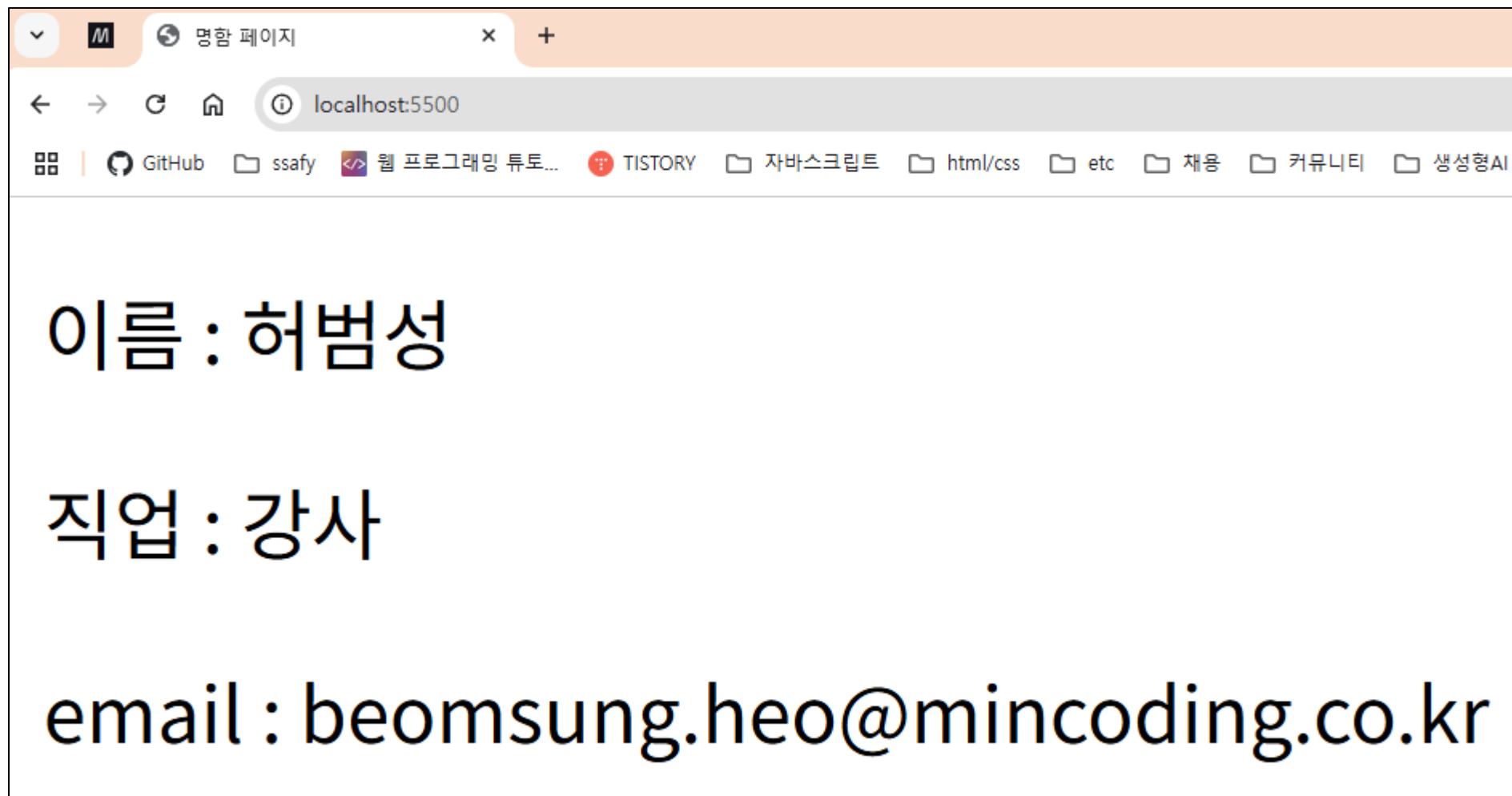
스마트팩토리 대시보드 메뉴

- 생산 현황
- 설비 관리
- 품질 분석
- 알람 내역

p태그 소개

- ✓ <p> 패러그래프 태그
- ✓ 하나의 문단을 나타냄
- ✓ 블록 레벨 요소

[리뷰] 명함 만들기



http://info.cern.ch - home of the first website

From here you can:

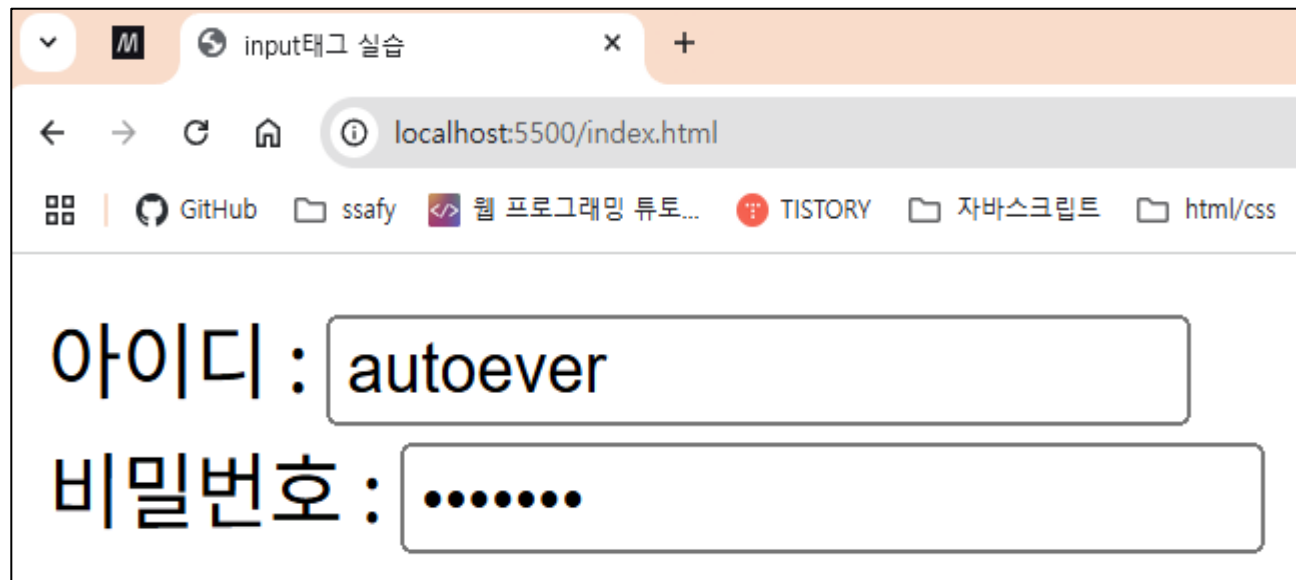
- [Browse the first website](#)
- [Browse the first website using the line-mode browser simulator](#)
- [Learn about the birth of the web](#)
- [Learn about CERN, the physics laboratory where the web was born](#)

input태그 소개

- ✓ <input> 인풋 태그
- ✓ 사용자로부터 데이터를 입력받음
- ✓ 다양한 유형의(type) 입력을 받을 수 있음
- ✓ Value 속성으로 기본값을 부여

[실습] input태그 사용

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>input태그 실습</title>
</head>
<body>
  아이디 : <input type="text" name="" id="">
  <br>
  비밀번호 : <input type="password" name="" id="">
</body>
</html>
```



[실습] input태그 사용

할 수 있는 프로그래밍 언어

c<input type="checkbox" name="" id="">

c++<input type="checkbox" name="" id="">

python<input type="checkbox" name="" id="">

java<input type="checkbox" name="" id="">

javascript<input type="checkbox" name="" id="">

<input type="submit" value="제출">

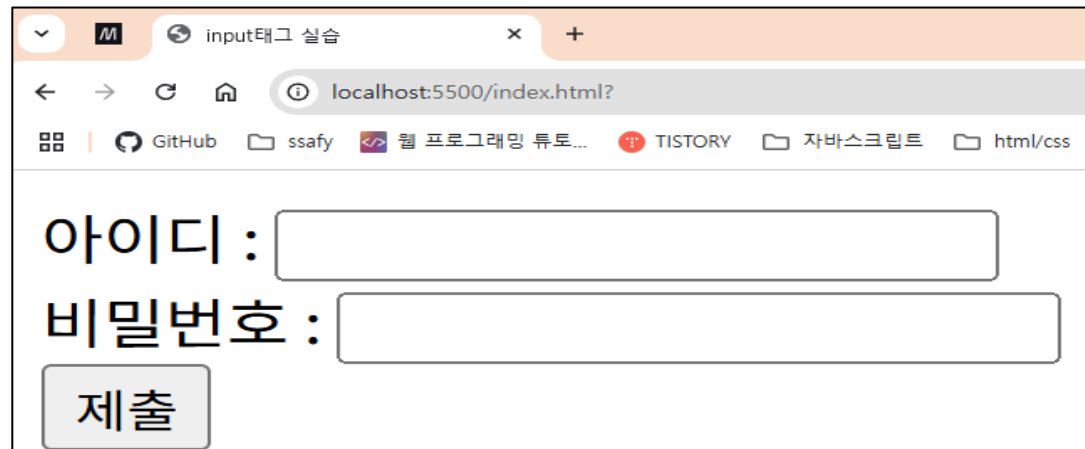


Form 태그 소개

- ✓ `<form>` form 태그
- ✓ 서버로 데이터를 전송하기 위한 문서 섹션

[실습] form태그 사용

```
<form action="">  
  아이디 : <input type="text" name="" id="">  
  <br>  
  비밀번호 : <input type="password" name="" id="">  
  <br>  
  <input type="submit" value="제출">  
</form>
```



[도전] 부품등록 양식 작성

- ✓ Form 태그 필수
- ✓ 비밀번호 암호화
- ✓ Radio 버튼은 한가지만 선택
- ✓ select는 2개이상 작성하기

부품 등록 폼

부품명:

시리얼 번호:

관리자 비밀번호:

부품 종류(한 가지만 선택)
센서 ☒ 모터 ☐ 디스플레이 ☐

제조사:

생산일:

색상:

매뉴얼 파일 업로드: 선택된 파일 없음

비고:

[참고] 추가 학습 자료

- ✓ Semantic web(시멘틱 웹)
- ✓ 요소 비교 (inline vs block vs inline-block)

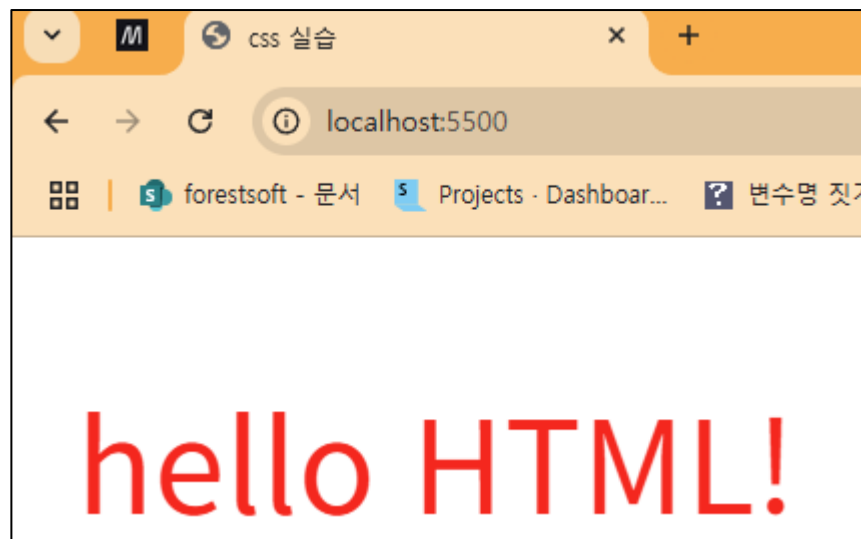
Chapter 04

CSS

Vue.js를 활용한 프론트 앤드 개발

css의 역할과 속성(Property) 적용

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>css 실습</title>
  <style>
    p {
      color: red;
    }
  </style>
</head>
<body>
  <p>
    hello HTML!
  </p>
</body>
</html>
```



[실습]css 적용

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    p{
      color: ■ red;
    }
  </style>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <p>hello HTML!</p>
  <p>hello HTML!</p>
  <p style="color: ■ yellow;">hello HTML!</p>
</body>
</html>
```

Selector

```
p {  
  color: red;  
}
```

Property Property value

Declaration

The diagram illustrates the components of a CSS rule. The selector 'p' is highlighted in yellow and labeled 'Selector' with a white arrow. The opening curly brace '{' is white. The property 'color' is highlighted in yellow and labeled 'Property' with a white bracket. The value 'red' is highlighted in white and labeled 'Property value' with a white bracket. The closing curly brace '}' is white. A large white bracket at the bottom spans the entire rule body and is labeled 'Declaration'.

선택자 종류

- ✓ Type selectors
- ✓ Class selectors
- ✓ ID selectors

Chapter 05

Flex box로 레이아웃 구성하기

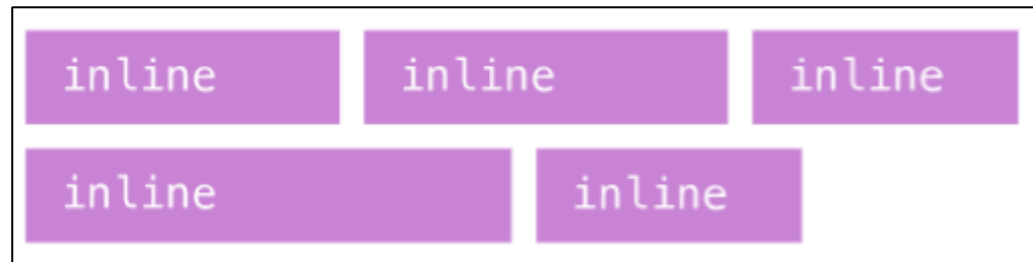
Vue.js를 활용한 프론트 앤드 개발

Layout - display property

- ✓ Inline
- ✓ Block
- ✓ Inline-block
- ✓ Flex
- ✓ Grid

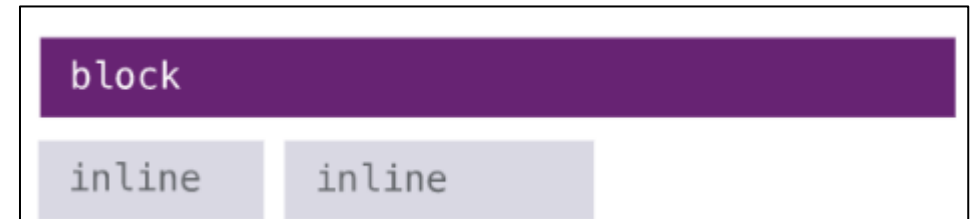
Display inline

- ✓ 줄 바꿈 없음
- ✓ 콘텐츠의 크기만큼 너비 차지(너비 조정불가)
- ✓ 종류 : a,span



Display block

- ✓ 줄 바꿈 있음
- ✓ 콘텐츠 크기를 포함하여 한줄을 차지한다(너비 조정가능)
- ✓ 종류 : p,ul,ol,form,h1~h6,div



Display inline-block

- ✓ 줄 바꿈 없음
- ✓ 콘텐츠의 크기만큼 너비 차지(너비 조정가능)
- ✓ 종류 : input,button,img,select,textarea



Display flex

- ✓ 메인 축을 기반으로 정렬
- ✓ 한개의 Flex-container와 flex-item들로 구성
- ✓ 부모에 flex 속성을 선언하면 부모는 flex-container
- ✓ 자식 요소들은 자동으로 flex-item

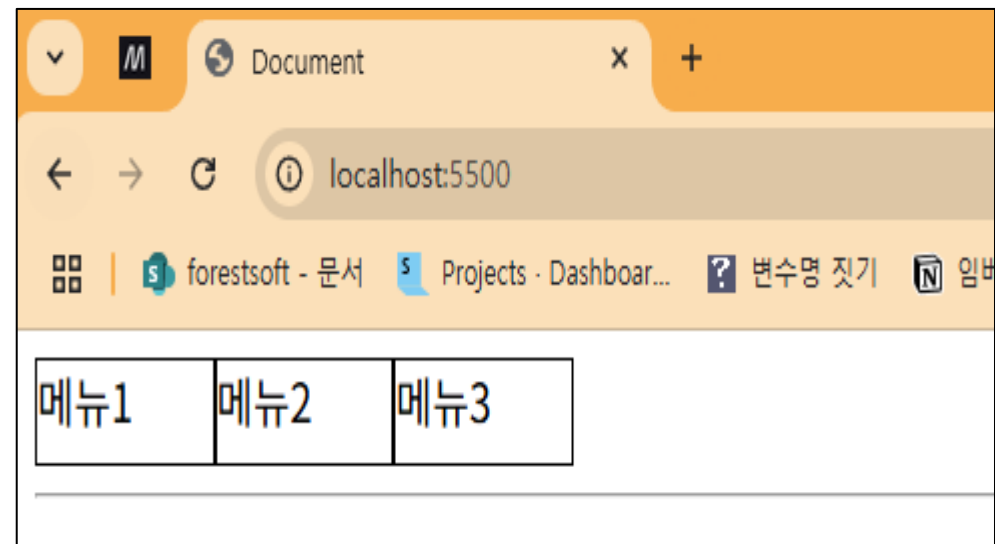
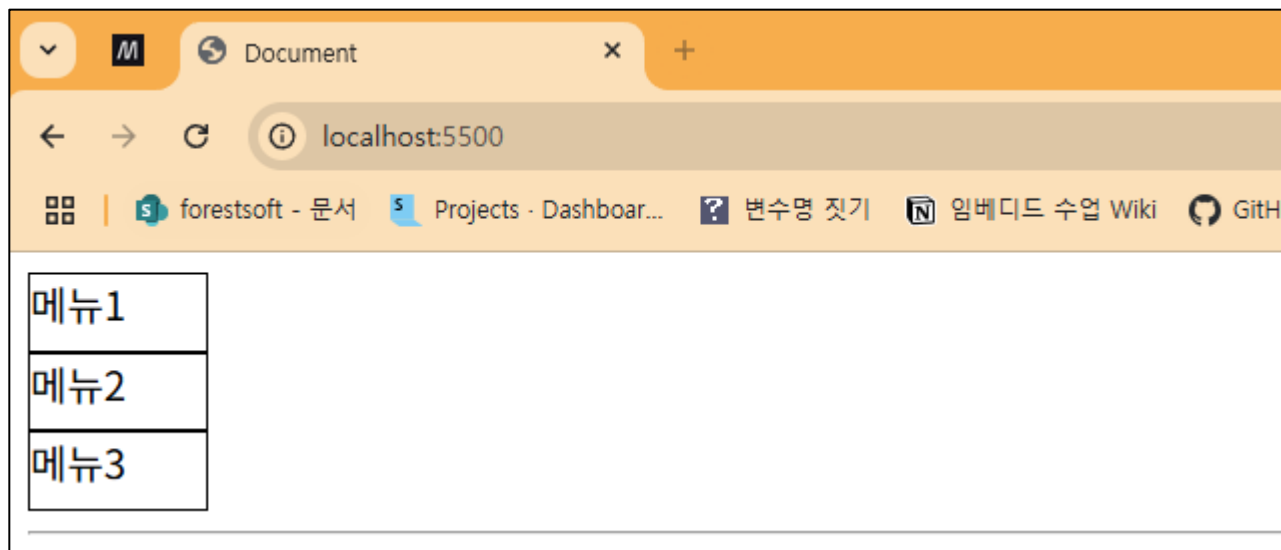
```
<div class="container">  
  <div class="item"></div>  
  <div class="item"></div>  
  <div class="item"></div>  
</div>
```

1
2
3

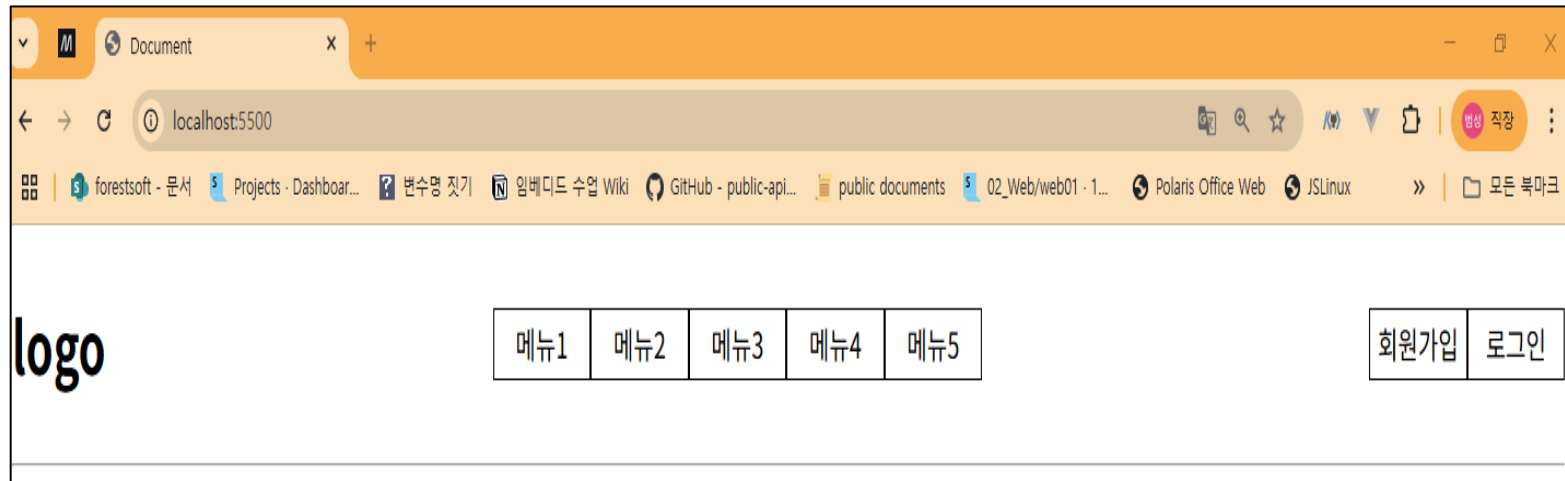
```
.container {  
  display: flex;  
}
```

123

[도전] flex 활용



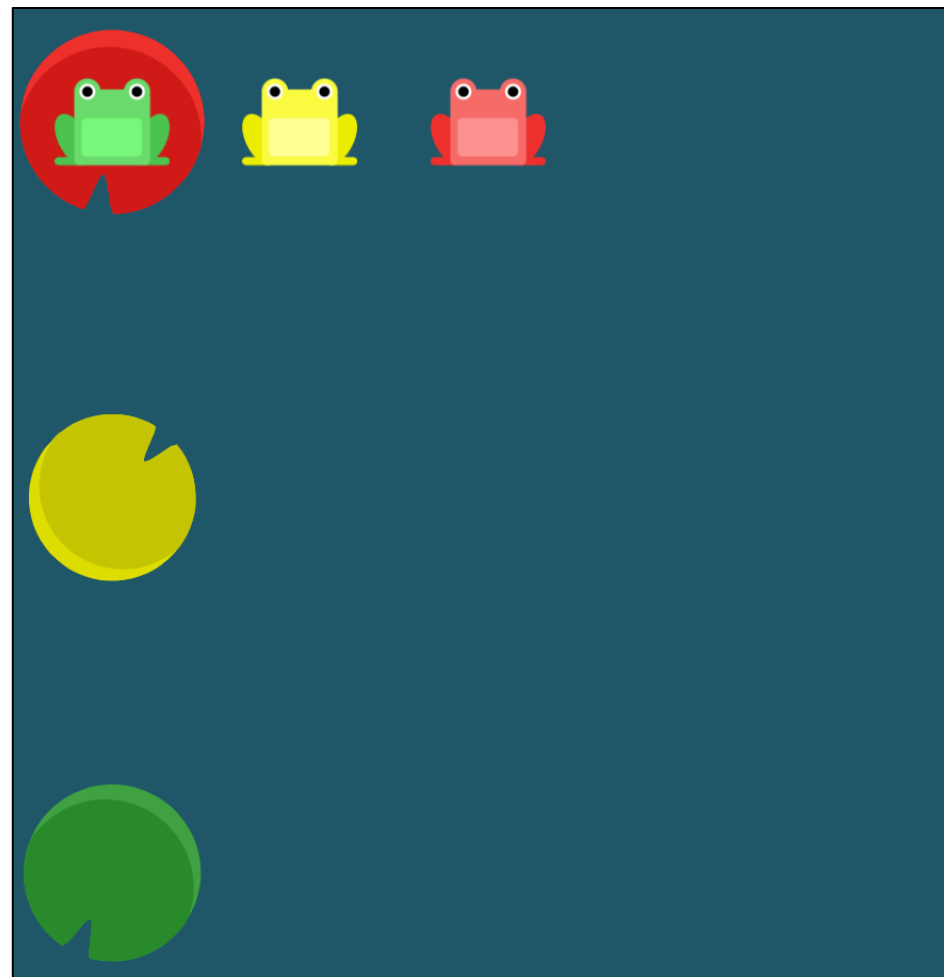
[도전] flex 활용



[참고] flex 개구리 게임

<https://flexboxfroggy.com/#ko>

- flex를 활용한 개구리 배치 게임 사이트



Chapter 07

폼 스타일링

Vue.js를 활용한 프론트 앤드 개발

폼 스타일링

- ✓ 사용자와 상호작용하는 사이트는 폼을 보유하고 있다.
- ✓ 폼에 효과를 적용하여 UX/UI를 향상시키자

부품 등록 폼

부품명

예: 센서

시리얼 번호

예: 1234

제조사

현대오토에버



비고

특이사항 입력

등록

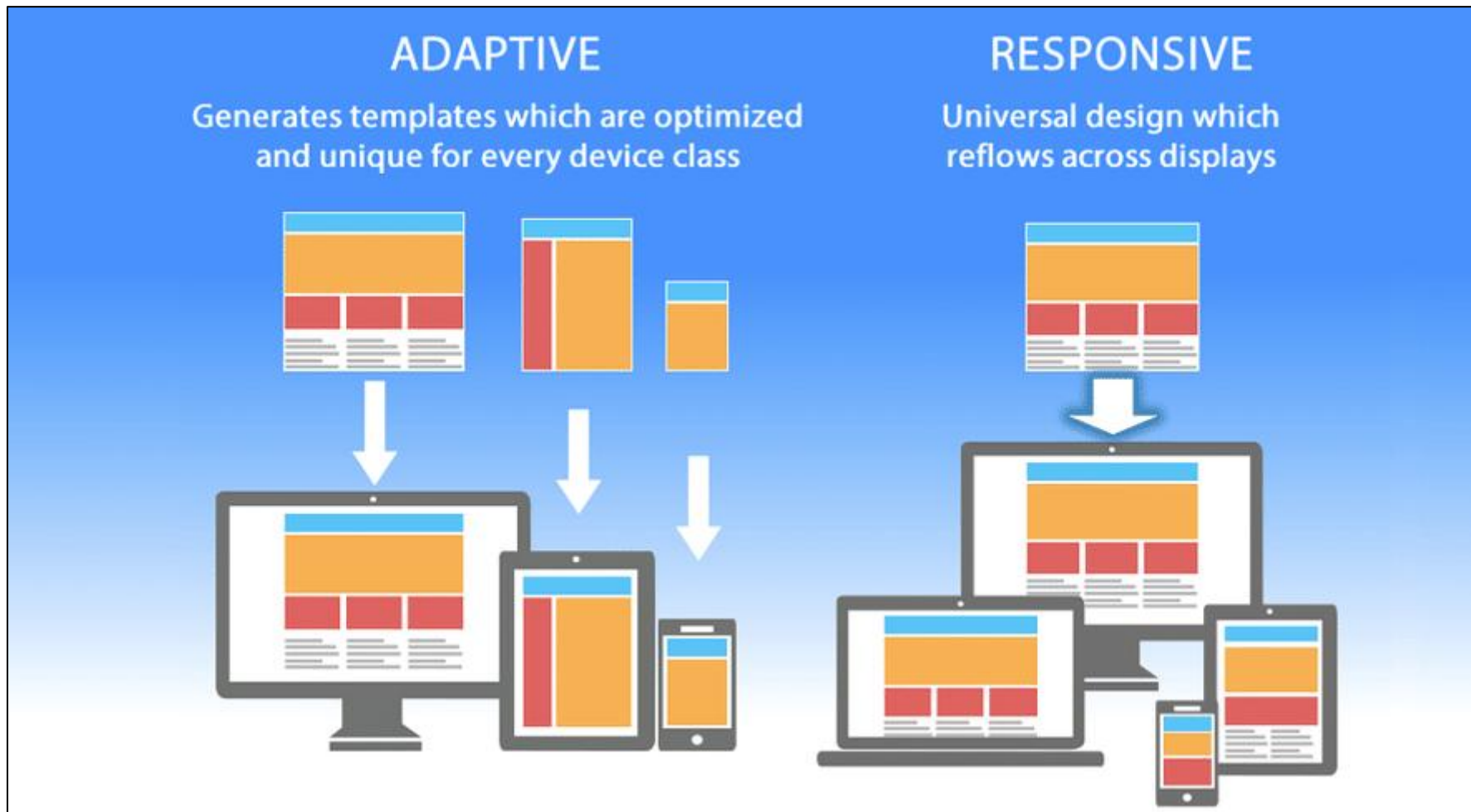
초기화

Chapter 07

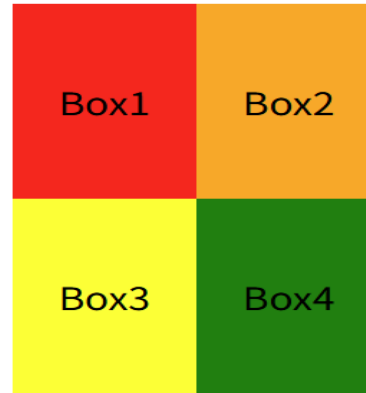
반응형 웹

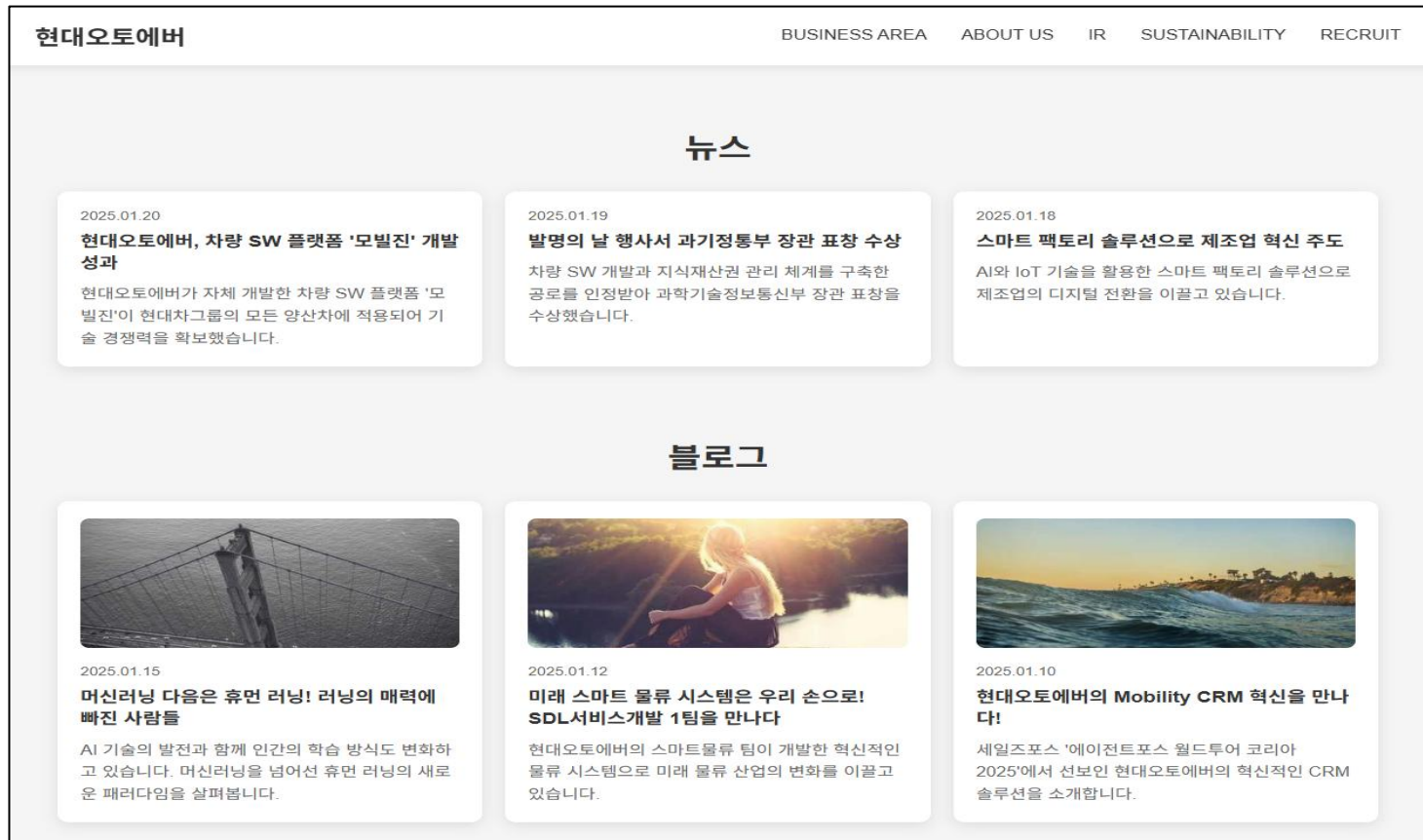
Vue.js를 활용한 프론트 앤드 개발

반응형 웹이란?









Chapter 8

자바스크립트 소개

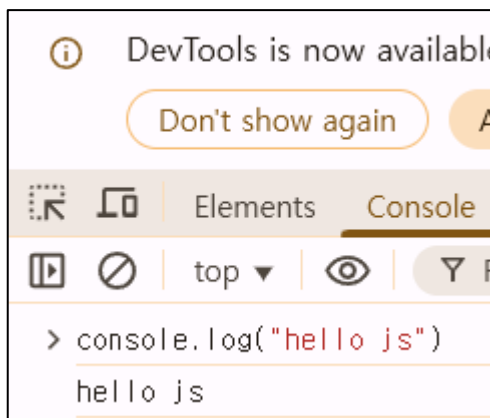
Vue.js를 활용한 프론트 앤드 개발

자바스크립트 사용이유?

- ✓ 웹 페이지에 동적인 작업을 할 수 있다
- ✓ 비동기 통신(ajax)
- ✓ 서버 구축가능(node.js)

Hello js

- ✓ 브라우저 콘솔 사용
- ✓ script 태그 사용



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>
    console.log('hello js!');
  </script>
</body>
</html>
```

Chapter 9

변수와 자료형

Vue.js를 활용한 프론트 앤드 개발

변수를 가장 먼저 배우는 이유

- ✓ 프로그램
 - ✓ 순서도
 - ✓ 순서도를 작성할때, 데이터가 중복 포함된경우(0,-1,1 제외)
 - ✓ 숫자, 문자, 결과 출력 등 과정에 데이터가 관여
- ✓ 데이터를 저장(자동으로 진행)하고, 불러오는 방법이 필요
 - ✓ 이름, 나이, 결과값 등을 **어딘가**에 저장됨, 불러오려면?
 - ✓ 변수 = 이 저장 공간에 이름을 붙인 것

변수란?

- ✓ 데이터(한값)를 저장하기 위한 메모리 공간에 붙인 이름
 - ✓ 숫자(정수)나 문자, 소수점 등을 저장할 수 있다.
- ✓ 변수(메모리)의 종류 예시



숫자(정수)가 들어가는
메모리 공간



글자가 들어가는
메모리 공간



소수점 숫자가 들어가는
메모리 공간

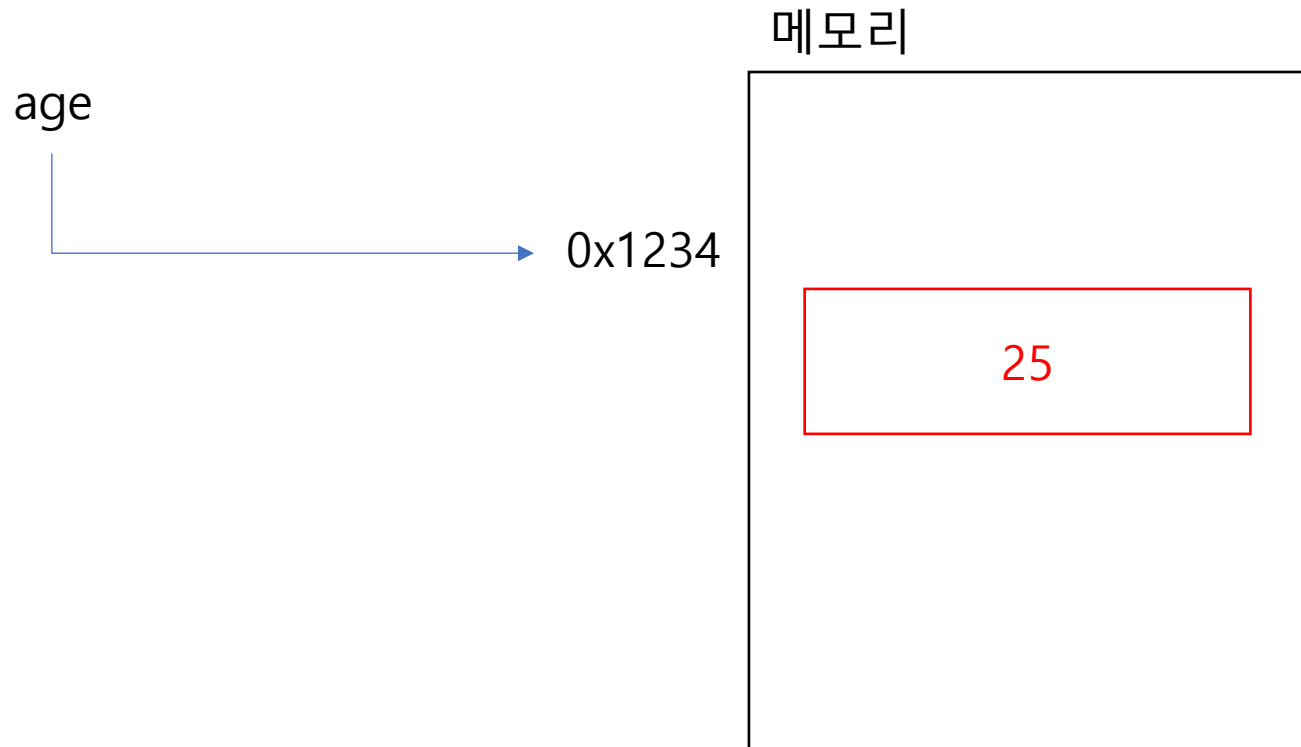
변수명 작명 규칙

- ✓ 영문자(Aa ~ Zz), 숫자(0~9), _ \$ 사용 가능
- ✓ 여러 단어 조합 시 카멜케이스(CamelCase) 사용 권장
 - ✓ 예시) username, totalScore, isLoggedIn
- ✓ 주의 사항
 - ✓ 숫자로 시작 불가 (예 : 1stNumber X)
 - ✓ 공백 사용 불가 (예 : user name X)
 - ✓ 예약어 사용 금지 (예 : const, class, function 등 X)
 - ✓ 대소문자 구분함 (예 : count != Count)

변수가 메모리에 저장되는 과정

- ✓ 변수는 값을 저장하거나 나중에 다시 사용하기 위해 필요
 - ✓ 입력값, 계산 결과, 출력 등 모든 흐름에 관여
- ✓ 메모리 구조 예시

Ex) `const age = 25`



변수 선언 방법

✓ 변수 선언 키워드 var, let, const

```
var age = 25;  
let name = "heobeomsung";  
const company = "mincoding";
```

변수 선언 키워드 비교

✓ 변수 선언 키워드 var, let, const

var

재선언, 재할당 가능 / 함수 레벨 스코프

let

재선언 불가 재할당 가능 / 블록 레벨 스코프

const

재선언 불가 재할당 불가 / 블록 레벨 스코프

```
var a = 1;
var a = 2;

var a = 3;
a = 4;

let b = 5;
let b = 6;

let c = 7;
c = 8;

const d = 9;
const d = 10;

const e = 11;
e = 12; //런타임 에러
```

용어 정리

- ✓(재)선언
- ✓(재)할당
- ✓스코프

참고) 함수 레벨 스코프 vs 블록 레벨 스코프

```
> function hello(){  
  var a = 5;  
}  
hello();  
console.log(a);
```

✖ ▶ Uncaught ReferenceError: a is not defined
at <anonymous>:5:13

VM182:5

```
{  
  var a = 5;  
}  
console.log(a)
```

5

VM287:4

```
> function hello(){  
  const a = 5;  
}  
hello();  
console.log(a);
```

✖ ▶ Uncaught ReferenceError: a is not defined
at <anonymous>:5:13

```
> {  
  const a = 10;  
}  
console.log(a)
```

✖ ▶ Uncaught ReferenceError: a is not defined
at <anonymous>:4:13

VM472:4

자바스크립트 자료형

✓ 자바스크립트 2가지 데이터 타입

원시자료형 Number / String / Boolean / Null / Undefined

참조 자료형(객체) Array / Object / Function

(문제) 다음 코드의 결과 예상하기

✓ 키워드 : 호이스팅

```
console.log(a)  
var a = 1
```

[도전] 메모리 그리기

- ✓ `var a = 1` 을 선언하였을때, 메모리에 적재되는 과정 작성하기

Chapter 10

배열과 객체

Vue.js를 활용한 프론트 앤드 개발

- ✓ 회사 직원들을 관리하기 위해 모두 저장하려고 한다

```
const name1 = "김대리"  
const name2 = "이과장"  
const name3 = "박부장"  
const name4 = "금차장"
```

- ✓ 팀원이 10명 이상이라면...?
 - ✓ 변수를 계속 만들어주어야 한다.
 - ✓ 반복적인 데이터를 다룰 때 다른 저장 방법이 필요하다.

배열

✓ 배열이란 ?

- ✓ 같은 자료형의 값들을 하나의 변수 이름으로 묶어서 저장
- ✓ 각 변수는 인덱스(index)로 구분한다.

✓ 인덱스

- ✓ 배열 안에 저장된 값 하나하나를 구분하기 위한 번호
- ✓ 배열의 인덱스는 0부터 시작

index	0	1	2
값	김대리	이과장	박부장

✓ 방법1. 선언 + 할당

```
const names = [];
```

✓ 방법2. 선언 + 초기값 대입

```
const names = ["김대리", "이과장", "박부장"];
```

index	0	1	2
값	김대리	이과장	박부장

배열

- ✓ 배열 관련 메서드 학습
- ✓ 배열 고차함수

[도전] 배열 문제

- ✓ [3,0,2,9,3,9,3]을 하드코딩 하고, 3이 몇개인지 카운팅
- ✓ ['A','U','T','O','E','V','E','R']를 하드코딩하고, 'A'가 존재하는지 검사하여 출력

객체

✓ 회사 직원들의 이름과 나이를 저장하고자 한다

```
const name1 = "김대리"  
const age1 = 31
```

```
const name2 = "이과장"  
const age2 = 38
```

```
const name3 = "박부장"  
const age3 = 45
```

```
const names = ["김대리", "이과장", "박부장"]  
const ages = [31, 38, 45]
```

객체 리터럴

```
const person1 = {  
  name : '김대리',  
  age : 31,  
}  
  
const person2 = {  
  name : '이과장',  
  age : 38,  
}  
  
const person3 = {  
  name : "박부장",  
  age : 45,  
}  
  
const persons = [person1, person2, person3]
```

객체 속성 접근/수정

```
const person = {  
  name : '김대리',  
  age : 31,  
}  
  
console.log(person.name)  
console.log(person.age);  
console.log(person['age']);  
person.age = 32  
console.log(person.age);  
person.job = '개발자'  
console.log(person)
```


객체 문법 : 구조 분해 할당(Destructuring)

```
const person = {  
  name : '김대리',  
  age : 31,  
}  
  
// const name = person.name;  
// const age = person.age;  
  
const {name, age} = person;
```

객체 문법 : 스프레드 연산자(Spread)

```
const person = {  
  name : '김대리',  
  age : 31,  
}  
  
// 객체 복사 및 확장  
const newPerson = { ...person, job: '개발자' };  
console.log(newPerson); // { name: '김대리', age: 31, job: '개발자' }
```

객체 문법 : Rest 파라미터

```
const person = {  
  name: '김대리',  
  age: 31,  
  job : '개발자',  
};  
  
const { name, ...rest } = person;  
  
console.log(name); // '김대리'  
console.log(rest); // { age: 31, job: '개발자' }
```

객체 문법 문제

- ✓ users 배열에서 첫 번째 사용자의 이름만 firstName에 할당하고,
- ✓ 나머지 사용자들의 이름만 뽑아서 names 배열에 할당하는 코드 작성

```
const users = [  
  { name: "철수", age: 21 },  
  { name: "영희", age: 22 },  
  { name: "민수", age: 23 },  
];
```

Chapter 11

연산자 및 제어문

Vue.js를 활용한 프론트 앤드 개발

연산자

- ✓ 산술 연산자(+, -, *, /, %, **, ++, -- ...)
- ✓ 대입 연산자(=, +=, -=, *=, /=, %=, **=)
- ✓ 비교 연산자(==, !=, ==, !=, ==, >, <, >=, <=)
- ✓ 논리 연산자(&&, ||, !)
- ✓ 비트 연산자(&)
- ✓ 삼항 조건 연산자(조건식 ? 참일때 값 : 거짓일때 값)

논리 연산자

- ✓ 논리적 표현식을 비교하여, 하나의 참 또는 거짓의 결과를 반환
 - &&(AND 연산자)
 - 가장 먼저 평가되는 falsy 값을 반환, 없을 시 가장 마지막의 값 반환
 - ||(OR 연산자)
 - 가장 먼저 평가되는 truthy 값을 반환, 없을 시 가장 마지막의 값 반환
 - !(NOT)연산자
 - 피연산자의 논리값을 반전(단항 논리 연산자중 우선순위 가장 높음)

[실습]결과 예상

```
console.log(false && "hello" && 123);  
console.log("hi" && 0 && "world");  
console.log("a" && 1 && true && "끝");  
  
console.log(null || 0 || "" || "JS" || 42);  
console.log(false || undefined || 0);
```


[도전]결과 예상

```
console.log(0 && "A" && null && "B" && undefined && "C");  
console.log("X" && 1 && true && [] && {} && "마지막");  
  
console.log(null || 0 || "" || undefined || NaN || false);  
console.log(null || 0 || "" || "첫 truthy" || "두번째 truthy");
```

```
const isLogin = isValidId() && isValidPW()
```

조건문이란 ?

- ✓ 주어진 조건에 따라 코드 실행 흐름을 분기 시키는 문장
- ✓ "만약 ~~ 라면, ~~를 실행하라" 형태의 논리 구조
- ✓ 예시
 - ✓ 지각 할 것 같으면 택시를 탄다.
 - ✓ 회식 메뉴가 소고기라면 참석한다.

if 문 구조

✓ 조건문 형태

```
if (조건식) {  
    // 조건이 참일 때 실행될 코드  
} else {  
    // 위의 모든 조건이 거짓일 때 실행  
}
```

✓ 조건식은 true 또는 false 를 반환해야 함

if-else / else if

✓ 다양한 조건을 처리하는 방법

```
if (조건식1) {  
    // 조건식1 이 참일 때 실행될 코드  
} else if (조건식2) {  
    // 조건식1 이 거짓이고, 조건식2 가 참일 때 실행될 코드  
} else {  
    // 위의 모든 조건이 거짓일 때 실행  
}
```

다중 조건 처리 - 중첩 조건문

- ✓ if 문 안에 또 다른 if 문을 사용하는 구조
- ✓ 논리적 순서를 분리하거나, 1차 조건 만족 후 세부 조건 검토 시 사용

```
if (조건식1) {  
    // 조건식1이 참일 때 실행될 코드  
    if (조건식2) {  
        // 조건식1 과 조건식2가 모두 참일 때  
    }  
}
```

[도전]조건문

✓ 여행지를 선택하고, 나이와 회원 여부에 따라 추천 혜택을 받는다.

✓ 여행지 이름과 예약 수량, 회원 여부와 나이가 다음과 같이 저장되어 있다.

```
const destinations = [ "제주도", "강릉", "부산" ];  
const bookingCounts = [ "2", "0", "1" ];  
const isMember = true;  
const age = 25;
```

✓ 요구사항

✓ 각 여행지의 예약 현황을 출력한다. (예약 수량은 정수로 변환하여 활용)

✓ 1인당 여행 비용은 100,000원으로 고정되어 있다.

✓ 전체 여행 예약 수가 3건 이상이고, 회원이며, 나이가 20세 이상이면

✓ "여행 추천 혜택 대상" 출력

✓ 아니라면 "일반 고객" 출력

출력 결과

[예약 현황]

제주도: 2건

강릉: 0건

부산: 1건

총 금액: 300000원

여행 추천 혜택 대상

반복문

- ✓ 코드를 여러 번 실행하고 싶을 때 사용하는 중요한 문법
- ✓ 조건을 만족하는 동안 특정 코드를 반복 실행하는 구조

반복문 종류	기본 구조	사용 지점
for	횟수 기반	반복 횟수가 정해진 경우
while	조건 기반	반복 횟수를 모를 때

for 문

✓ 반복 횟수가 정해진 경우 활용

- ✓ 시작부터 조건이 만족할 때 까지 특정 횟수를 반복

✓ for문 기본 형태

```
for (시작점; 조건식; 매번 반복 후) {  
    // 조건이 만족하는 동안 반복될 코드  
}
```

✓ 기본 예제

- ✓ 횟수 반복 ("안녕하세요" 5번 출력)

```
for(let i = 0; i < 5; i++){  
    console.log("안녕하세요")  
}
```

while 문

- ✓ 조건이 참인 동안 코드를 계속 반복하는 문법
- ✓ 반복 횟수가 미리 정해지지 않은 상황에 유용
- ✓ 기본 구조

```
while (조건식) {  
    // 조건이 true 인 동안 실행될 코드  
}
```

while 문

```
let i = 1;
while (1 <= 5) {
    console.log(i, "번째 반복");
    i++;
}
```

[도전] 조건과 반복

✓ 문제

- ✓ 여러 개의 여행지가 있고, 각각은 계절 정보를 포함합니다.
- ✓ 사용자에게 계절을 입력 받아, 해당 계절에 맞는 여행지를 추천해주는 프로그램을 작성하세요.
- ✓ 여행지 목록과 계절 정보는 인덱스로 연결되어 있음
 - ✓ places[0] = 제주도, seasons[0] = 여름 -> 제주도는 여름에 추천

```
const places = ["제주도", "강릉", "부산", "설악산", "여수"];  
const seasons = ["여름", "겨울", "여름", "가을", "봄"];
```

- ✓ 일치하는 여행지가 없다면 "해당 계절의 추천 여행지가 없습니다." 출력

[도전] 조건과 반복

✓ 실행 예시

✓ 사용자 입력 대기

추천받고 싶은 계절을 입력하세요:

✓ 여름 입력 시

추천받고 싶은 계절을 입력하세요: 여름

[추천 여행지]

- 제주도
- 부산

✓ 없는 계절 입력 시

추천받고 싶은 계절을 입력하세요: 봄같은 가을

[추천 여행지]

해당 계절의 추천 여행지가 없습니다.

Chapter 12

함수와 파라미터

Vue.js를 활용한 프론트 앤드 개발

중복되는 로직의 문제점

✓ 같은 코드가 여러 번 중복되는 경우

✓ 예제

✓ 각 여행지의 가격이 정해져 있고, 할인 쿠폰을 적용할 수 있다.

✓ 할인 쿠폰 = (2만원 * 10% 추가) 할인

```
const jejuPrice = 200000;  
console.log("제주도 가격: " + (jejuPrice - 20000 * 1.1) + "원");  
  
const gangneungPrice = 150000;  
console.log("강릉 가격: " + (gangneungPrice - 20000 * 1.1) + "원");  
  
const busanPrice = 180000;  
console.log("부산 가격: " + (busanPrice - 20000 * 1.1) + "원");
```

중복되는 로직의 문제점

✓ 다음과 같은 상황에서 코드 유지보수를 어떻게 해야 할까 ?

- ✓ 할인 금액이 변경되는 경우
- ✓ 추가 할인 비율이 변경되는 경우
- ✓ 금액을 관리해야 할 도시가 더 많을 경우

```
const jejuPrice = 200000;  
console.log("제주도 가격: " + (jejuPrice - 20000 * 1.1) + "원");  
  
const gangneungPrice = 150000;  
console.log("강릉 가격: " + (gangneungPrice - 20000 * 1.1) + "원");  
  
const busanPrice = 180000;  
console.log("부산 가격: " + (busanPrice - 20000 * 1.1) + "원");
```

같은 로직이 반복되는 경우

리스크 ↑

시간 낭비 ↑

함수란

✓ 로직이 동일하고, 입력 값만 다를 때 코드를 따로 묶어주자

✓ 예제

```
function getDiscountedPrice(basePrice, discountAmount, discountRate) {  
    return basePrice - discountAmount * discountRate;  
}
```

함수

```
const discountAmount = 20000;  
const discountRate = 1.1;
```

```
const jejuPrice = getDiscountedPrice(200000, discountAmount, discountRate);  
const gangneungPrice = getDiscountedPrice(150000, discountAmount, discountRate);  
const busanPrice = getDiscountedPrice(180000, discountAmount, discountRate);
```

```
console.log("제주도 가격: " + jejuPrice + "원");  
console.log("강릉 가격: " + gangneungPrice + "원");  
console.log("부산 가격: " + busanPrice + "원");
```

함수 정의문

```
function 함수이름(전달받을값){//함수 선언식
    //실행할 코드
    return 1;
}

const 변수명 = function(전달받을값){//함수 표현식
    //실행할 코드
    return 2;
}

const 화살표함수 = () => { //화살표 함수
    //실행할 코드
    return 3;
}
```

함수 호출

```
function 함수이름(전달받을값){//함수 선언식
    //실행할 코드
    return 1;
}

const 변수명 = function(전달받을값){//함수 표현식
    //실행할 코드
    return 2;
}

const 화살표함수 = () => { //화살표 함수
    //실행할 코드
    return 3;
}

함수이름();
변수명();
화살표함수();
```

함수 정의 : 함수가 역할 작성

함수 호출 : 정의한 함수를 실행

함수 호출

```
function 함수이름(전달받을값){//함수 선언식
    //실행할 코드
    return 1;
}

const 변수명 = function(전달받을값){//함수 표현식
    //실행할 코드
    return 2;
}

const 화살표함수 = () => { //화살표 함수
    //실행할 코드
    return 3;
}

함수이름();
변수명();
화살표함수();
```

함수 정의 : 함수가 역할 작성

함수 호출 : 정의한 함수를 실행

파라미터 (Parameter)

- ✓ 외부에서 값을 전달 받아야 하는 경우
- ✓ 함수 내에서 이름을 함께 출력하고 싶은 경우
- ✓ 호출할 때마다 다른 값을 전달받아서 활용

파라미터

```
function sayHello(name) {  
  console.log("환영합니다! " + name + "님!");  
}
```

이름을 전달받아, 함수 내부에서 활용
파라미터 : 정의 시 받기로 한 값

```
sayHello("JS");    // "환영합니다! JS님!" 출력  
sayHello("VUE");   // "환영합니다! VUE님!" 출력
```

함수 호출 시 이름을 전달
인자 (Argument) : 호출 시 전달한 실제 값

전달인자 (Argument)

반환값 (Return Value)

- ✓ 함수를 실행한 반환값을 외부에서 사용할 수 있다
- ✓ 숫자 2개를 더한 값을 반환 (return 키워드)

```
function add(a, b) {  
  return a + b;  
}  
  
const result = add(3, 5);  
console.log(result);
```

[도전]3가지 방법으로 함수 정의

- ✓ 숫자 2개를 전달받아 더한 값을 출력
- ✓ 변수명, 함수명, 매개변수명 자유
- ✓ 함수 호출하여 결과 확인 및 에러 확인

[도전] 가격 계산기 만들기

✓ 가격 계산기 만들어보기

✓ 이름이 `getDiscountedPrice` 인 함수를 정의한다.

✓ 함수 구성

✓ 아래 3가지 데이터를 전달 받는다.

✓ `basePrice` : 기본 가격(int)

✓ `discountAmount`: 할인 가격(int)

✓ `discountRate`: 추가 할인 비율(double)

✓ 기본 가격 - (할인 가격 * 추가 할인 비율) 을 반환한다.

✓ 아래 조건에 맞게 3번 호출한다.

✓ 할인 가격: 20,000, 할인 비율: 1.1

✓ 기본 가격: 제주도(200000), 강릉(150000), 부산(180000)

• 출력 결과

제주도 가격: 178000원

강릉 가격: 128000원

부산 가격: 158000원

콜백함수

- ✓ 다른 함수의 인자로 전달되어, 그 함수 내부에서 호출되는 함수
- ✓ 나중에 호출될 함수
- ✓ 인자로 전달되어, 해당 함수 내부에서 실행되는 함수

```
function greet(name, callback) {  
  console.log(`안녕하세요, ${name}님!`);  
  callback(); // 전달받은 콜백 함수 실행  
}  
  
function sayBye() {  
  console.log("안녕히 가세요!");  
}  
  
greet("허범성", sayBye);
```

Chapter 13

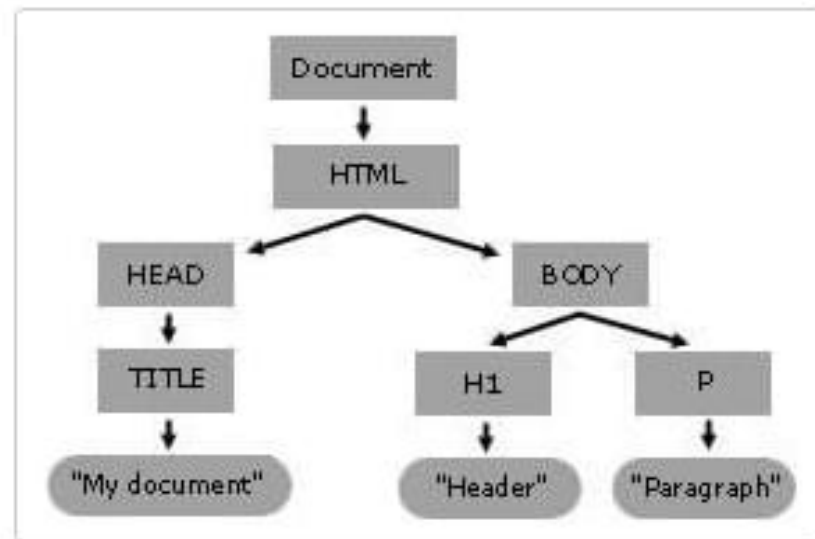
이벤트 처리

Vue.js를 활용한 프론트 앤드 개발

DOM

✓ HTML 문서의 구조적 표현 방법

```
<html lang="en">  
  <head>  
    <title>My Document</title>  
  </head>  
  <body>  
    <h1>Header</h1>  
    <p>Paragraph</p>  
  </body>  
</html>
```



[실습]QuerySelector

- ✓ H1태그의 콘텐츠를 hello world로 변경

```
<html lang="en">
  <head>
    <title>My Document</title>
  </head>
  <body>
    <h1>Header</h1>
    <p>Paragraph</p>
  </body>
  <script>
    document.querySelector('h1').textContent = 'hello world!'
  </script>
</html>
```

Event Driven

✓ 브라우저에서 event(사건) 발생시, 지정된 함수 호출

1. 함수를 만든다

2. 이벤트 발생시, 함수가 자동 호출 되도록 "등록" 한다

3. 이벤트가 발생하면, 함수가 호출된다

용어 정리

✓ 콜백 함수

1. 다른 코드의 Argument로 함수 이름을 넘겨 주는 함수
2. 소스코드의 직접 호출이 아닌, 이벤트로 인해 호출되는 함수

이벤트 핸들러

1. 이벤트 발생시 호출되는 함수
2. "콜백 함수" 중 용도를 명확하게 나타낸 용어

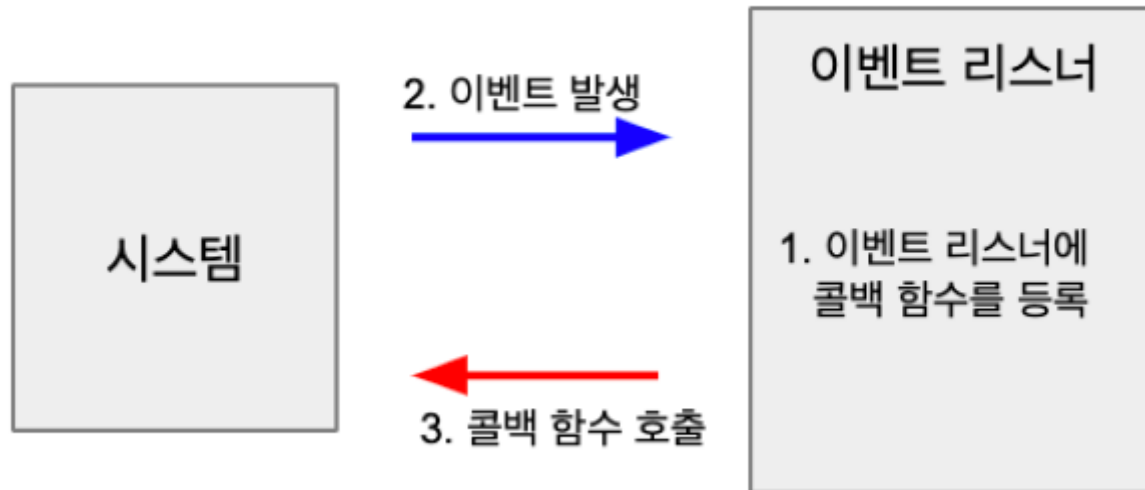
이벤트 리스너

1. 이벤트가 발생하는지 감지하여, 예약된 이벤트 핸들러를 호출해주는 객체

이벤트 발생 동작 원리

- ✓ 이벤트 리스너에 함수를 등록한다

addEventListener 메서드를 사용하여 이벤트 핸들러(콜백함수)를 등록한다
`addEventListener("이벤트이름","핸들러이름");`



[실습] 버튼 클릭시 함수 호출

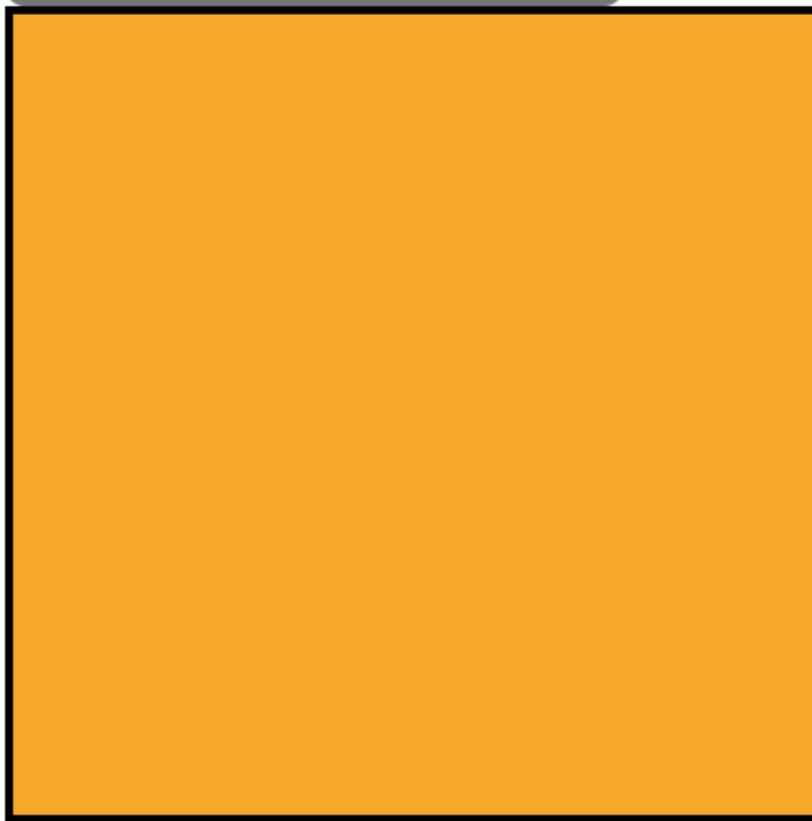
```
<html lang="en">
  <head>
    <title>My Document</title>
  </head>
  <body>
    <input type="button" value="버튼">
  </body>
  <script>
    document.querySelector('input').addEventListener("click", btnClick)

    function btnClick(){
      console.log("hello world!")
    }
  </script>
</html>
```


[실습] 버튼 클릭시 함수 호출 2

```
<html lang="en">
  <head>
    <title>My Document</title>
  </head>
  <body>
    <input type="button" value="버튼" onclick="btnClick()">
  </body>
  <script>
    function btnClick(){
      console.log("hello world!")
    }
  </script>
</html>
```

색상바꾸기



Change Event

✓ 값이 바뀔 때 발생하는 이벤트

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>change event</title>
</head>
<body>
  <input></input>
  <div></div>

  <script>
    const input = document.querySelector('input')
    const div = document.querySelector('div')

    input.addEventListener('change', changeEvent)

    function changeEvent(event){
      div.textContent = event.target.value
    }
  </script>
</body>
</html>
```

autoever

autoever

Keydown Keyup Event

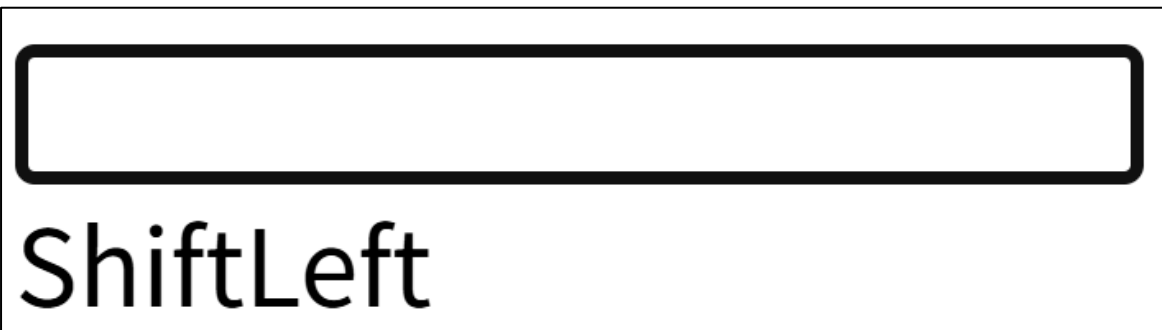
✓ 키를 눌렀을 때 반응하는 event

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>change event</title>
</head>
<body>
  <input></input>
  <div></div>

  <script>
    const input = document.querySelector('input')
    const div = document.querySelector('div')

    input.addEventListener('keydown', changeEvent)

    function changeEvent(event){
      div.textContent = event.code
    }
  </script>
</body>
</html>
```



[도전]회원가입 검사하기

- ✓ Id 및 pw가 둘다 공백일때, 회원가입 방지하기

아이디

비밀번호

회원가입

아이디와 비밀번호를 모두 입력하세요.

Chapter 14

비동기 처리

Vue.js를 활용한 프론트 앤드 개발

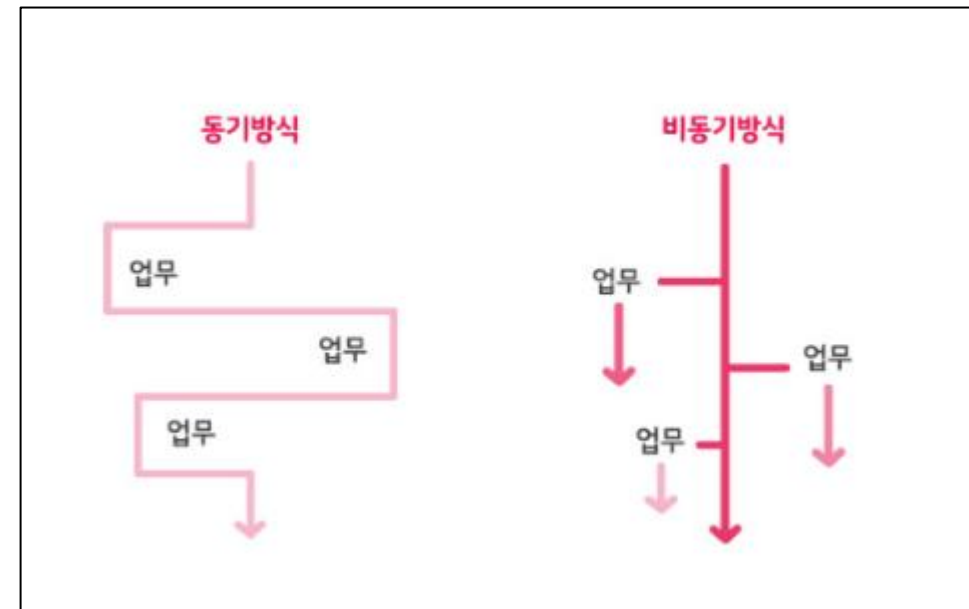
비동기

✓ 동기 방식

한 업무가 끝나야, 다음 업무를 시작한다

비동기 방식

다른 업무를 기다리지 않고, 진행한다



비동기

- ✓ 비동기란, 非(아닐비) 동기 -> 동기가 아니다
- ✓ 동기란, 同氣(같은동, 기약할기) -> 동시성의



코드의 결과를 예측해보자

```
console.log(1)  
console.log(2)  
console.log(3)
```

코드의 결과를 예측해보자

```
console.log('1')  
setTimeout(() => {  
  console.log('2')  
}, 1000);  
console.log('3')
```

```
console.log('1')  
setTimeout(() => {  
  console.log('2')  
}, 0);  
console.log('3')
```

[도전]axios 활용

- ✓ axios 비동기 통신하여, 데이터 활용하기
- ✓ 서버 주소 : <https://jsonplaceholder.typicode.com/todos>
- ✓ 해당 서버에서 data값을 받아 배열에 할당한후

completed : false,
userId가 짝수,
title의 글자수가 10자 이상

위의 조건에 해당하는 데이터만 배열에 할당후 출력

Chapter 15

개발 환경 추가

Vue.js를 활용한 프론트엔드 개발

개발 환경 추가

- ✓ Node.js LTS 설치
- ✓ Node.js 공식 홈페이지방문
- ✓ <https://nodejs.org/ko>
- ✓ LTS 인스톨러다운로드

Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

[Get Node.js®](#)

[Get security support](#)
for EOL Node.js versions



Node.js is proudly supported by the partners above and more.

Create an HTTP ServerWrite TestsRead and Hash a FileStreams PipelineWork with Threads

```
1 // server.mjs
2 import { createServer } from 'node:http';
3
4 const server = createServer((req, res) => {
5   res.writeHead(200, { 'Content-Type': 'text/plain' });
6   res.end('Hello World!\n');
7 });
8
9 // starts a simple http server locally on port 3000
10 server.listen(3000, '127.0.0.1', () => {
11   console.log('Listening on 127.0.0.1:3000');
12 });
13
14 // run with `node server.mjs`
```

JavaScript</> 클립보드에 복사

Learn more what Node.js is able to offer with our Learning materials.

Node.js LTS 설치

Node.js® 다운로드

Node.js®

v24.14.0 (LTS)

를

Windows

환경에서

Docker

방식으로

npm

를(을) 사용해 설치하세요.

정보

 Want new features sooner? Get the [latest Node.js version](#) instead and try the latest improvements!

```
1 # Docker는 각 운영 체제별로 설치 지침을 제공합니다.
2 # 공식 문서는 https://docker.com/get-started/에서 확인하세요.
3
4 # Node.js Docker 이미지를 풀(Pull)하세요:
5 docker pull node:24-alpine
6
7 # Node.js 컨테이너를 생성하고 셸 세션을 시작하세요:
8 docker run -it --rm --entrypoint sh node:24-alpine
9
10 # Verify the Node.js version:
11 node -v # Should print "v24.14.0".
12
13 npm 버전 확인:
14 npm -v # 11.9.0가 출력되어야 합니다.
```

PowerShell

</> 클립보드에 복사

Docker는 컨테이너화 플랫폼입니다. 문제가 발생하면 [Docker's 웹사이트](#)를 방문하세요.

또는

x64

 아키텍처가 실행 중인

Windows

 환경에서 미리 빌드된 Node.js®를 다운로드하세요.

📄

 Windows 설치 프로그램 (.msi)

📄

 Standalone Binary (.zip)

설치 확인

- ✓ Node.js LTS 설치 확인
 - cmd실행후, 다음두가지명령어를입력해서
- ✓ 버전체크
 - node -version
 - npm --version

```
C:\Users\User>node -v
v22.17.0

C:\Users\User>npm --version
10.9.2
```



Vue (Official)

Vue  vuejs.org

 5,900,062

     (124)

 Sponsor

Language Support for Vue

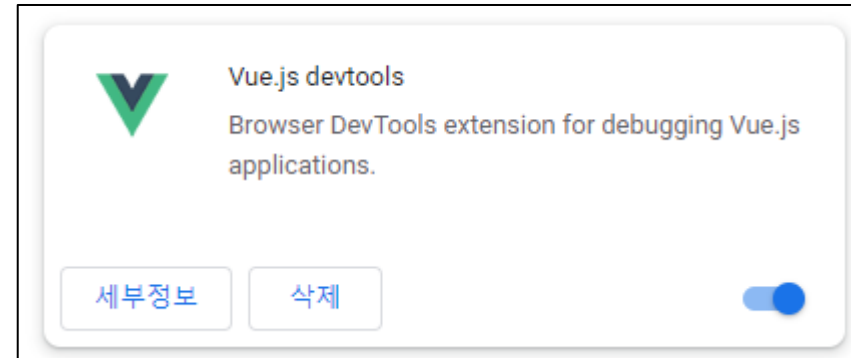
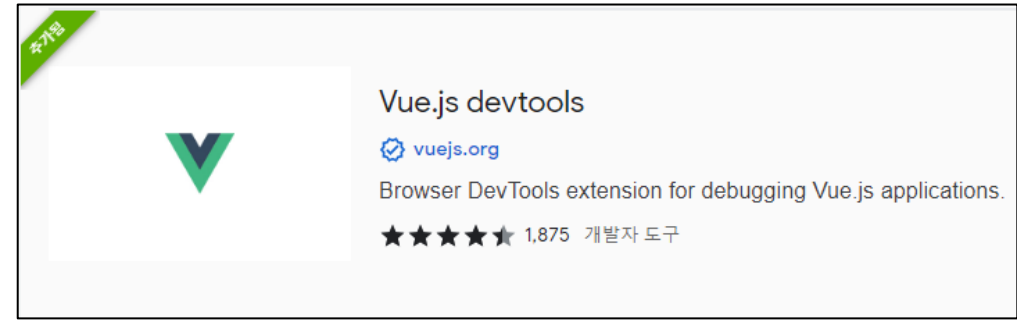
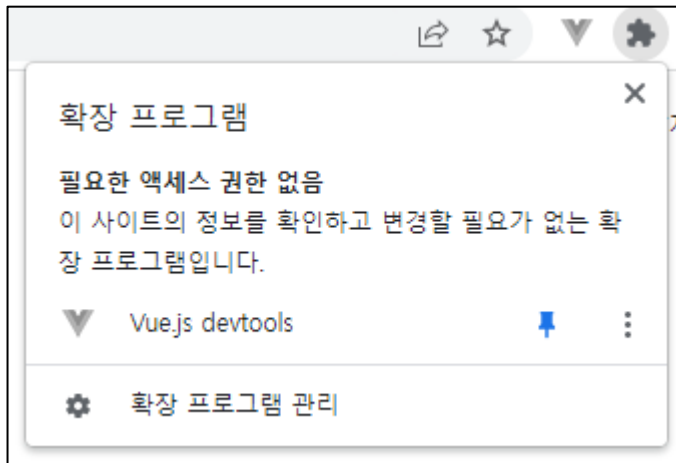
Disable 

Uninstall 

☒ Auto Update 

크롬 확장프로그램 설치

- ✓ 크롬 웹스토어 접속 후, 확장 설치
 - Vue.js devtools
- ✓ 설치 후, 확장이 잘 보이도록 고정
- ✓ 확장 프로그램 관리 버튼 클릭
 - 세부정보 클릭
 - 시크릿 모드에서 허용
 - 파일 URL에 대한 액세스 허용
 - 이후, 크롬창 재접속



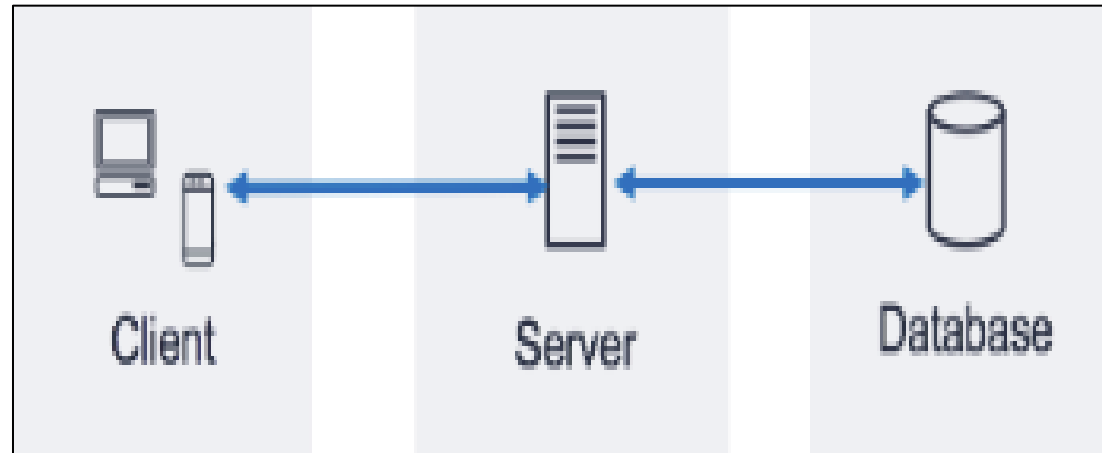
Chapter 16

Vue.js 개요 및 특징

Vue.js를 활용한 프론트 앤드 개발

웹의 기본 구조와 동작원리

- ✓ 서버와 클라이언트 개념



✓ SSR (Server Side Rendering)

- 기존 렌더링 방식
- 서버에서 렌더링 파일 준비하여 클라이언트로 전송
 - 초기 렌더링 빠름
- 검색엔진 최적화 (SEO) 쉬움
- 화면 변경사항 발생
 - 서버로 다시 렌더링 요청
 - 서버 부담 증가
 - 화면 변경 시 새로고침 발생

✓ CSR (Client Side Rendering)

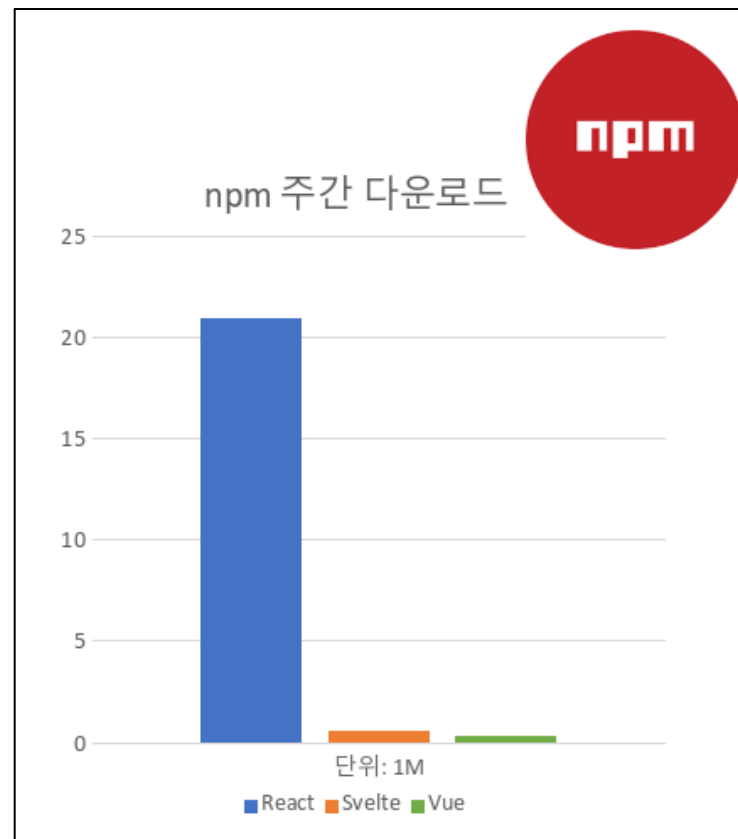
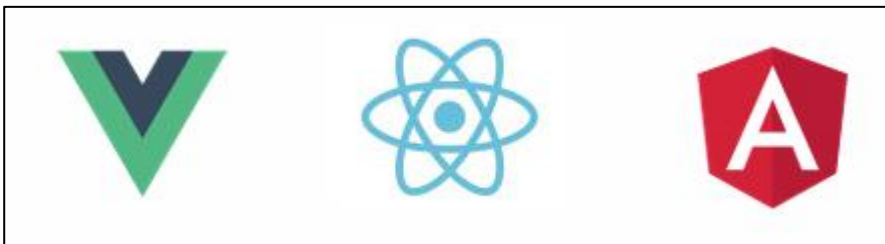
- 모던 렌더링 방식
- 클라이언트에서 렌더링 파일 준비
 - 초기 렌더링 느림
- 검색엔진 최적화 (SEO) 어려움
- 화면 변경사항 발생
 - 서버에 요청하지 않고 변화가 존재하는 부분만 변경
 - 서버 부담 감소
 - 새로고침 발생 안 함: 부드러운 화면 전환
- Vue.js에서 쓰이는 방식

Client-side frameworks 소개

✓ 클라이언트 측에서 UI와 상호작용을 개발하기 위해 사용되는 JavaScript 기반 프레임워크

SPA 프레임 워크 다운로드 비교

1. React
2. Svelte
3. Vue



왜 vue를 학습하는가?

1. HTML 코드의 변화가 적음

- React 는 JavaScript 안에서 HTML 을 사용 (JSX)
- 웹 퍼블리셔와 프론트엔드 개발자의 업무 분리 수월함

2. React 보다 쉬운 문법

- HTML, CSS, JavaScript 로 철저히 분리되어 가독성이 훨씬 좋음
- 완만한 학습곡선

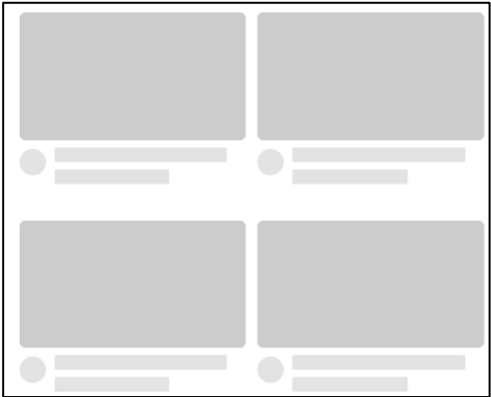
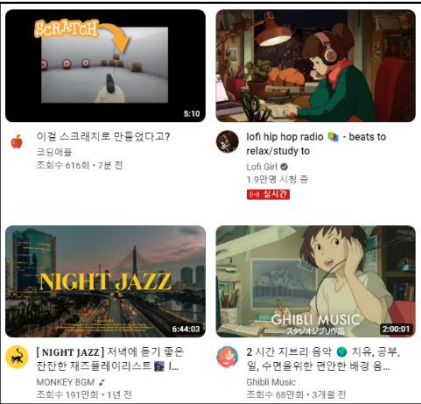
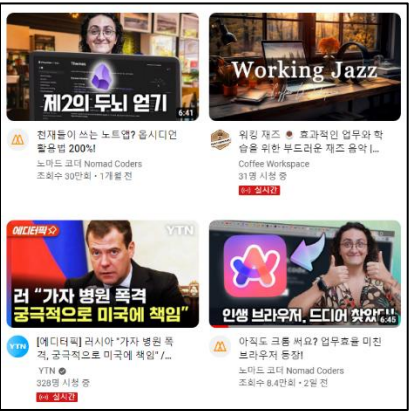
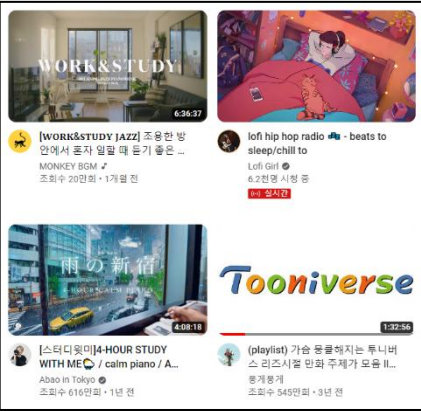
SPA(Single Page Application)이란?

- ✓ SPA는 하나의 HTML 페이지에서 동작하는 웹 애플리케이션으로, 초기 로딩 시 모든 리소스를 로드
 - ✓ 이후에는 필요한 데이터만을 서버로부터 비동기적으로 받아, 화면의 일부만 동적으로 갱신
- ✓ JavaScript를 통해 클라이언트 측에서 콘텐츠를 동적으로 생성 및 업데이트하며, 전체 페이지를 다시 로드하지 않아도 사용자와의 상호작용에 빠르게 반응

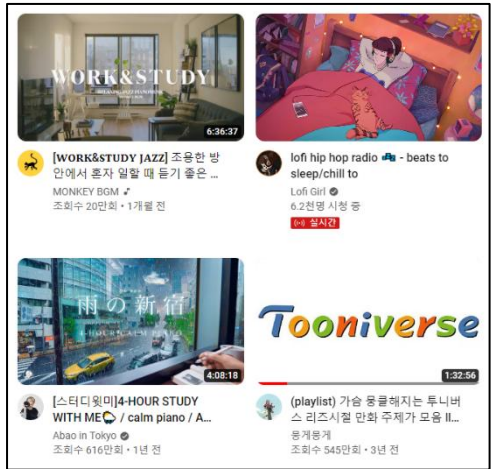
MPA vs SPA

C v# "개발 방식

} v# "개발 방식



먼저 틀을 받아옴



페이지가 아니라
데이터만 받아옴

새로운 페이지 접속할때마다 전체 페이지 받아옴

SPA 장점과 단점

✓ 회사 비용 절감

새로고침/다른 페이지 이동 발생 시,

- MPA 개발: 서버에 완성된 페이지 요청
- SPA 개발: 페이지가 아니라 데이터만 요청
 - 서버 트래픽을 줄여 비용 절감
 - 개발자 협업 및 업무 분할 편리
- 화면 각 부분을 **컴포넌트** 단위로 나눠서 개발
- 프론트엔드와 백엔드를 독립적으로 개발 가능

✓ SPA와 CSR의 단점

- 최초 로딩속도 느림
- SEO (검색엔진최적화) 불편
- nuxt로 해결

- ✓ 사용자 인터페이스를 구축하기 위한 JavaScript 프레임워크

The **Progressive** JavaScript Framework

An approachable, performant and versatile framework for building web user interfaces.

 Why Vue

Get Started →

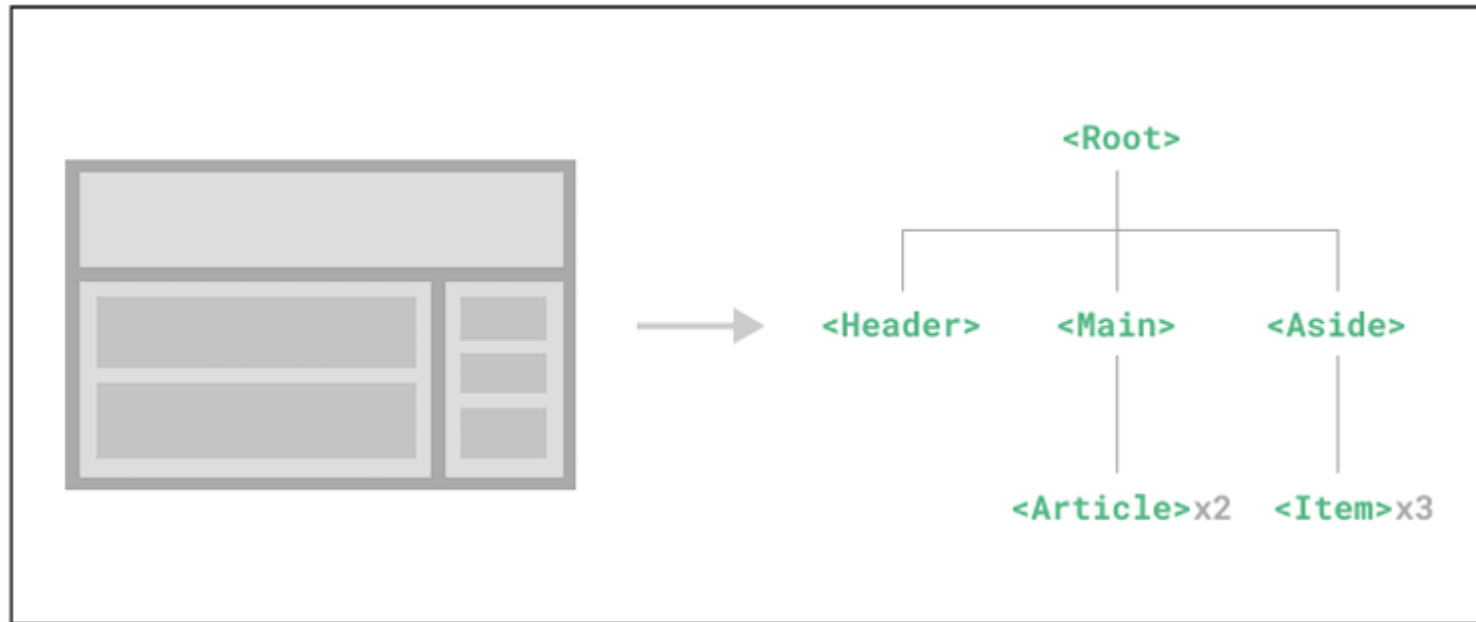
Install

Vue.js 특징

- ✓ **컴포넌트 기반 구조**를 통해 UI를 독립적이고 재사용 가능한 단위로 구성할 수 있어 유지보수와 확장이 용이
- ✓ **양방향 데이터 바인딩**을 지원하여 모델과 뷰 간의 실시간 데이터 동기화가 가능, DOM 조작을 줄임
- ✓ **가상 DOM(Virtual DOM)**을 사용해 효율적으로 DOM 변경을 감지하고 최소한의 렌더링으로 성능을 향상
- ✓ **단방향 데이터 흐름**도 지원, 상위-하위 컴포넌트 간 데이터 전달이 명확하며 예측 가능한 코드 작성이 가능
- ✓ **템플릿 문법이 직관적**이고 HTML 기반이므로 웹 개발자들이 쉽게 접근
- ✓ **공식 도구**(Vue CLI, Vue Router, Pinia 등)와 에코시스템이 잘 갖추어져 있어 프로젝트 설정부터 상태 관리, 라우팅까지 통합적인 개발이 가능

컴포넌트 기반 개발

- ✓ 재사용 가능한 코드 블록
- ✓ UI를 독립적이고 재사용 가능한 일부분으로 분할하고 각 부분을 개별적으로 다룰 수 있음 ➤ 자연스럽게 애플리케이션은 중첩된 Component의 트리 형태로 구성됨



Chapter 17

Vite로 Vue 프로젝트 시작하기

Vue.js를 활용한 프론트 앤드 개발

Vite 소개

- ✓ Vite(비트) 빠른 개발 환경을 위한 빌드 도구와 개발 서버 제공
- ✓ 번들러 + 빌드 툴
- ✓ 주요기능
 - 번들러 : 기본적인 프로젝트 세팅 제공
 - 번들러 없이 하나부터 열까지 세팅 복잡
 - 빌드 툴 : 프로젝트 빌드
 - .vue 파일을 HTML CSS JS 파일로 빌드
 - 개발 서버 제공



빌드?

- ✓ 프로젝트의 소스 코드를 최적화 + 번들링(하나로 합쳐) 배포할 수 있도록 변경
 - Vue 프로젝트를 서버에 올리기 위해 다음 파일로 변환하는 것
 - 하나의 HTML (SPA)
 - JavaScript
 - CSS
 - 이미지, 파비콘 등 기타 파일
- ✓ 배포: distribution. 빌드된 파일을 Apache, NginX 등의 HTTP Server 에 올림

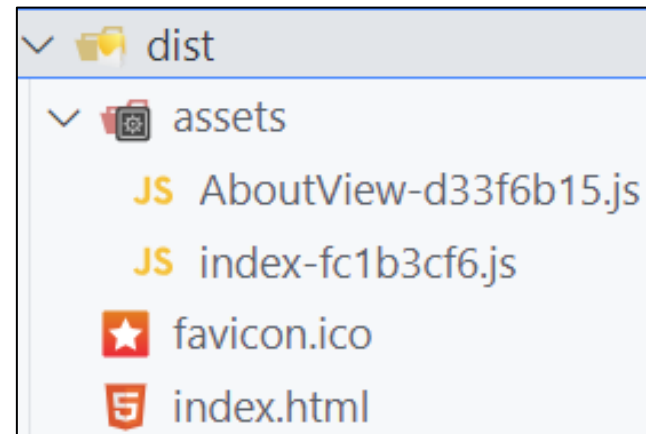
빌드 진행

- ✓ 명령어 : `npm run build`
- dist/ 디렉터리 생성됨
- 용량 변화 : 99M -> 177K
- HTML 파일은 하나만 확인됨
 - index.html
 - SPA 이기 때문

```
$ npm run build

> vite-project@0.0.0 build
> vite build

vite v4.4.11 building for production...
✓ 75 modules transformed.
dist/index.html                0.39 kB | gzip: 0.27 kB
dist/assets/index-63d04e84.js 172.66 kB | gzip: 57.67 kB
✓ built in 968ms
```



프로젝트 생성1

- ✓ cmd 실행 후, 원하는 디렉터리 경로설정 ex) 바탕화면
 - ✓ 다음 명령어를 입력해서 프로젝트 생성
 - ✓ 프로젝트이름 지정 ex) vue-project

```
$ npm create vue@latest  
  
> npx  
> create-vue  
  
  Vue.js - The Progressive JavaScript Framework  
◇ Project name (target directory):  
  vue-project  
◇ Select features to include in your project: (↑/↓ to navigate, space to select, a to toggle all, enter to confirm)  
  none  
◇ Select experimental features to include in your project: (↑/↓ to navigate, space to select, a to toggle all, enter to confirm)  
  none
```

프로젝트 생성2

- ✓ tab 클릭 시, default값 사용
 - ✓ enter 입력 시, 해당 값 적용하여 다음으로 넘어감

```
Vue.js - The Progressive JavaScript Framework
◆ Project name (target directory):
vue-project
```

프로젝트 생성3

- ✓ space 입력 시, 옵션 선택(중복) 가능
 - ✓ enter 입력 시, 해당 값 적용하여 다음으로 넘어감

```
Select features to include in your project: (↑/↓ to navigate, space to select, a to toggle all, enter to confirm)
☐ TypeScript
☐ JSX Support
☒ Router (SPA development)
☒ Pinia (state management)
☐ Vitest (unit testing)
☐ End-to-End Testing
☐ ESLint (error prevention)
☐ Prettier (code formatting)
```

프로젝트 생성4

✓ 프로젝트 생성 완료

Done. Now run:

```
cd vue-project  
npm install  
npm run dev
```

Optional: Initialize Git in your project directory with:

```
git init && git add -A && git commit -m "initial commit"
```

프로젝트 실행

- ✓ 프로젝트 생성 후, 다음 명령어를 입력
- ✓ cd 디렉터리 이름: 해당 디렉터리로 이동
- ✓ npm install : dependencies 설치 (추후 상세히 설명)
- ✓ npm run dev : 프론트엔드 서버 구동
- ✓ 서버 구동 성공 메시지 확인 후, localhost:5173으로 접속

```
cd vue-project
```

```
npm install
```

```
added 145 packages, and audited 146 packages in 10s  
  
43 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities
```

```
$ npm run dev  
  
> vue-project@0.0.0 dev  
> vite  
  
VITE v7.0.4 ready in 1343 ms  
  
→ Local:   http://localhost:5173/  
→ Network: use --host to expose  
→ Vue DevTools: Open http://localhost:5173/__devtools__/ as a separate window  
→ Vue DevTools: Press Alt(⌘)+Shift(⇧)+D in App to toggle the Vue DevTools  
→ press h + enter to show help
```



You did it!

You've successfully created a project with
Vite + Vue 3.



Documentation

Vue's [official documentation](#) provides you with all information you need to get started.



Tooling

This project is served and bundled with [Vite](#). The recommended IDE setup is [VSCode](#) + [Vue - Official](#). If you need to test your components and web pages, check out [Vitest](#) and [Cypress](#) / [Playwright](#). More instructions are available in [README.md](#).



Ecosystem

Get official tools and libraries for your project: [Pinia](#), [Vue Router](#), [Vue Test Utils](#), and [Vue Dev Tools](#). If you need more resources, we suggest paying [Awesome Vue](#) a visit.



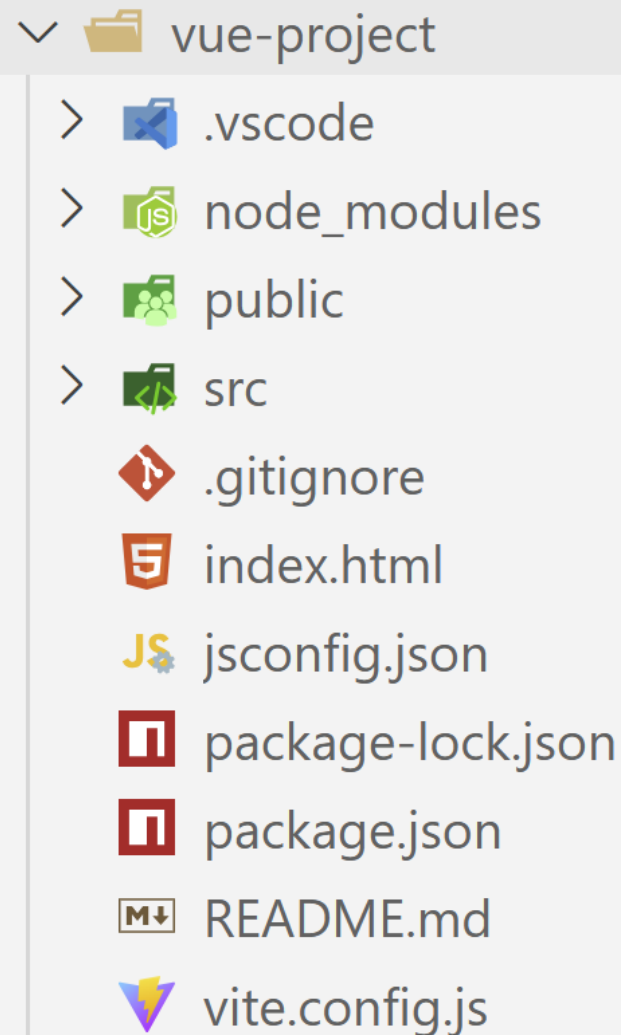
Community

Got stuck? Ask your question on [Vue Land](#) (our official Discord server), or [StackOverflow](#). You should also follow the official [@vuejs.org](#) Bluesky account or the [@vuejs](#) X account for latest news in the Vue world.



Support Vue

As an independent project, Vue relies on community backing for its sustainability. You can help us by [becoming a sponsor](#).



Vue.js 구조 이해

✓ public 디렉토리

주로 다음 정적 파일을 위치 시킴

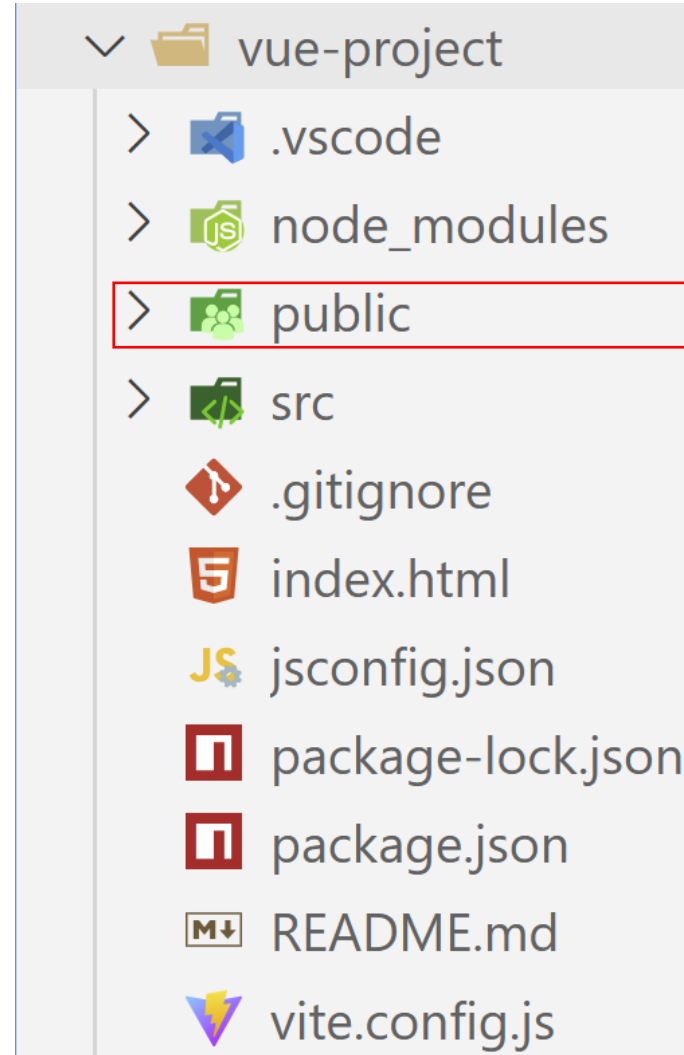
소스코드에서 참조되지 않는

항상 같은 이름을 갖는

import 할 필요 없는

항상 root 절대 경로를 사용하여 참조

public/icon.png는 소스 코드에서 /icon.png로 참조 할 수 있음

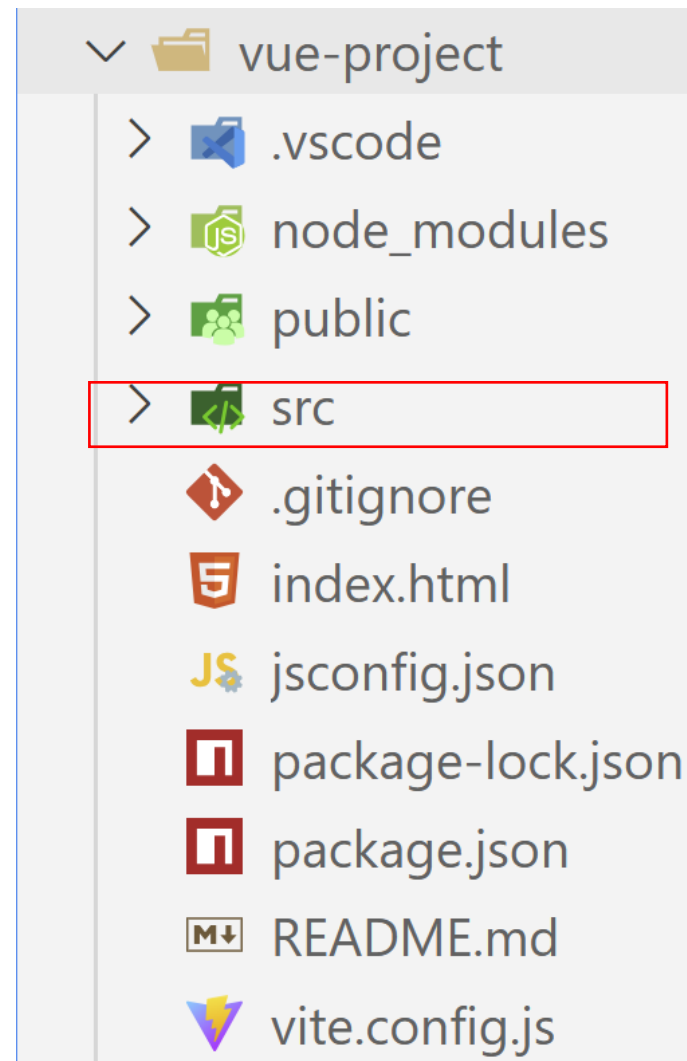


링크

Vue.js 구조 이해

✓ src 디렉토리

프로젝트의 주요 소스 코드를 포함하는 곳
실제로 우리가 작업하게 될 대부분의 소스 코드가 위치
컴포넌트, 스타일, 라우팅 등 프로젝트의 핵심 코드를 관리



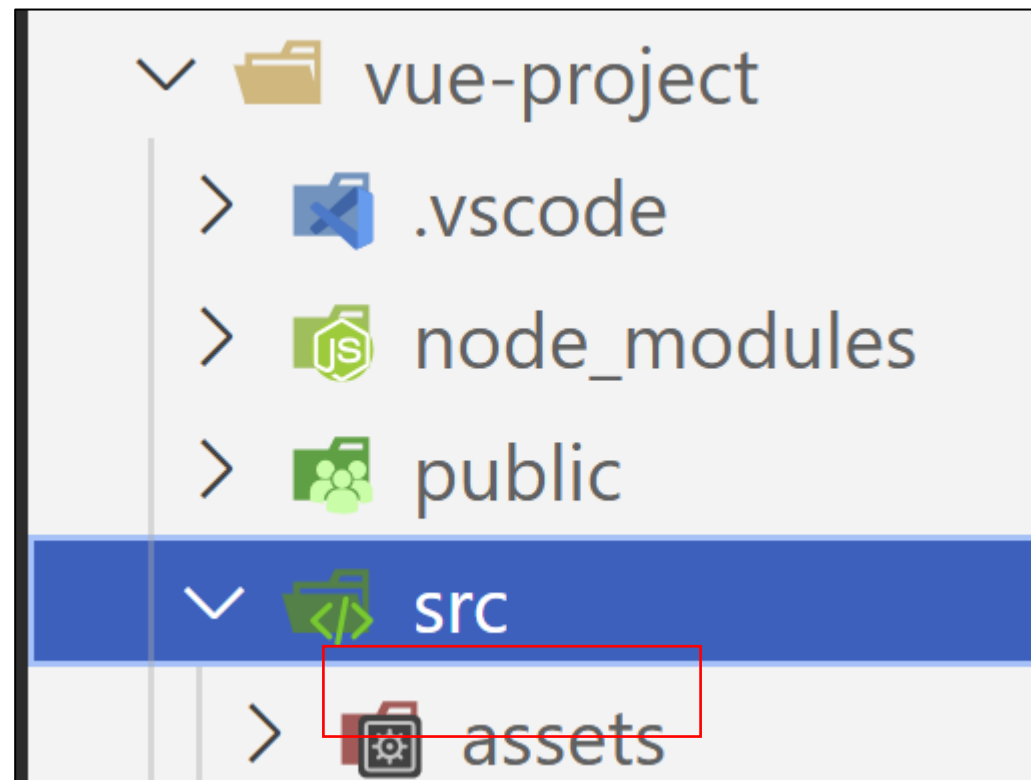
Vue.js 구조 이해

✓ src/assets 디렉토리

프로젝트 내에서 사용되는 정적 자원(이미지, 폰트, 스타일 시트 등)을 관리

컴포넌트 자체에서 참조하는 내부 파일을 저장하는데 사용

컴포넌트가 아닌 곳에서는 public 디렉토리에 위치한 파일을 사용



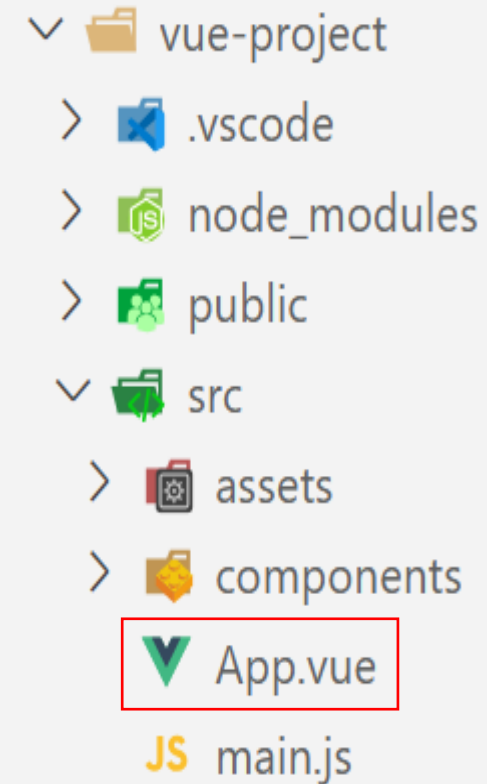
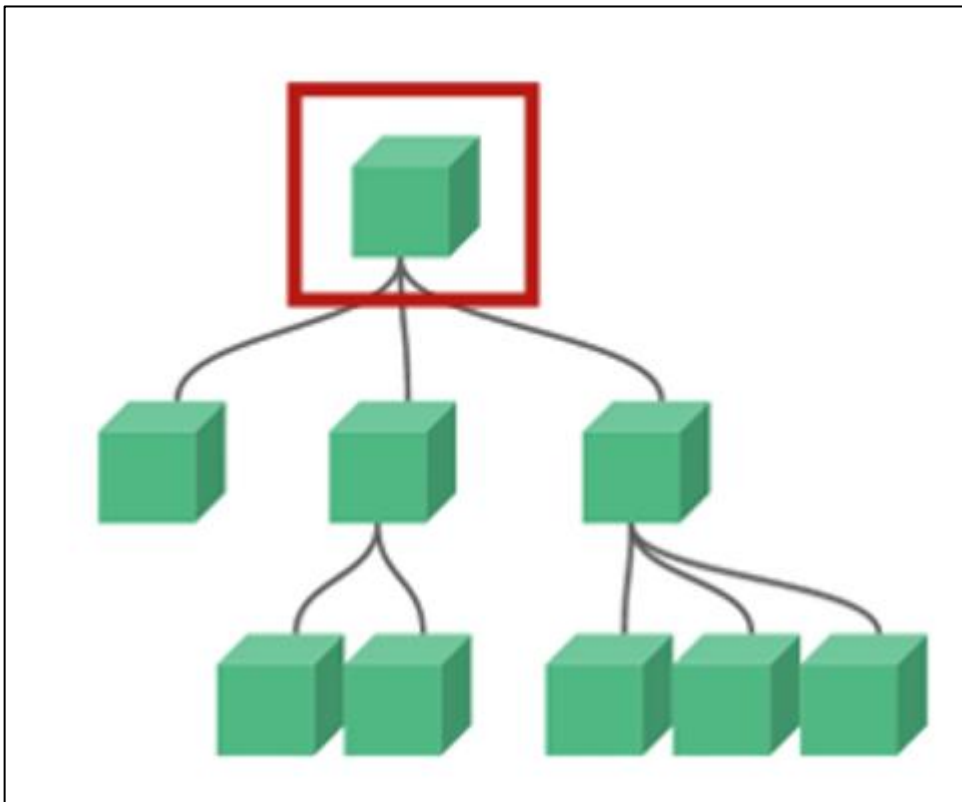
Vue.js 구조 이해

✓ src/App.vue 파일

Vue 앱의 Root 컴포넌트

다른 하위 컴포넌트들을 포함

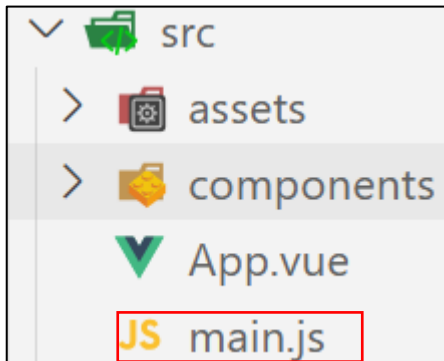
애플리케이션 전체의 레이아웃과 공통적인 요소를 정의



Vue.js 구조 이해

✓ src/main.js

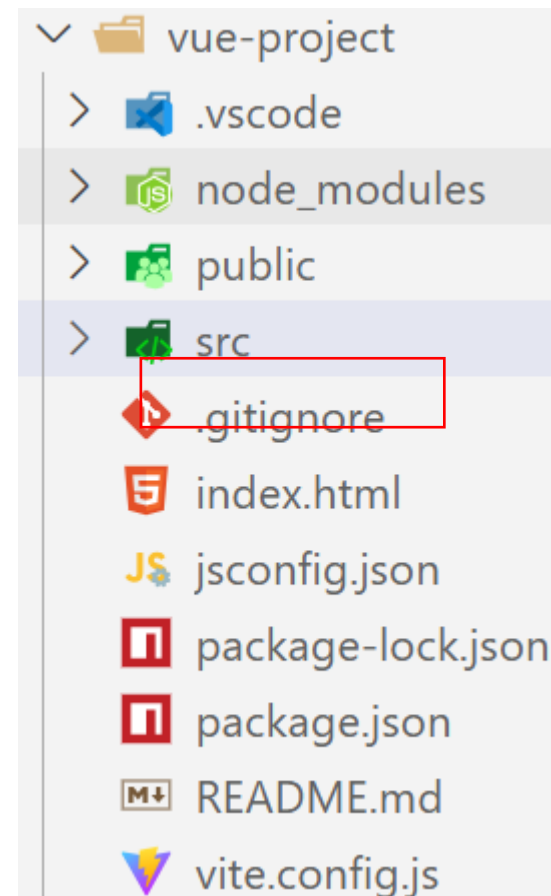
- Vue 애플리케이션을 초기화하고, App.vue를 DOM에 마운트하는 시작점
- 필요한 라이브러리를 import 하고 전역 설정을 수행



Vue.js 구조 이해

✓ index.html

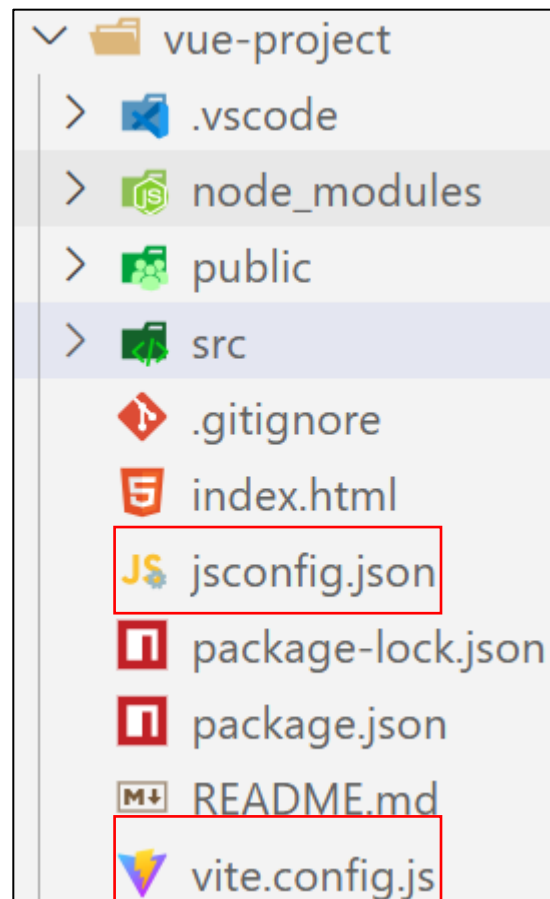
- Vue 앱의 기본 HTML 파일
- main.js에서 App.vue 컴포넌트를 렌더링하여 index.html의 특정 위치에 마운트
 - Vue 앱이 SPA인 이유
- 필요한 스타일 시트, 스크립트 등의 외부 리소스를 로드할 수 있음
- (ex. bootstrap CDN)



Vue.js 구조 이해

✓ 기타 설정 파일

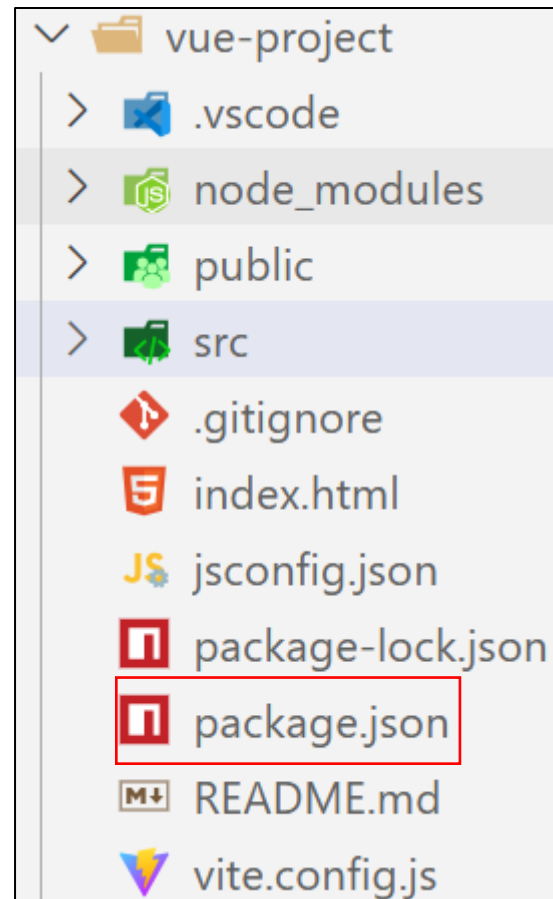
- jsconfig.json
 - 컴파일 옵션, 모듈 시스템 등 설정
- vite.config.js
 - Vite 프로젝트 설정 파일
 - 플러그인, 빌드 옵션, 개발 서버 설정 등



Vue.js 구조 이해

✓ package.json

- 프로젝트에 관한 기본 정보와
- 패키지 의존성을 정의하는 "설계도" 파일
- (메타데이터 파일)



package.json 역할

- ✓ 프로젝트가 어떤 패키지를 사용하고, 어떤 스크립트를 실행할 수 있는지 명시
 - npm install 시 이를 참조하여 패키지를 설치
 - 어떤 패키지를 설치해야 하는지 결정하는 기준 제공

package.json 특징

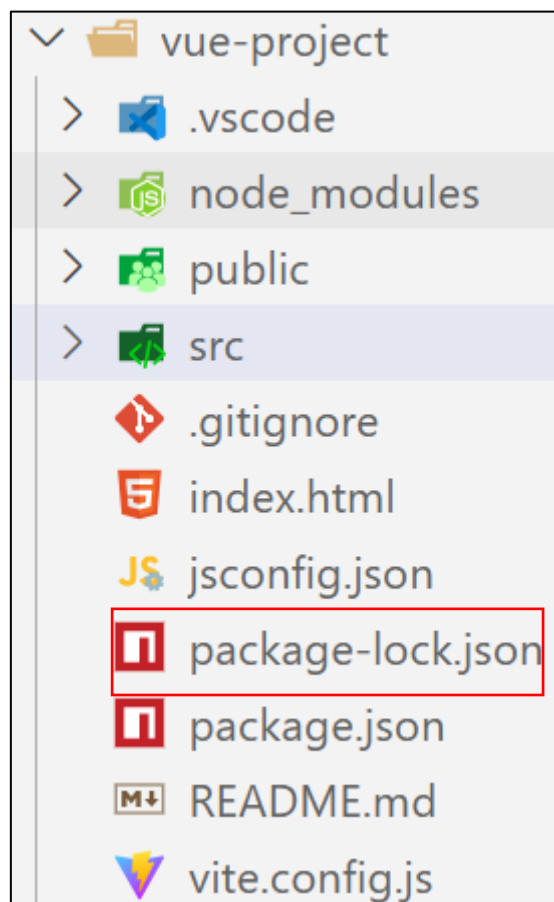
✓ 프로젝트 메타 데이터

- 프로젝트 이름, 버전, 스크립트 명령, 패키지 의존성 등의 정보가 명시됨
- 의존성(Dependencies) 목록
 - 어떤 패키지를 사용하는지, 어떤 버전 범위를 허용하는지를 기록
- 집을 짓기 전에 필요한 재료 목록과 건축 계획서
 - (필요한 재료(패키지)와 대략적 규격(버전 범위)을 알려주는 문서)

Vue.js 구조 이해

✓ package-lock.json

- package.json을 기반으로 실제 설치된 패키지들의 "정확한 버전 정보"를 기록하는 파일

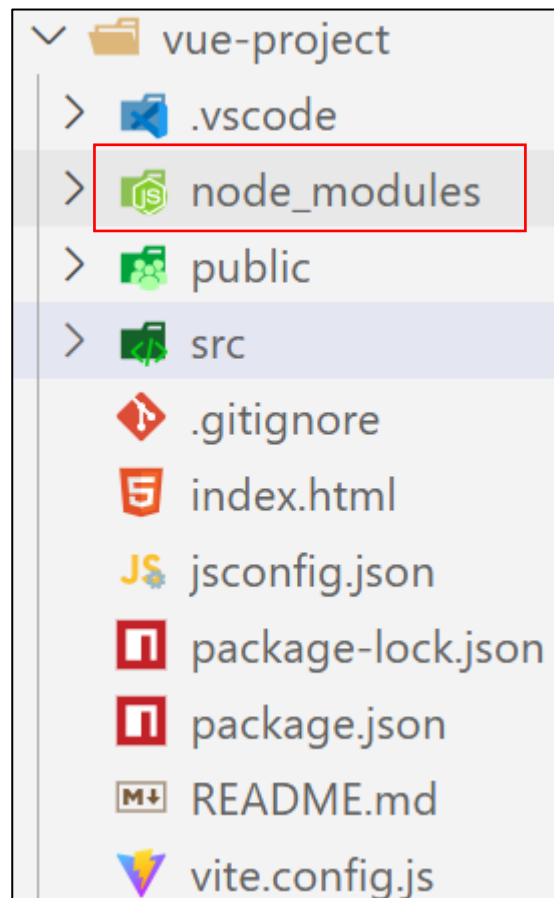


package-lock.json 역할

- ✓ 실제로 어느 버전의 패키지가 설치되었는지 확정하고 기록
 - 다른 환경에서도 동일한 패키지 구성을 재현 가능하게 함
- 팀원 간, 혹은 다른 환경(서버, 클라이언트 PC)에서 동일한 버전의 패키지를 재현 가능하게 함
- "장바구니에 실제로 담긴 물건들의 브랜드, 생산 일자까지 모두 기록한 상세 구매 내역서"

node_modules

- ✓ package.json과 package-lock.json에 따라 실제로 설치된 모든 패키지가 저장되는 곳
 - 프로젝트 실행 시 필요한 모든 라이브러리와 코드 파일을 보관
 - 애플리케이션 구동 시 참조되는 실제 데이터 저장소



node_modules 특징

- npm install을 통해 설치된 모든 패키지(모듈)들이 실제로 저장
- 개발 시 직접 수정할 필요는 없으며, npm install 시 자동 관리됨
 - 직접 수정하지 않고, 필요 시 npm install로 언제든지 재생성 가능
- 용량이 매우 클 수 있으며, 협업 시 일반적으로 Git으로 추적하지 않음(.gitignore에 포함)
- "계획서와 내역서대로 확보한 실제 건축 자재들이 쌓여 있는 창고"

(설계와 구매 목록을 바탕으로 실제 물리적 자재(파일)들이 모여 있는 장소)

정리

- ✓ package.json
 - 어떤 패키지가 필요하고 어떤 버전 범위를 허용할지 정의하는 "설계도"
- ✓ package-lock.json
 - 실제로 설치한 패키지의 정확한 버전을 기록하는 "상세 내역서"
- ✓ node_modules
 - 이 설계도와 내역서에 따라 내려 받은 실제 패키지 "자재 창고"

Chapter 18

Vue 기본 문법 - 데이터바인딩, 이벤트 처리

Vue.js를 활용한 프론트 앤드 개발

Vue 기본 문법

✓ script, template, style

- 컴포넌트 : HomeView.vue 처럼, .vue 확장자가 붙은 파일
 - 상세한 내용은 컴포넌트 챕터에서 다룸
- 총 세 개의 영역으로 나뉨
 - <script setup> : JavaScript
 - <template> : HTML
 - <style scoped> : CSS

```
<script setup>
// js code
</script>

<template>
<!-- html code -->
</template>

<style scoped>
/* css code */
</style>
```


템플릿 문법

- ✓ 데이터 바인딩의 가장 기본적인 형태는 "머스태시" 문법(이중 중괄호)을 사용하는 텍스트 보간
 - JavaScript 표현식 사용하기 가능

```
<template>
  <p>텍스트 보간 사용: {{ rawHtml }}</p>
  <p>v-html 디렉티브 사용: <span v-html="rawHtml"></span></p>

  {{ number + 1 }}

  {{ ok ? "YES" : "NO" }}

  {{ message.split("").reverse().join("") }}

  <div :id="`list-${id}`">1</div>
</template>

<script setup>
const rawHtml = "<span style='color: red'>이것은 빨간색이어야 합니다.</span>";
const number = 1;
const ok = true;
const message = "hello world";
const id = 1;
</script>

<style scoped>
</style>
```

ref(반응성 기본)

✓ 반응형 상태를 선언하는 함수

- ref는 인자를 받아 .value 속성이 있는 ref 객체로 감싸 반환
- ref로 선언된 값이 변경되면, 해당값을 이용하는 template도 자동 업데이트
- **템플릿**에서 사용할때는 .value를 작성할 필요 없음

```
<template>
  <button @click="count++">
    {{ count }}
  </button>
</template>

<script setup>
import { ref } from "vue";

const count = ref(0);
</script>
```

Vue 기본 문법 정리

- ref: 상태 (state) 생성 시 사용
- 상태는 일반 변수와 다르다

상태가 변하면, 화면에서 해당 부분을 다시 렌더링

렌더링: 화면을 그린다는 뜻

ref 사용 시 알아야 할 점

- 사용하려면 vue 에서 import 해야 함
- argument 로 들어가는 값은 message 변수의 기본값이 됨
현재 message 변수의 기본값은 "Hello vue"
- 하나의 변수를 리턴받음

message: **state**

- {{ }} : 콧수염을 뜻하는 "머스테치" 라고 부름
화면에서 변수 사용
- 머스테치를 활용해, HTML 태그에서 상태 사용

```
<script setup>
  import { ref } from 'vue'
  const count = ref(0)
  const msg = ref("hello world")
</script>

<template>
  <button @click="count++">
    {{ msg }} | Count is: {{ count }}
  </button>
</template>

<style scoped>

</style>
```

반응형 변수 vs 일반 변수

```
<template>
  <p>ref 변수 : {{ refValue }}</p>
  <p>그냥 변수 : {{ value }}</p>
  <button @click="add">값 증가</button>
</template>

<script setup>
import { ref } from "vue";
const refValue = ref(1);
const value = 1;
const add = () => {
  refValue.value++;
  value.value++;
};
</script>
```

ref 변수 : 2

그냥 변수 : 1

값 증가

vue에서 이벤트 사용

- ✓ v-on 디렉티브(@ 축약 가능) DOM 이벤트 리스닝

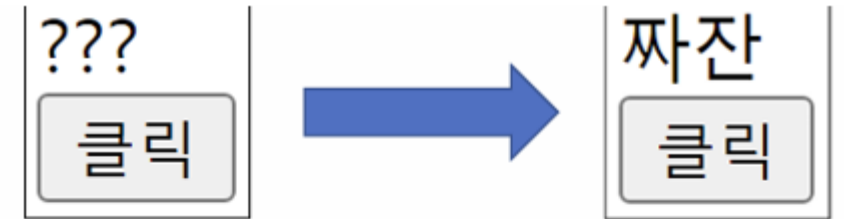
```
<template>
  <button @click="count++">Add 1</button>
  <p>Count is: {{ count }}</p>
</template>

<script setup>
import { ref } from "vue";
const count = ref(0);
</script>
```

v-on(@)

✓ 이벤트 받을 때 사용

- 이벤트 : 브라우저에서 일어나는 모든 종류의 사건
- ex) click, change, keyup, scroll...
- 축약 : v-on:click="changeText" @click="changeText"



```
1 <script setup>
2 import { ref } from "vue";
3
4 const text = ref("???");
5
6 function changeText() {
7   text.value = "짜잔";
8 }
9 </script>
10
11 <template>
12   <div>{{ text }}</div>
13   <button v-on:click="changeText">클릭</button>
14 </template>
```

이벤트명

함수

v-on:click="changeText"

[도전] 로또 번호 한자리 생성

✓ 1 부터 45 사이의 번호를 랜덤으로 생성

※ 1부터 45 사이 난수 생성

```
const num = Math.floor(Math.random() * 45) + 1;
```

로또 번호 생성기

번호 생성

31

Chapter 19

Vue Directive - 조건문과 반복문

Vue.js를 활용한 프론트 앤드 개발

directive

- ✓ 'v-' 접두사가 있는 특수 속성
 - ✓ directive의 속성 값은 단일 javascript 표현식(v-for, v-on 제외)
 - ✓ 표현식의 값이 변경될 때 DOM에 자동으로 업데이트 적용

```
<template>
  <h1 v-if="awesome">Vue는 멋져요!</h1>
</template>

<script setup>
  const awesome = true;
</script>
```

`v-on:submit.prevent="onSubmit"`

이름

v- 로 시작함.
단축 문법 사용 시
생략 가능.

인수

콜론(:) 또는 단축 문법에
뒤이어 사용.

수식어

점으로 시작함.

값

JavaScript
표현식으로 해석.

자주 사용 하는 내장 Directives 종류

- ✓ v-if
- ✓ v-else
- ✓ v-else-if
- ✓ v-for
- ✓ v-on
- ✓ v-bind
- ✓ v-model
- ✓ 등

V-bind 사용처

- ✓ Attribute Bindings (속성 바인딩)
- ✓ HTML의 속성 값을 Vue의 상태 속성 값과 동기화 되도록 함
- ✓ v-bind shorthand (약어) :콜론

```
<div>  
    
  <a v-bind:href="hrefUrl">Move to url</a>  
    
  <a :href="hrefUrl">Move to url</a>  
</div>
```

- ✓ Dynamic attribute name (동적 인자 이름)
- ✓ 대괄호([])로 감싸서 directive argument에 JavaScript 표현식을 사용
- ✓ 표현식에 따라 동적으로 평가된 값이 최종 argument 값으로 사용됨
- ✓ 대괄호 안에 작성하는 이름은 반드시 소문자로만 구성 가능
- ✓ (브라우저가 속성 이름을 소문자로 강제 변환)
- ✓ 위 코드에서 key는 변수 또는 표현식이 될 수 있으며, 해당 값이 실제 attribute 이름으로 사용됨

- ✓ 예를 들어, key가 "id"라면 <button id="myValue"> </button>과 동일하게 동작

```
<template>  
  <button :[attribute]="dynamic"></button>  
</template>
```

```
<script setup>  
  const attribute = "id";  
  const dynamic = "myBtn";  
</script>
```

- ✓ Binding to Objects
 - 객체를 :class에 전달하여 클래스를 동적으로 전환
- ✓ isActive의 Boolean 값에 의해 active 클래스의 존재가 결정됨

```
<template>
  <div
    class="static"
    :class="{ active: isActive, 'text-danger': hasError }"
  ></div>
</template>

<script setup>
const isActive = ref(true);
const hasError = ref(false);
</script>
```

- ✓ Binding to Objects
- ✓ inline 방식이 아닌 반응형 변수를 활용해 객체를 한 번에 작성하는 방법

```
<template>
  <div :class="classObject"></div>
</template>

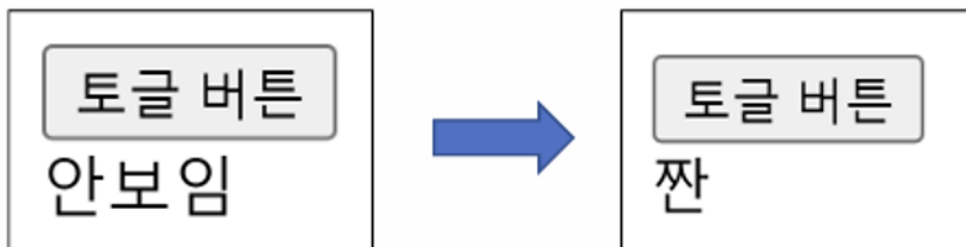
<script setup>
const classObject = reactive({
  active: true,
  "text-danger": false,
});
</script>
```


binding : 단방향 vs 양방향

- ✓ binding : 화면의 데이터와 Vue 데이터의 연결
 - 데이터가 바뀌면, 새로고침 없이 화면도 즉각 변경
- ✓ 단방향, 양방향 두 가지 종류가 존재
 - 구분 기준 : 사용자의 입력이 Vue 데이터를 바꿀 수 있는가
- ✓ 단방향
 - v-bind (:)
 - <input :value="message"> 는 데이터 변경 불가능
- ✓ 양방향
 - v-model
 - <input v-model="message"> 는 데이터 변경 가능

v-if, v-else-if, v-else

- ✓ v-if, v-else-if, v-else
 - ✓ 조건에 일치하면 태그를 보여줌
- ✓ Skill
 - ✓ v-on(@) 사용했지만, 함수 미사용
 - ✓ 특히, 토글을 구현할 때 자주 쓰는 기법
- ✓ v-if="isActive": isActive가 true일 때만 해당 div가 보임
- ✓ v-else: isActive가 false일 때만 해당 div가 보임



```
<script setup>
import { ref } from "vue";

const isActive = ref(false);
</script>

<template>
  <div>
    <button @click="isActive = !isActive">토글 버튼</button>
    <div v-if="isActive">짤</div>
    <div v-else>안보임</div>
  </div>
</template>
```

✓ 태그 반복

- menus 엘리먼트 각각을 menu로 받음
- menu.name 출력

✓ key를 중복 가능성이 없는 값으로 지정

- menu.name은 중복 가능성이 있기 때문에 부적
- menu.id로 처리

- 짜장면
- 짬뽕
- 탕수육

```
1  <script setup>
2  import { ref } from "vue";
3
4  const menus = ref([
5    {
6      id: 1,
7      name: "짜장면",
8    },
9    {
10     id: 2,
11     name: "짬뽕",
12   },
13   {
14     id: 3,
15     name: "탕수육",
16   },
17 ]);
18 </script>
19
20 <template>
21   <ul>
22     <li v-for="menu in menus" :key="menu.id">{{ menu.name }}</li>
23   </ul>
24 </template>
```

v-for="menu in menus" :key="menu.id"

V-for 구조

- ✓ v-for는 alias in expression 형식의 특수 구문을 사용
 - ✓ alias는 반복되는 각 항목의 별칭, expression은 반복 대상(배열/객체)
- ✓ 인덱스(객체에서는 key)에 대한 별칭을 지정할 수 있음
- ✓ v-for는 배열 또는 객체를 반복하여 태그를 생성할 때 사용
- ✓ 배열 반복 시 (item, index) 형태로 인덱스 별칭 지정 가능
- ✓ 객체 반복 시 (value, key) 또는 (value, key, index) 형태로 value, key, index 별칭 지정 가능

```
<div v-for="item in items" :key="item.text">{{ item.text }}</div>

<div v-for="(item, index) in arr" :key="index">{{ item }}</div>

<div v-for="value in object" :key="value">{{ value }}</div>
<div v-for="(value, key) in object" :key="key">{{ key }}: {{ value }}</div>
<div v-for="(value, key, index) in object" :key="index">
  {{ index }}: {{ key }}: {{ value }}
</div>
```

중첩 for

- ✓ 각 v-for의 하위 영역(scope)은 상위 영역에 접근할 수 있음

```
<script setup>
import { ref } from "vue";
const arrays = ref([
  { id: 1, title: "js", contents: ["변수", "함수", "배열"] },
  { id: 2, title: "vue", contents: ["ref", "v-for"] },
]);
</script>

<template>
  <div>
    <ul v-for="array in arrays" :key="array.id">
      <li v-for="content in array.contents" :key="content">
        {{ array.title }} - {{ content }}
      </li>
    </ul>
  </div>
</template>
```

v-model

- ✓ 사용자 입력값 변수로 받기
- ✓ `<input>` 태그에 v-model 반드시 사용됨
- ✓ message 변수 기본값 빈 문자열 ""
- ✓ `<input>` 태그에서 v-model 사용해 message 변수 연결
- ✓ 입력 할 때마다 `{{ message }}` 부분이 바뀜

```
<script setup>
import { ref } from "vue";
const message = ref("");
</script>

<template>
  <p>message: {{ message }}</p>
  <input type="text" v-model="message" />
</template>

<style scoped>
</style>
```

message: Hello Vue

input에 v-bind 사용

- ✓ <input> 에는 v-model 을 사용한다고 배웠지만,
- ✓ 만약 <input :value=""> 를 사용한다면?
- ✓ v-bind 는 사용자 입력값을 받을 수 없음
- ✓ 직접 차이를 확인해보자.

```
<script setup>
import { ref } from "vue";

const message = ref("글자를 지워보자");
</script>

<template>
  <input type="text" v-model="message" />
  <input type="text" :value="message" />
</template>
```

글자를 지워보자

글자를 지워보자

글자를 지워보자

글자를 지워보자

[도전] 사용자에게 입력값을 받아 색 변경

color

color

코드

[도전] 회원가입 양식

- ✓ 아이디, 비밀번호 입력창 오른쪽에 입력값 즉시 출력
 - ✓ 체크박스 오른쪽에 체크 여부 boolean 출력
 - ✓ 라디오버튼 아래에 현재 값 출력
 - ✓ <select> 아래에 현재 값 출력
-
- ✓ 라디오버튼과 <select>는 사용법을 찾아보고 value attribute를 활용

testid	testid
.....	testpw
☒ true	
☒ 빨강 ☐ 파랑	
red	
직업 학생	
student	

코드

Chapter 20

컴포넌트 기반 개발 - props와 emits

Vue.js를 활용한 프론트 앤드 개발

컴포넌트의 필요성

- ✓ 해당 화면을 구현했다고 가정
 - 하나의 index.html 파일에 구현됨
 - 코드 수는 약 5천줄 정도



컴포넌트의 필요성

상황1 : 아이콘 변경

하트 아이콘을 별 모양으로 변경

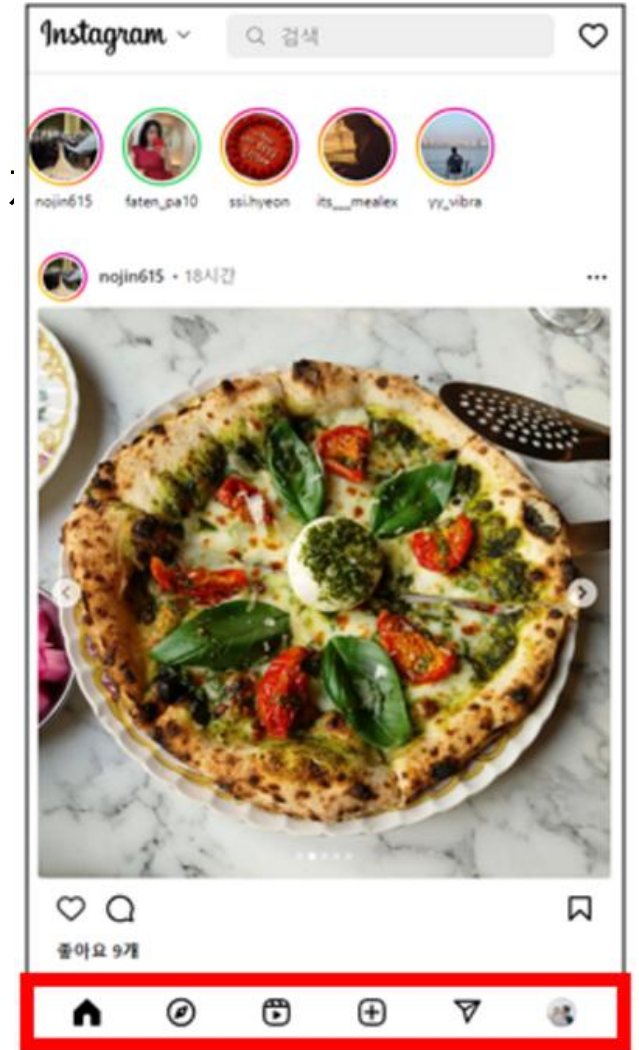
- index.html 파일을 연다
- **몇 천줄의 코드**에서 하트를 찾는다. (총 두 군데)
- 다른 아이콘으로 교체한다



컴포넌트의 필요성

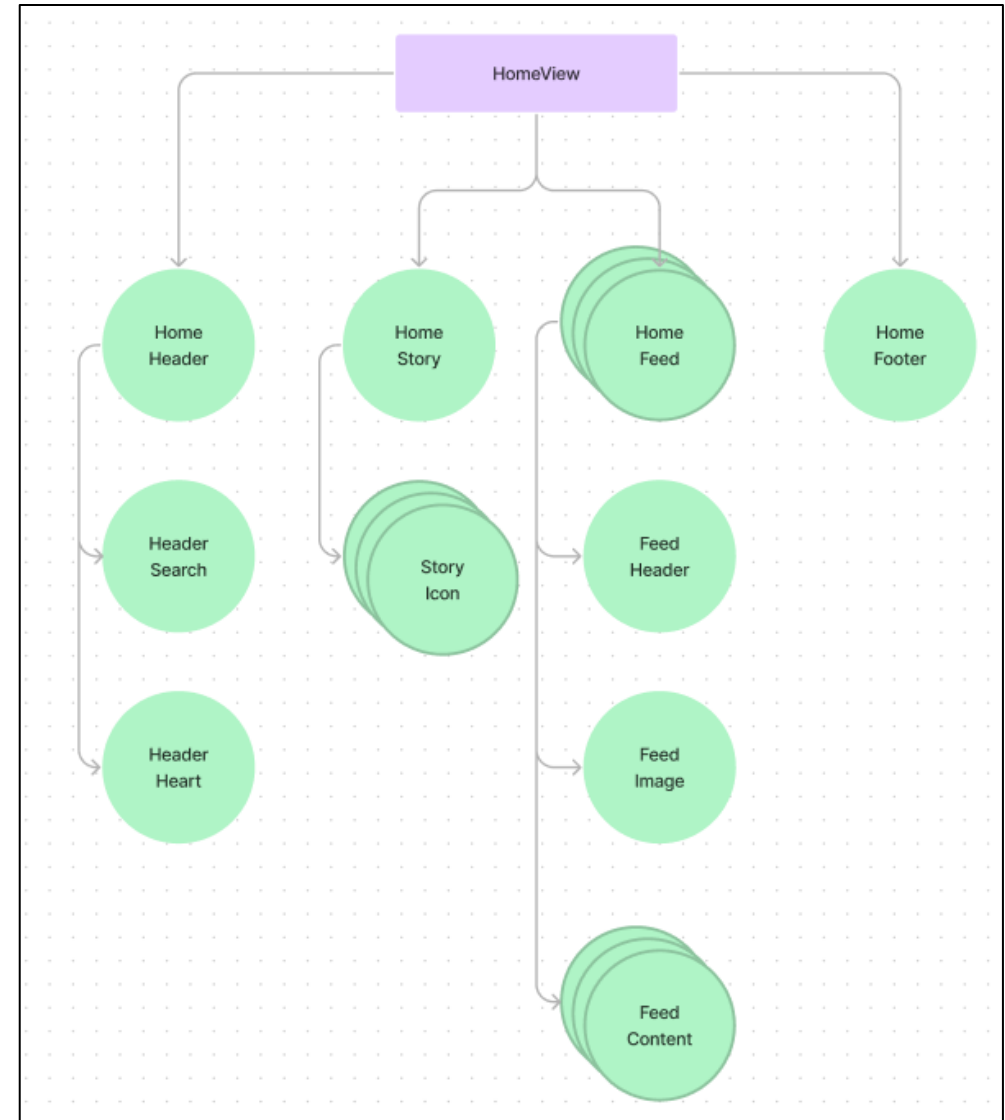
상황2 : 아이콘 리스트 변경

- <footer> 태그의 새 아이콘 리스트 적용
 - 디자이너는 신입 개발자에게 <footer> 태그의 새 아이콘 리스트
 - 다른 선배 개발자들은 메인 피드 영역 작업중
 - 하나의 index.html 파일을 양쪽에서 작업 - 다소 위험!
- 두 가지 상황 모두, 몇 천줄의 코드를 다루기 때문에 주의 필요
- 아이디어: 만약, 하나의 HTML 파일을 여러 파일로 나눌 수 있다면?



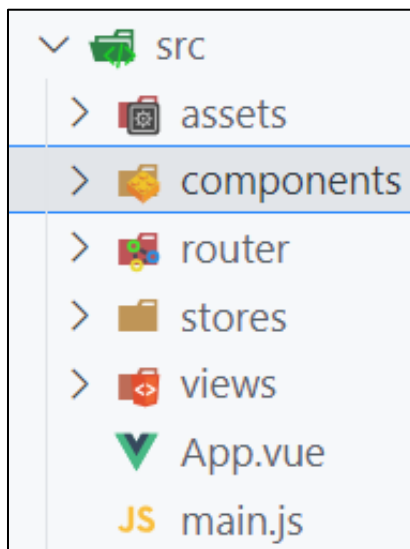
컴포넌트의 소개

- 인스타그램 화면을 컴포넌트 트리로 변경한 예
 - 트리의 요소는 하나의 파일 (.vue)
 - **각각의 파일: 컴포넌트**
 - 컴포넌트가 반복될 경우, 겹쳐서 표현
구현 시 v-for 사용
ex) StoryIcon, HomeFeed, FeedContent
- 처음 두 가지 상황을 돌아보자



컴포넌트 구현 준비

- ✓ src/ 디렉터리에 components/ 디렉터리 생성
 - ✓ HomeView.vue 파일을 기본 코드로 변경



```
<script setup>
</script>

<template>
  <h1>HomeView</h1>
</template>

<style scoped>
</style>
```

컴포넌트 구현 3단계

1. 파일 생성
2. 자식 import
3. 부모에 붙이기

구현 1단계 – 파일 생성

- ✓ HomeView 컴포넌트는 총 3개의 자식 컴포넌트를 가질 계획
- ✓ components/ 디렉터리에 다음 세 개의 파일을 제시된 양식으로 작성
 - HomeHeader.vue
 - HomeMain.vue
 - HomeFooter.vue
- ✓ **파일명 네이밍 시 주의사항**
 - 확장자 .vue
 - 반드시 대문자로 시작
 - App.vue 를 제외하고 두 개 이상 명사 PascalCase 결합
 - 어떻게 이름지어야 할 지 모르겠을 땐, 첫 번째 명사를 부모와 연관 있게 네이밍
 - ex) HomeView 의 자식 컴포넌트 => HomeChild

```
<script setup>
</script>

<template>
  <h2>컴포넌트이름</h2>
</template>

<style scoped>
</style>
```

구현 2단계 – 자식 import

- ✓ HomeView 컴포넌트를 다음과 같이 작성
 - ✓ 자식 컴포넌트를 가져옴
 - @ : src/ 를 의미
 - 경로 정확히 써줄 것
 - 확장자 반드시 붙여야 함

```
<script setup>
import HomeHeader from "@/components/HomeHeader.vue";
import HomeMain from "@/components/HomeMain.vue";
import HomeFooter from "@/components/HomeFooter.vue";
</script>

<template>
  <h1>HomeView</h1>
</template>

<style scoped>
</style>
```

구현 3단계 – 부모에 붙이기

✓ 컴포넌트를 붙일 때

- PascalCase 사용
- 닫는 기호 / 를 넣어줌

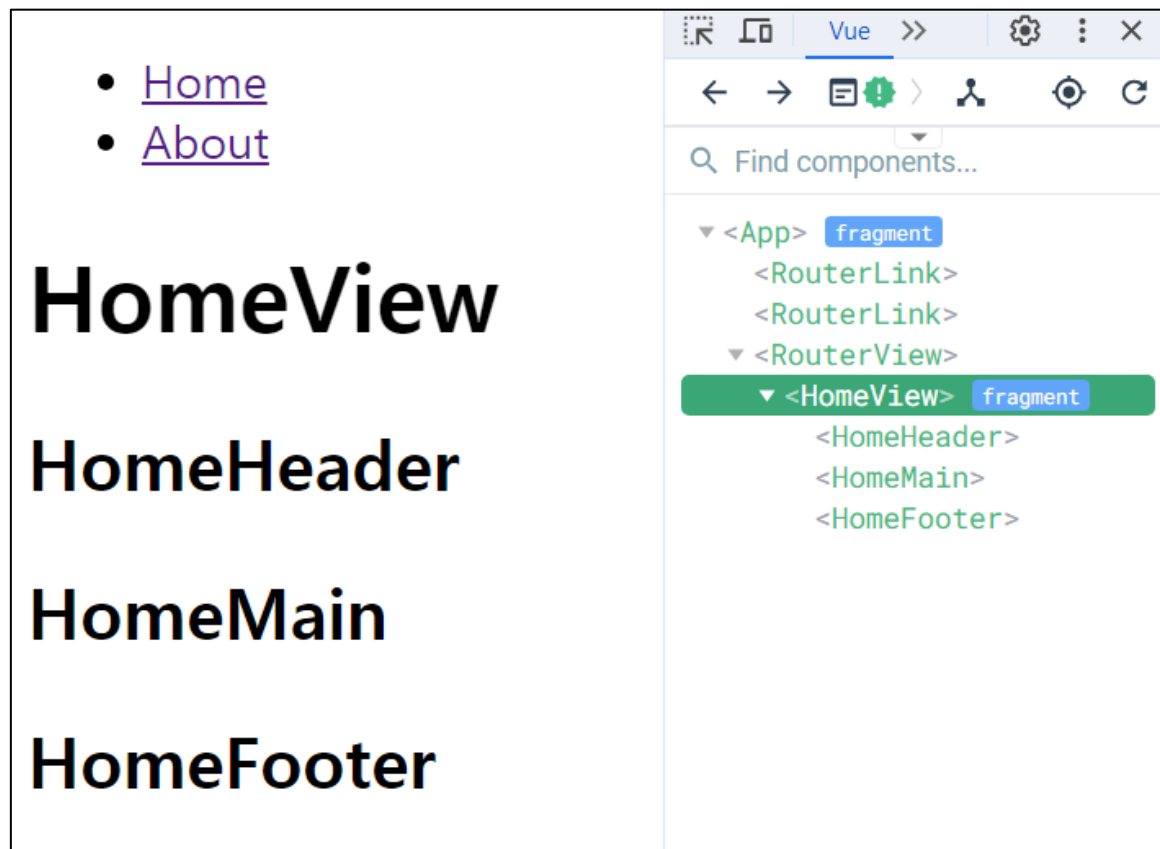
```
<script setup>
import HomeHeader from "@/components/HomeHeader.vue";
import HomeMain from "@/components/HomeMain.vue";
import HomeFooter from "@/components/HomeFooter.vue";
</script>

<template>
  <h1>HomeView</h1>
  <HomeHeader />
  <HomeMain />
  <HomeFooter />
</template>

<style scoped>
</style>
```

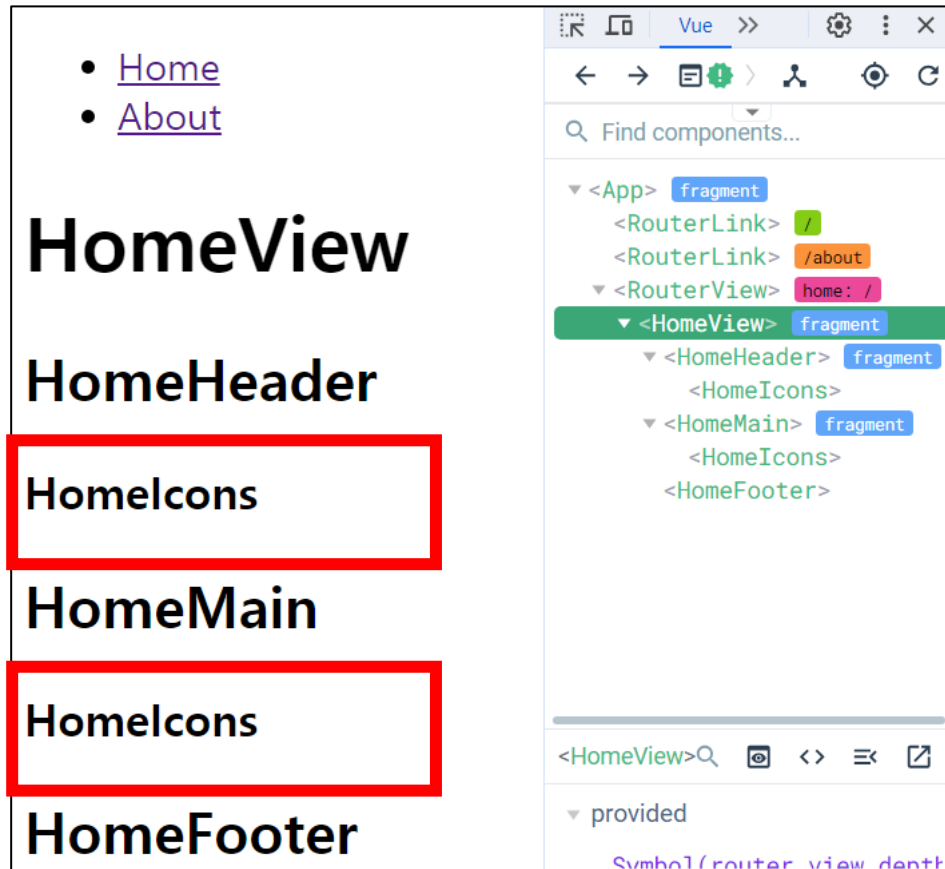
구현 확인

- ✓ F12 입력해 Vue.js Devtools 열기
 - ✓ 부모 자식 관계가 잘 설정되었는지 확인



[도전]컴포넌트 재사용

- ✓ Homelcons 컴포넌트 생성 후, 다음 두 컴포넌트에서 import
 - HomeHeader
 - HomeMain



- ✓ props : 부모 컴포넌트로 부터 자식 컴포넌트로 데이터를 전달하는데 사용되는 속성
- ✓ 부모 속성이 업데이트되면 자식으로 전달, 자식은 부모에게 x
 - 자식 컴포넌트 내부에서 props를 변경하려고 시도해서는 안되며 불가능
- ✓ 부모 컴포넌트가 업데이트될 때마다 이를 사용하는 자식 컴포넌트의 모든 props가 최신 값으로 업데이트
 - 부모 컴포넌트에서만 변경하고 이를 내려 받는 자식 컴포넌트는 자동갱신

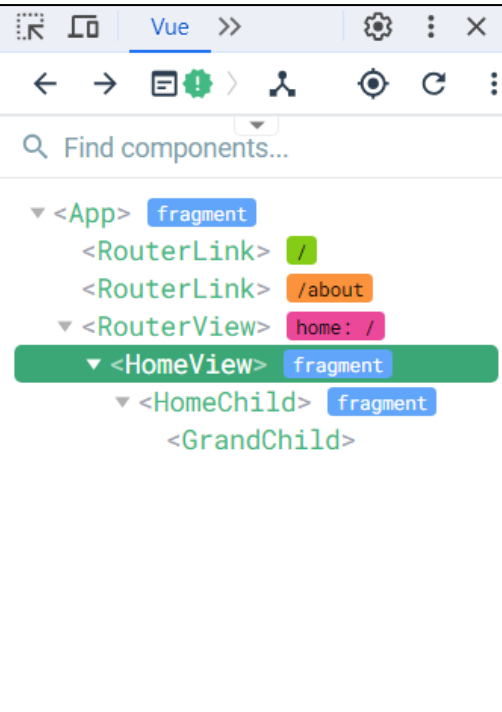
✓ 다음 구조로 컴포넌트 구현

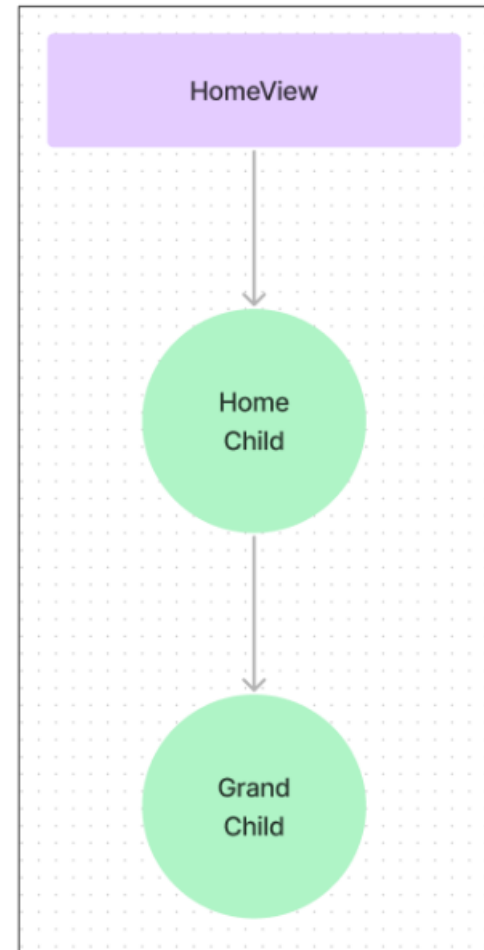
- [Home](#)
- [About](#)

HomeView

HomeChild

GrandChild





- ✓ HomeView 컴포넌트 코드를 다음과 같이 변경

```
<script setup>
import HomeChild from "@/components/HomeChild.vue";
import { ref } from "vue";

const name = ref("david");
const age = ref(10);
</script>

<template>
  <h1>HomeView</h1>
  <p>{{ name }}</p>
  <p>{{ age }}</p>
  <HomeChild />
</template>

<style scoped>
</style>
```

HomeView

david

10

HomeChild

GrandChild

컴포넌트 분리 장단점

✓ 장점

- 하나의 화면을 각 컴포넌트별로 개발 후, 하나의 HTML 문서로 결합
- 생산성 향상 및 유지보수 편리

✓ 단점

- 분리된 컴포넌트간 데이터 연동 고려해야 함
- **부모 컴포넌트가 자식 컴포넌트에 데이터를 전달하고 싶다면?**

✓ 현재 변수는 HomeView 에 위치

- name: "david"
- age: 10

✓ 이 변수들을 자식 컴포넌트인 HomeChild 로 내려보낼 것

✓ 대체 왜?

HomeView

david

10

HomeChild

GrandChild

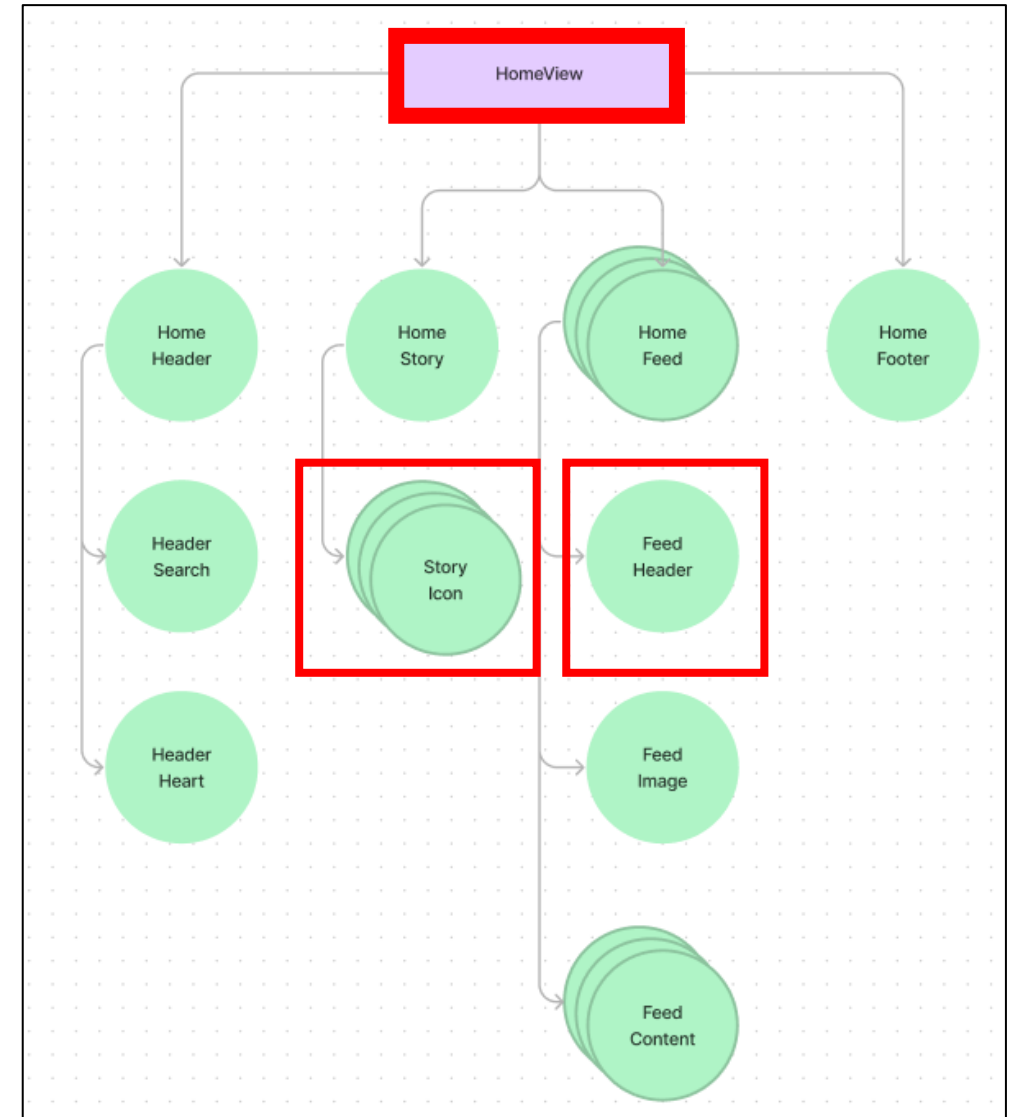
props의 필요성

- ✓ 하나의 화면에서 상태 중복 발생
 - 프로필사진
 - 닉네임
- ✓ 그러나, 서로 다른 컴포넌트에서 상태 사용 중
- ✓ 만약 컴포넌트별로 별도의 상태로 관리한다면?
 - 프로필사진, 닉네임 변경 발생
 - 각각의 컴포넌트는 서버로 별도의 변경 요청
- ✓ 만약, 두 개가 아니라 열 개의 서로 다른 컴포넌트라면?
 - 이런 상황이 수천 명의 사용자에게 발생한다면?



- ✓ 해결책: 상태를 위로 끌어올리기
 - lifting state up
- ✓ 즉, 공통된 부모에서 상태를 관리하고,
- ✓ 필요할 때 자식에게 내려보내기
- ✓ props
 - 부모 컴포넌트에서 자식 컴포넌트로 전달된 데이터
 - 반드시 부모 자식 관계여야 가능

profile_src
username



props 기본

✓ HomeView 컴포넌트: <HomeChild> 부분 변경

- v-bind 사용
- 마치 attribute 사용하듯 작성
- 즉, 컴포넌트에 "속성" 추가
 - name
 - age
- 팁: 이름을 일치시켜주면 헛갈리지 않는다

자식에서
받을이름

부모에서
보낼상태

: name="name"

```
<script setup>
import HomeChild from "@/components/HomeChild.vue";
import { ref } from "vue";

const name = ref("david");
const age = ref(10);
</script>

<template>
  <h1>HomeView</h1>
  <p>{{ name }}</p>
  <p>{{ age }}</p>
  <HomeChild :name="name" :age="age" />
</template>
```

props 기본

HomeChild 컴포넌트

- defineProps 함수 사용
객체 안에서, 각각의 prop 타입을 대문자로 시작해 작성
- prop 사용은 상태 사용과 동일

```
<script setup>
import GrandChild from "@/components/GrandChild.vue";

defineProps({
  name: String,
  age: Number,
});
</script>

<template>
  <h2>HomeChild</h2>
  <p>{{ name }}</p>
  <p>{{ age }}</p>
  <GrandChild />
</template>
```

HomeView

david

10

HomeChild

david

10

GrandChild

props 값 변경

- ✓ HomeView 에서 클릭 버튼 추가해 age 변경
- ✓ props 로 내린 자식의 age 역시 변경됨

```
<script setup>
import GrandChild from "@/components/GrandChild.vue";

defineProps({
  name: String,
  age: Number,
});
</script>

<template>
  <h2>HomeChild</h2>
  <p>{{ name }}</p>
  <p>{{ age }}</p>
  <GrandChild />
</template>
```

HomeView

david

10

HomeChild

david

10

GrandChild

props 손자까지 내리기

- ✓ 자식의 자식으로 전달하려면?
부모인 HomeChild 에서 받은 props 를
자식인 GrandChild 로 재전달
- ✓ 이번엔 name 만 손자까지 보내보기

```
<script setup>
import GrandChild from "@/components/GrandChild.vue";

defineProps({
  name: String,
  age: Number,
});
</script>

<template>
  <h2>HomeChild</h2>
  <p>{{ name }}</p>
  <p>{{ age }}</p>
  <GrandChild :name="name" />
</template>
```

```
<script setup>
defineProps({
  name: String,
});
</script>

<template>
  <h3>GrandChild</h3>
  <p>{{ name }}</p>
</template>
```

HomeView

david

10

나이를 줄이자

HomeChild

david

10

GrandChild

david

자식에서 props 변경 불가

- ✓ HomeChild 에서 name 을 바꿔보자
 - <script setup> 에서 props 에 접근할 땐,
defineProps 의 리턴값으로 props 을 받아서 접근
ex) name => props.name
- ✓ 변경 불가 : readonly
- ✓ 자식은 부모가 준 props 를 변경할 수 없다.
- ✓ 이 경우, 부모에게 "변경 요청" 을 해야 한다.

```
<script setup>
import GrandChild from "@/components/GrandChild.vue";

const props = defineProps({
  name: String,
  age: Number,
});

function changeName(newName) {
  props.name = newName;
}
</script>

<template>
  <h2>HomeChild</h2>
  <p>{{ name }}</p>
  <p>{{ age }}</p>
  <button @click="changeName('tom')">이름 변경</button>
  <GrandChild :name="name" />
</template>
```

⚠ [Vue warn] Set operation on key "name" failed: target is readonly.

- emit : 자식에서 부모로 보내는 이벤트
- defineEmits 에서 배열로 정의
defineProps, defineEmit 은 import 불필요 (컴파일타임 매크로)
- 사용 시, emit("이벤트명", 매개변수, 매개변수, ...)

emit ★ +

동사 격식 (빛·열·가스·소리 등을) 내다[내뿜다]

```
<script setup>
import GrandChild from "@/components/GrandChild.vue";

defineProps({
  name: String,
  age: Number,
});
const emit = defineEmits(["changeName"]);

function changeName(newName) {
  emit("changeName", newName);
}
</script>

<template>
  <h2>HomeChild</h2>
  <p>{{ name }}</p>
  <p>{{ age }}</p>
  <button @click="changeName('tom')">이름 변경</button>
  <GrandChild :name="name" />
</template>
```

- ✓ 부모 컴포넌트에서 v-on 으로 이벤트 받음

```

<script setup>
import HomeChild from "@components/HomeChild.vue";
import { ref } from "vue";

const name = ref("david");
const age = ref(10);

function changeAge(newAge) {
  age.value = newAge;
}

function changeName(newName) {
  name.value = newName;
}
</script>

<template>
  <h1>HomeView</h1>
  <p>{{ name }}</p>
  <p>{{ age }}</p>
  <button @click="changeAge(age - 1)">나이를 줄이자</button>
  <HomeChild :name="name" :age="age" @changeName="changeName" />
</template>

```

자식에서 보낸 이벤트

실행할 함수

@changeName="changeName"

HomeView

david

10

나이를 줄이자

HomeChild

david

10

이름 변경

GrandChild

david

HomeView

tom

10

나이를 줄이자

HomeChild

tom

10

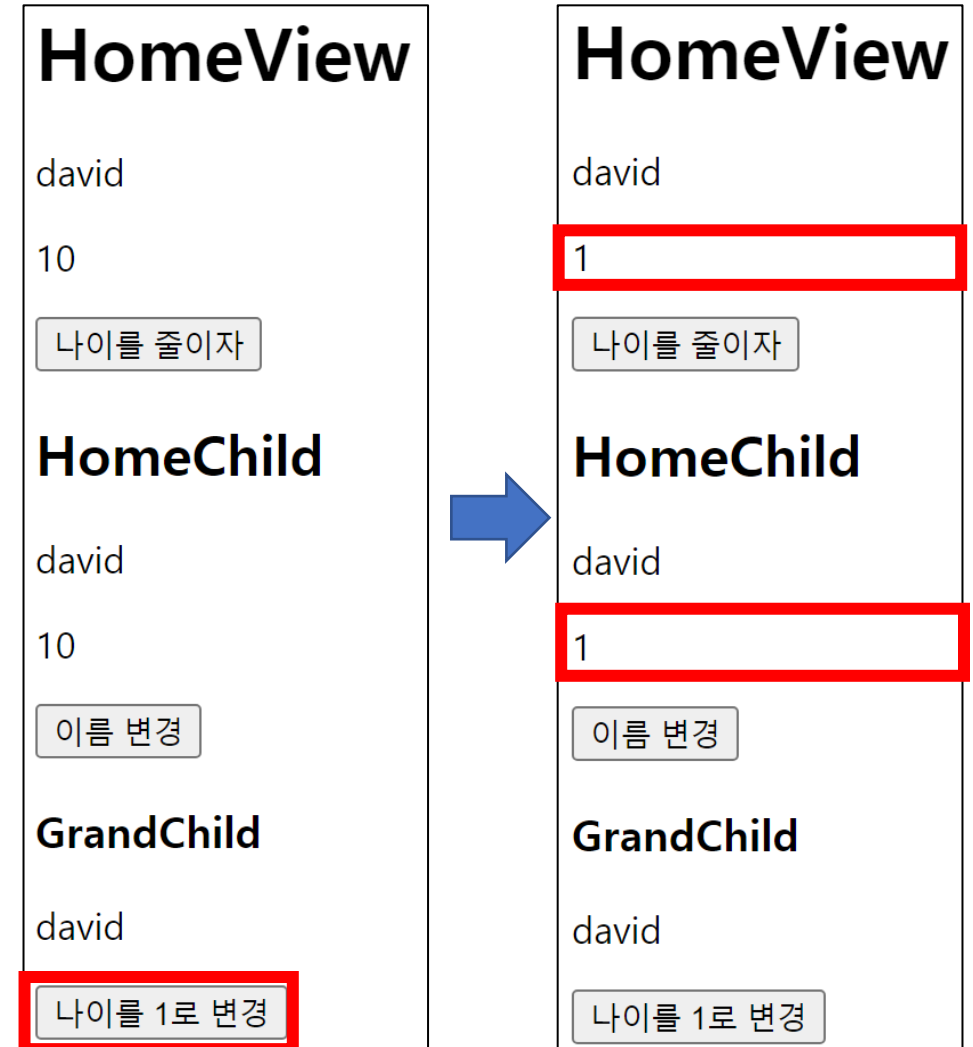
이름 변경

GrandChild

tom

손자에서 변경하기

- ✓ 손자에 버튼을 달고, 클릭 시 모든 age 를 1 로 변경



[도전] 할일 목록 만들기

✓ 부모 컴포넌트(App.vue)는 할 일 목록(todos)을 관리

자식 컴포넌트(TodoList.vue)는 할 일 배열을 props로 받아 목록을 렌더링

각 할 일 옆에는 "삭제" 버튼이 존재

"삭제" 버튼이 클릭되면, 자식 컴포넌트가 emit을 통해 해당 인덱스를 부모 컴포넌트로 전달

부모 컴포넌트는 해당 인덱스를 받아 해당 할 일을 삭제

App.vue코드

TodoList코드

Chapter 21

Vue Router로 페이지 이동 처리하기

Vue.js를 활용한 프론트 앤드 개발

✓ Vue 공식 패키지

- 라우트에 컴포넌트를 매핑
- 어떤 URL 에서 해당 컴포넌트를 렌더링할지 결정

✓ 다음과 같이 작업

- components/ 디렉터리의 모든 파일 삭제
- HomeView 컴포넌트를 기본 코드로 변경

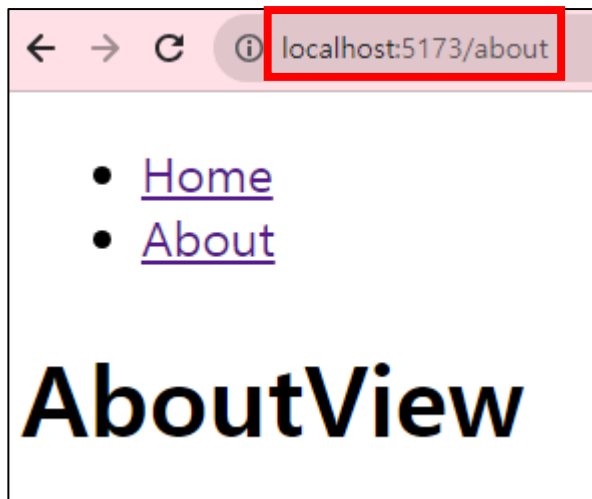
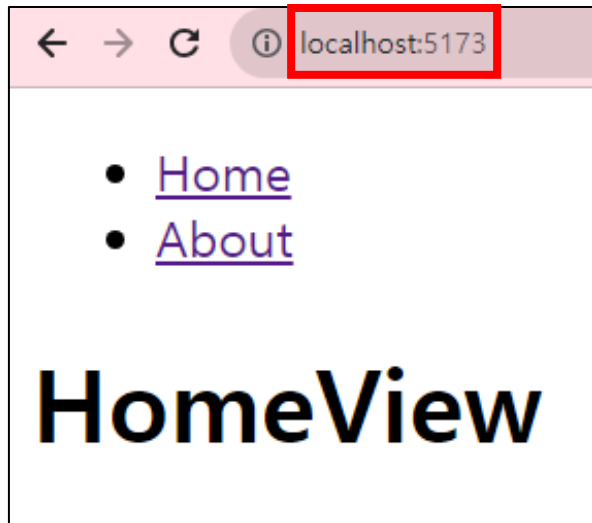
```
<script setup>
</script>

<template>
  <h1>HomeView</h1>
</template>

<style scoped>
</style>
```

Vue Router

- ✓ Home, About 링크를 클릭
 - URL 과 화면이 바뀜
 - / => HomeView
 - /about => AboutView
- ✓ 그러나, 새로고침 없이 부드럽게 화면 전환
 - 주소만 다를 뿐이지, 사실 하나의 HTML 이기 때문 (SPA)



router/index.js

- ✓ Vue Router 설정파일
- ✓ import 를 사용해 /views 에서 원하는 컴포넌트 가져옴
- ✓ 핵심은 routes 배열
 - path
 - route 경로
 - / => localhost:5173 접속 시 보여줄 페이지
 - name : route 이름
 - component : route 에 해당하는 component

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'home',
      component: HomeView
    },
    {
      path: '/about',
      name: 'about',
      // route level code-splitting
      // this generates a separate chunk (About.[hash].js) for this route
      // which is lazy-loaded when the route is visited.
      component: () => import('../views/AboutView.vue')
    }
  ]
})

export default router
```

용어 정리

- ✓ router : route 를 관리하는 객체
- ✓ route : 각각의 화면 객체
- ✓ route component
 - 각각의 route 에 연결 되어있는 특별한 컴포넌트
 - 각 화면의 루트 컴포넌트 역할
 - /views 디렉터리에 저장되며, 맨 끝에 -View 를 붙여 네이밍

```
routes: [  
  {  
    path: '/',  
    name: 'home',  
    component: HomeView  
  },  
  {  
    path: '/about',  
    name: 'about',  
    // route level code-splitting  
    // this generates a separate chunk (About.[hash].js) for this route  
    // which is lazy-loaded when the route is visited.  
    component: () => import('../views/AboutView.vue')  
  }  
]
```

Lazy Loading

- ✓ route 객체를 정의하는 두 가지 방법 중 하나
- ✓ 사용자가 방문하지 않은 route 의 컴포넌트를 다운로드하지 않고,
 - route 가 방문 될 때만 해당 컴포넌트를 가져옴
- ✓ 동적 import 로 구현
- ✓ 페이지 로딩 시간을 줄이는 효과
- ✓ Vue Router 공식문서에서 권장하는 방법

```
routes: [  
  {  
    path: '/',  
    name: 'home',  
    component: HomeView  
  },  
  {  
    path: '/about',  
    name: 'about',  
    // route level code-splitting  
    // this generates a separate chunk (About.[hash].js) for this route  
    // which is lazy-loaded when the route is visited.  
    component: () => import('../views/AboutView.vue')  
  }  
]
```

Lazy Loading

- ✓ 따라서, 오른쪽과 같이 코드 변경
 - ※ import HomeView 구문 삭제

```
import { createRouter, createWebHistory } from 'vue-router'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'home',
      component: () => import('../views/HomeView.vue')
    },
    {
      path: '/about',
      name: 'about',
      component: () => import('../views/AboutView.vue')
    }
  ]
})

export default router
```

App.vue

- ✓ <RouterLink to="/경로">
 - 화면에 보이는 모습은 <a> 태그와 동일
 - 링크를 클릭하면 해당 경로로 향함
 - 그러나, <a> 태그와 다르게 새로고침 발생 안함
- ✓ <RouterView />
 - route에 해당하는 화면이 실제 렌더링 되는 영역

```
<script setup>
import { RouterLink, RouterView } from "vue-router";
</script>

<template>
  <header>
    <nav>
      <ul>
        <li>
          <RouterLink to="/">Home</RouterLink>
        </li>
        <li>
          <RouterLink to="/about">About</RouterLink>
        </li>
      </ul>
    </nav>
  </header>
  <RouterView />
</template>
```

Named Routes

- ✓ Vue Router에선 path 대신 name 호출 권장
 - `<RouterLink :to="{ name: '이름' }">`
- ✓ v-bind (:) 사용해야 함
- ✓ 링크 클릭 시
 - 각각의 name 에 해당하는 컴포넌트가 `<RouterView />`에 보여짐

```
<ul>
  <li>
    <RouterLink to="/">Home</RouterLink>
  </li>
  <li>
    <RouterLink to="/about">About</RouterLink>
  </li>
</ul>
```



```
<ul>
  <li>
    <RouterLink :to="{ name: 'home' }">Home</RouterLink>
  </li>
  <li>
    <RouterLink :to="{ name: 'about' }">About</RouterLink>
  </li>
</ul>
```

Named Routes

- ✓ path 대신 name 호출 시 장점
 - 하드 코딩 된 URL 없음 : 오타 방지
 - params 자동 인코딩/디코딩 : 성능 향상

```
<ul>
  <li>
    <RouterLink to="/">Home</RouterLink>
  </li>
  <li>
    <RouterLink to="/about">About</RouterLink>
  </li>
</ul>
```



```
<ul>
  <li>
    <RouterLink :to="{ name: 'home' }">Home</RouterLink>
  </li>
  <li>
    <RouterLink :to="{ name: 'about' }">About</RouterLink>
  </li>
</ul>
```

Vue Router 설계 순서

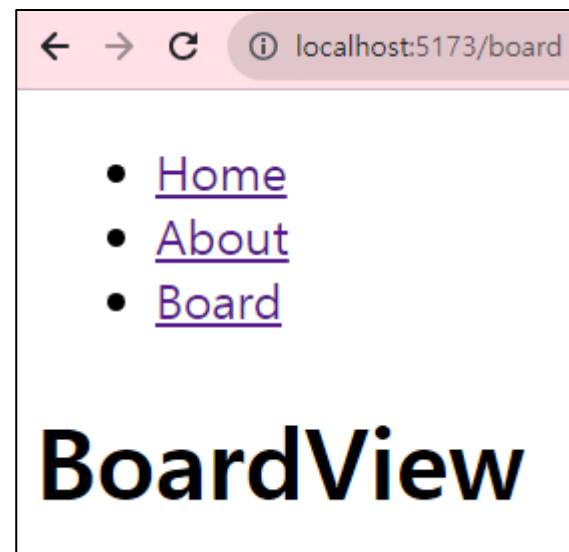
✓ 경로 설계

- views 디렉터리 안에 "-View.vue" route component 생성
- router/index.js에서 route 객체 정의
- 테스트

[도전] : route 생성

- ✓ 만들어야 할 route
 - path - /board
 - name - board
 - component - BoardView
- ✓ Lazy Loading 방식으로 생성
- ✓ 클릭 시 path 대신 name 호출

※ 링크 클릭 접속, URL 접속 둘 다 테스트



Javascript 에서 route 이동

- ✓ <template> 에서 링크로 route 이동 뿐만 아니라,
- ✓ JavaScript 에서 함수로 이동 가능
- ✓ 버튼을 눌렀을 때 다른 route로 이동
 - vue-router에서 제공하는 useRouter의 리턴으로 router 객체 받음
 - router.push({ name: "route 이름" })

```
<script setup>
import { useRouter } from "vue-router";

const router = useRouter();

function goToBoardRoute() {
  router.push({ name: "board" });
}
</script>

<template>
  <h1>HomeView</h1>
  <button @click="goToBoardRoute">게시판으로 가자</button>
</template>
```

Router 심화 params

✓ 만약 다음을 구현하고 싶다면?

- board/1 : 1번 글 상세 페이지
- board/2 : 2번 글 상세 페이지

✓ 경로에 params 변수를 추가하면 구현 가능!

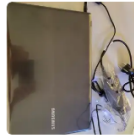
검색 결과 5,115 개의 상품 정렬 방식

현재 페이지의 상품 가격을 비교해봐요

평균 가격이에요 평균 394,823 원	가장 높은 가격이에요 최고 3,200,000 원	가장 낮은 가격이에요 최저 20,000 원
--------------------------	-------------------------------	----------------------------



삼성 S 노트북 카
본 제품입니다!
400,000원
경기도청북부청사
역 | 3분 전
Pay



삼성 노트북 중고
NT550P5C-...
400,000원
백운동 | 12분 전
Pay



i3-5세대>삼성 슬
림 15인치 사무용...
308,000원
구월4동 | 13분 전
Pay



삼성전자 갤럭시북3
프로 NT940XFT...
1,749,000원
롯데ON | 광고



삼성 노트북 P580
메인보드
20,000원
강동역 | 25분 전



홈 / 노트북PC / 노트북 / 삼성 노트북 중고NT550P5C-S85S입니다 ₩

삼성 노트북 중고NT550P5C-S85S입니다 ₩

400,000원 Pay

이 상품 시세조리와의 가격

14분 전 조회 30 명 0

배송비: 미함비 별도

중고나라 판매 사용

결제예비: 중고나라페이 첫 결제 실적
판매금 지급 수수료 무료

무이자대출: 1만원 이상 무이자 할부

거래방법: 직거래 택배거래 배송비 별도 물고

구매하기

params

- ✓ 상세 페이지를 담당할 route component 필요
 - BoardDetailView 생성

```
<script setup>
</script>

<template>
  <h1>Board Detail</h1>
</template>
```

params

- ✓ router/index.js 에 다음과 같이 작성
- ✓ path 작성 시, 받고 싶은 params 를 **":변수"** 형태로 정의

```
{  
  path: "/board/:id",  
  name: "detail",  
  component: () => import("../views/BoardDetailView.vue")  
}
```

params

- ✓ router/index.js 에 다음과 같이 작성
- ✓ path 작성 시, 받고 싶은 params 를 **":변수"** 형태로 정의
- ✓ 적용이 잘 되는지 알 수 없음
- ✓ 적어도 **몇 번 페이지**인지 화면에서 알 수 있어야 함

- [Home](#)
- [About](#)
- [Board](#)

Board Detail

params

- ✓ BoardDetailView를 다음과 같이 수정
- ✓ `useRouter`가 아니라, `useRoute` 사용
 - `useRouter` -> `router` 객체 생성
 - `useRoute` -> `route` 객체 생성
- ✓ `route.params` 객체의, `id` 변수에 접근
- ✓ 다음 경로 테스트
 - `/board/1`
 - `/board/2`

```
<script setup>
import {useRoute} from "vue-router";

const route = useRoute();
</script>

<template>
  <h1>{{ route.params.id }}번 글 상세페이지</h1>
</template>
```

✓ 다음 경로 테스트

- /board/1
- /board/2

/board/1

1번 글 상세페이지

/board/2

2번 글 상세페이지

params

- ✓ 현재 컴포넌트 : DetailView.vue
- ✓ <template> 과 <script setup> 에서 params 사용법
 - <RouterLink :to="{ name: 'route 이름', params: { 변수: '값' } }">
 - router.push({ name: "route 이름", params: { 변수: '값' } })

```
<script setup>
import { useRouter } from 'vue-router';

const router = useRouter();

function goToDetail(n) {
  router.push({ name: "detail", params: { id: n } })
}
</script>

<template>
  <h1>Board</h1>
  <div>
    <RouterLink :to="{ name: 'detail', params: { id: '1' } }">
      1번 페이지
    </RouterLink>
  </div>
  <button @click="goToDetail(1)">1번 페이지</button>
</template>
```

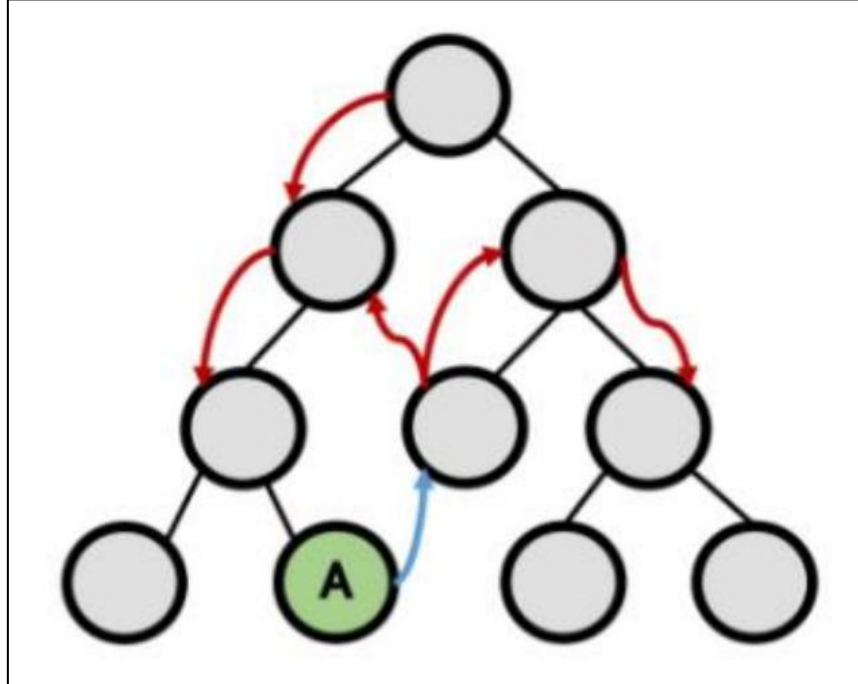
Chapter 22

Pinia를 활용한 상태 관리

Vue.js를 활용한 프론트 앤드 개발

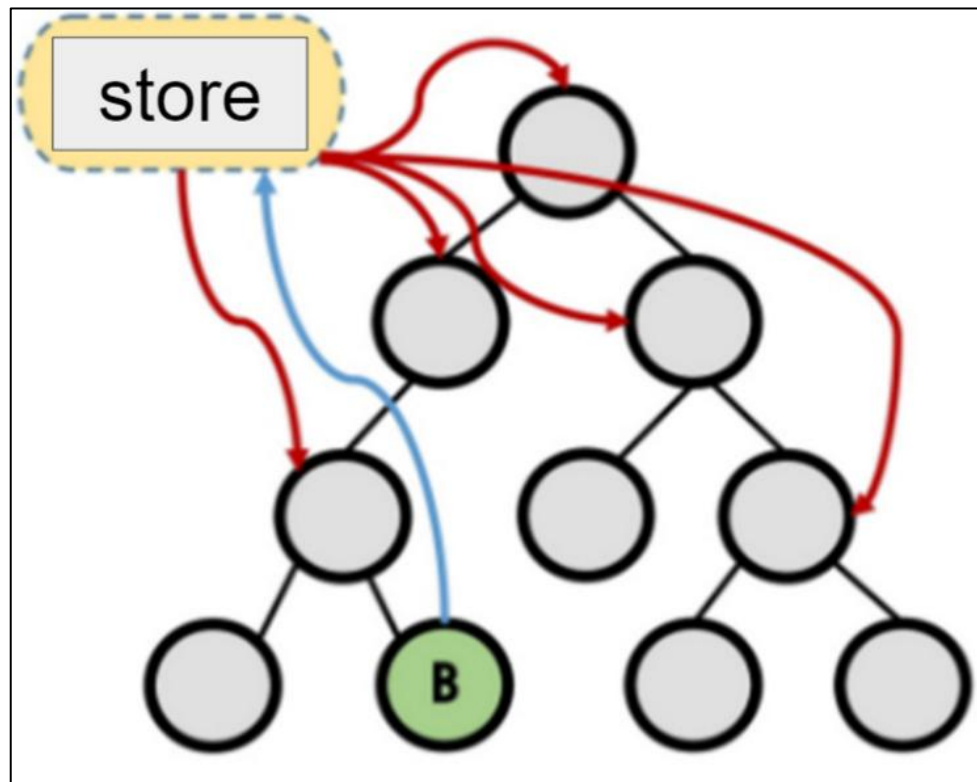
상태관리의 어려움

- ✓ 컴포넌트 구조가 복잡해지면, props/emit 으로 상태 관리가 매우 힘들어짐



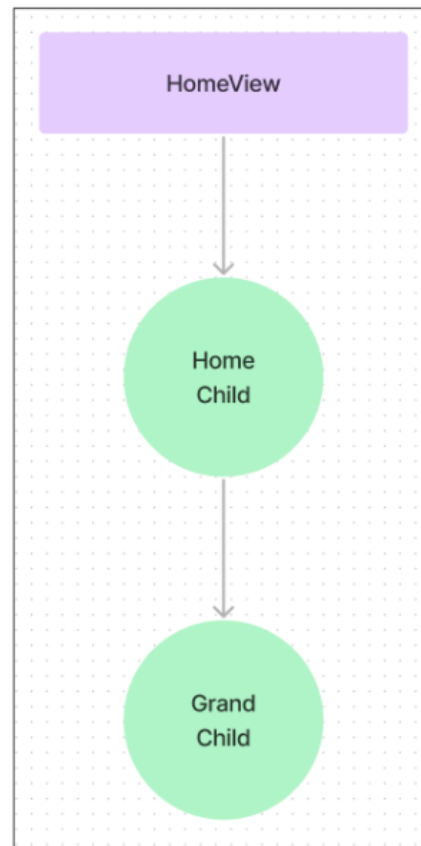
Pinia(피냐)

- ✓ 스페인으로 파인애플
- ✓ Vue 의 상태 관리 패턴 + 라이브러리
- ✓ store 라는 중앙 집중식 저장소에
 - ✓ 컴포넌트 간 공유가 많은 상태 저장
- ✓ 각 컴포넌트마다 상태를 가져오거나 변경
 - 컴포넌트 구조가 복잡해도, store 에 접근
 - 부모 자식 관계가 아니더라도, store 에 접근



✓ 다음 형태 준비

- 각각의 버튼은 함수 설정하지 말고, 빈 버튼으로 둬



HomeView

나이를 줄이자

HomeChild

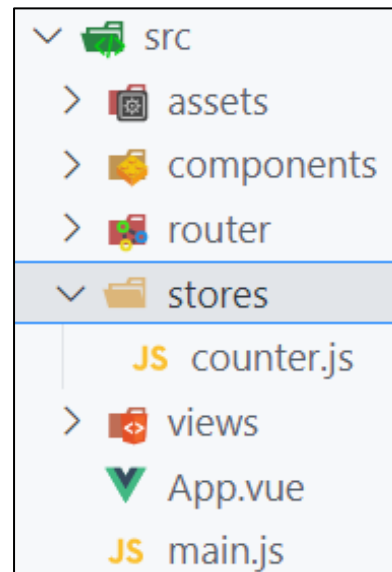
이름 변경

GrandChild

한살로 돌아가자

Pinia stores/

- ✓ src/stores/ 디렉터리에서 pinia 의 store 정의 가능
- 현재 counter.js 존재
- store 는 여러 개 정의 가능



Pinia stores/counter.js

✓ defineStore 사용해 store 정의

- pinia 에서 import 필요
- 첫번째 argument
 - store 이름
 - 전체 프로젝트에서 중복되면 안됨
 - 파일명과 일치하게 네이밍
- 두번째 argument
 - 콜백함수
 - state, computed, function 정의
 - 정의한 모든 것을 모아 리턴
 - 리턴값 네이밍 원칙
 - export const 로 선언
 - "use" + "store이름" + "Store"
 - ex) store 이름 counter => useCounterStore
 - 또는, store 이름 counterStore => useCounterStore

```
import { ref, computed } from 'vue'
import { defineStore } from 'pinia'

export const useCounterStore = defineStore('counter', () => {
  const count = ref(0)
  const doubleCount = computed(() => count.value * 2)
  function increment() {
    count.value++
  }

  return { count, doubleCount, increment }
})
```

Pinia stores/counter.js

✓ 세 가지가 pinia 의 핵심 개념

- state
 - 모든 컴포넌트에서 사용 가능한 변수
 - ref 를 사용해 선언
 - count
- getter
 - state 의 값을 변경시키지 않고, 다른 형태로 보여주기 위해 사용
 - computed 를 사용해 선언
 - doubleCount
 - count 의 값을 변경시키지 않고, count 두 배를 보여주기 위해 존재
- function
 - state 변경 로직 정의
 - function 키워드 사용
 - increment

```
import { ref, computed } from 'vue'
import { defineStore } from 'pinia'

export const useCounterStore = defineStore('counter', () => {
  const count = ref(0)
  const doubleCount = computed(() => count.value * 2)
  function increment() {
    count.value++
  }

  return { count, doubleCount, increment }
})
```


Pinia 컴포넌트에서 사용

✓ HomeView 를 다음과 같이 정의

- useCounterStore 를 counter store 의 경로에 접근해 import
- useCounterStore 의 리턴값은 객체이며, store 이름과 일치시킴
- counter 객체에 접근해 count state 사용

```
<script setup>
import {useCounterStore } from "@stores/counter";
import HomeChild from "@components/HomeChild.vue";

const counter = useCounterStore();
</script>

<template>
  <h1>HomeView</h1>
  <p>count: {{ counter.count }}</p>
  <button>나이를 줄이자</button>
  <HomeChild />
</template>
```

HomeView

count: 0

나이를 줄이자

Pinia 컴포넌트에서 사용

- ✓ 값 변경하는 법 두 가지
 - state 직접 변경
 - store 에 정의된 함수를 사용해 변경
 - ※ value 는 불이지 않음!

```
<script setup>
import {useCounterStore } from "@/stores/counter";
import HomeChild from "@/components/HomeChild.vue";

const counter = useCounterStore();
counter.count++ // 0 -> 1
counter.increment() // 1 -> 2
</script>
```

HomeView

count: 2

나이를 줄이자

Getters

- ✓ Store의 모든 getters 또한 state 처럼 직접 접근할 수 있음

```
<script setup>
console.log(store.doubleCount)
</script>

<template>
  <div>
    <p>getters : {{ store.doubleCount }}</p>
  </div>
</template>
```

Pinia 컴포넌트에서 사용

- ✓ <template> 에서 doubleCount (getter) 도 사용해보자

```
<template>
  <h1>HomeView</h1>
  <p>count: {{ counter.count }}</p>
  <p>doubleCount: {{ counter.doubleCount }}</p>
  <button>나이를 줄이자</button>
  <HomeChild />
</template>
```

HomeView

count: 2

doubleCount: 4

나이를 줄이자

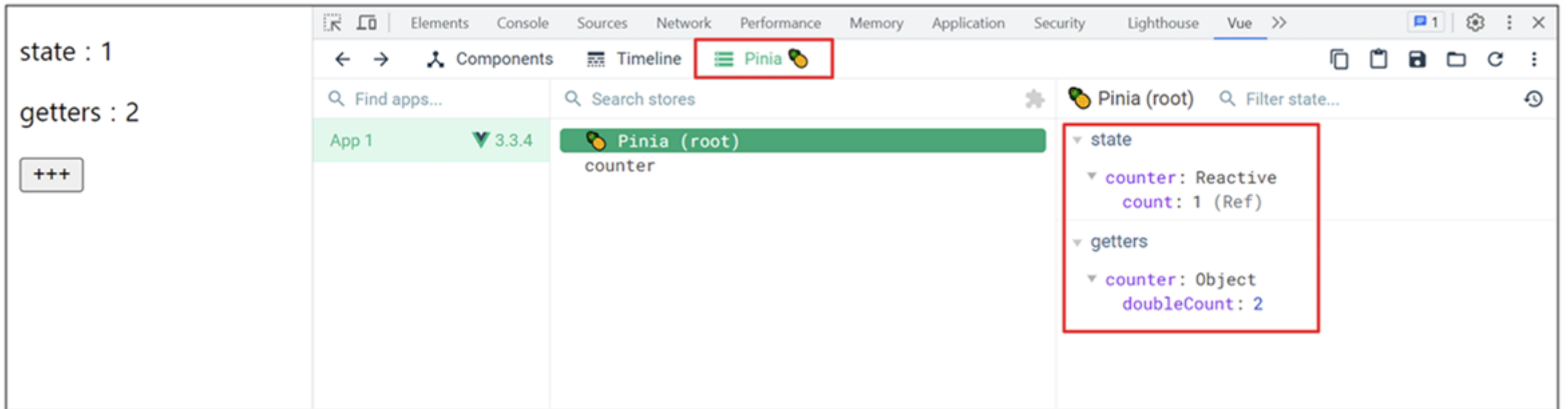
Actions

- ✓ Store의 모든 actions 또한 state 처럼 직접 접근할 수 있음
 - Getters와 달리 state 조작, 비동기, API 호출이나 다른 로직을 진행 할 수 있음

```
<template>
  <div>
    <button @click="store.increment()">+</button>
  </div>
</template>
```

참고)Vue devtools로 pinia 구성요소 확인

- ✓ pinia 를 배웠으면, props/emit 이 필요없을까?
 - pinia 를 도입해도 props/emit 과 함께 사용 가능
 - 부모 자식 컴포넌트 간 간단한 데이터 통신은 props/emit 사용
 - pinia 없이도 간단한 프로젝트 개발 가능



[도전] person store

- ✓ /stores 디렉터리에 person.js 를 생성
 - name: "david"
 - age: 10
- ✓ HomeView
 - name, age 출력
 - "나이를 줄이자" 버튼 클릭 시
 - age: 10 -> 9 -> 8 -> ...
- ✓ HomeChild
 - name, age 출력
 - "이름 변경" 버튼 클릭 시
 - age: "david" -> "tom"
- ✓ GrandChild
 - name 만 출력
 - "나이를 1로 변경" 버튼 클릭 시
 - age: 10 -> 1

HomeView

david

10

나이를 줄이자

HomeChild

david

10

이름 변경

GrandChild

david

나이를 1로 변경

[도전] 나의 형제에게

- HomeView 컴포넌트 아래 두 개의 자식 컴포넌트
 - HomeInput : 형제1. 입력
 - HomeDisplay : 형제2. 출력
- 형제1 에서 입력 즉시 형제2 에서 출력하라

나의 형제에게

형제1

안녕 나의 형제여

형제2

안녕 나의 형제여

777 미미 77 라 미미

```
import { ref } from "vue";
import { defineStore } from "pinia";

export const useMessengerStore = defineStore(
  "messenger", () => {
    const message = ref("");

    return { message };
  });
```

K 777 777 777 777

```
<script setup>
import HomeInput from "@/components/HomeInput.vue";
import HomeDisplay from "@/components/HomeDisplay.vue";
</script>

<template>
  <h1>나의 형제에게</h1>
  <HomeInput />
  <HomeDisplay />
</template>
```

K 스기7N 107B 207

```
<script setup>
import { useMessengerStore } from "@/stores/messenger";

const messenger = useMessengerStore();
</script>

<template>
  <h2>형제1</h2>
  <input
    type="text"
    placeholder="값을 입력하세요"
    v-model="messenger.message"
  />
</template>
```

K 스기7 K 107B 207

```
<script setup>
import { useMessengerStore } from "@/stores/messenger";

const messenger = useMessengerStore();
</script>

<template>
  <h2>형제2</h2>
  <p>{{ messenger.message }}</p>
</template>
```

Chapter 23

Axios로 백엔드와 통신

Vue.js를 활용한 프론트 앤드 개발

Fetch API

- ✓ 서버에서 JSON 데이터를 받아보자.
- ✓ JSONPlaceholder 활용 예정
 - GET /todos

{JSON} Placeholder

Free fake API for testing and prototyping.

Powered by [JSON Server](#) + [LowDB](#). Tested with [XV](#).

Serving ~2 billion requests each month.

```
▼ [
  ▼ {
    "userId": 1,
    "id": 1,
    "title": "delectus aut autem",
    "completed": false
  },
  ▼ {
    "userId": 1,
    "id": 2,
    "title": "quis ut nam facilis et officia qui",
    "completed": false
  },
  ▼ {
    "userId": 1,
    "id": 3,
    "title": "fugiat veniam minus",
    "completed": false
  },
]
```

Fetch API

- ✓ todos : 패치 후의 데이터 저장
- ✓ 화면 시작과 동시에, 데이터 패치 실행
- ✓ 데이터를 화면에 출력

No Issues

data fetch [HomeView.vue:3](#)

> |

```
<script setup>
import { ref } from "vue";

const todos = ref([]);

function dataFetch() {
  console.log("data fetch");
}

dataFetch();
</script>

<template>
  <h1>Data Fetch</h1>
</template>
```

Fetch API

✓ URL 변수 추가

- `const URL = "https://~~"`
 - URL 은 변경될 일이 없기에, 상태로 선언 안함

✓ 함수 작성

- `async/await` : 비동기 구문
 - `async` : 비동기 함수임을 명시
 - `await` : 해당 구문에선 동기적으로 동작
- `{ method: "GET" }` : HTTP GET 방식으로 데이터 가져옴
 - 데이터는 `response.json()` 호출 시 획득 가능
 - 가져온 데이터를 `data` 상태에 저장

```
<script setup>
import { ref } from "vue";

const todos = ref([]);
const URL = "https://jsonplaceholder.typicode.com/todos"
;

async function dataFetch() {
  try {
    const response = await fetch(URL, { method: "GET" })
    ;
    console.log(response);
    const jsonData = await response.json();
    console.log(jsonData);
    todos.value = jsonData;
  } catch (error) {
    console.error(error);
  }
}

dataFetch();
</script>

<template>
  <h1>Data Fetch</h1>
  <ul>
    <li v-for="todo in todos" :key="todo.id">{{ todo.title }}</li>
  </ul>
</template>
```

Data Fetch

- delectus aut autem
- quis ut nam facilis et officia qui
- fugiat veniam minus
- et porro tempora
- laboriosam mollitia et enim quasi
- qui ullam ratione quibusdam voluptatem
- illo expedita consequatur quia in
- quo adipisci enim quam ut ab

[도전] Fetch API

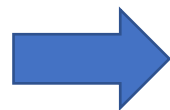
✓ 모든 코드 삭제

✓ JSONPlaceholder 에서, **GET /todos** 데이터 출력

✓ 사용자가 지루하지 않도록, 데이터가 들어오지 않았다면 loading... 출력

Data Fetch

loading...



Data Fetch

- delectus aut autem
- quis ut nam facilis et officia qui
- fugiat veniam minus


```
<script setup>
import { ref } from "vue";

const URL = "https://jsonplaceholder.typicode.com/todos";
const todos = ref([]);

async function dataFetch() {
  try {
    const response = await fetch(URL, { method: "GET" });
    console.log(response);
    const jsonData = await response.json();
    console.log(jsonData);
    todos.value = jsonData;
  } catch (error) {
    console.error(error);
  }
}

dataFetch();
</script>

<template>
  <h1>Data Fetch</h1>
  <ul v-if="todos.length">
    <li v-for="todo in todos" :key="todo.id">
      <p>{{ todo.title }}</p>
    </li>
  </ul>
  <p v-else>loading...</p>
</template>
```

A X I O S

시작하기

브라우저와 node.js를 위한 Promise 기반 HTTP 클라이언트 라이브러리

Axios는 브라우저와 node.js를 위한 간단한 Promise 기반 HTTP 클라이언트입니다.
Axios는 확장 가능한 인터페이스를 가진 작은 패키지로 사용하기 쉬운 라이브러리를 제공합니다.

시작하기

GitHub 보러가기

```
import axios from "axios"
```

fetch를 axios로 변경

```
<script setup>
import { ref } from "vue";
import axios from "axios";
const URL = "https://jsonplaceholder.typicode.com/todos";
const todos = ref([]);
async function dataFetch() {
  try {
    const response = await axios.get(URL);
    console.log(response);
    const jsonData = response.data;
    console.log(jsonData);
    todos.value = jsonData;
  } catch (error) {
    console.error(error);
  }
}
dataFetch();
</script>
<template>
  <div>
    <h1>Data Fetch</h1>
    <ul v-if="todos.length">
      <li v-for="todo in todos" :key="todo.id">
        <p>{{ todo.title }}</p>
      </li>
    </ul>
    <p v-else>loading...</p>
  </div>
</template>
```

코드

✓ Fetch 사용 권장

- 간단한 API 호출
- 추가 라이브러리 설치를 원하지 않을 때
- 최신 브라우저 기능 활용

✓ Axios 사용 권장

- 복잡한 API 통신
- 자동 JSON 파싱이 필요할 때
- 인터셉터, 타임아웃 등 고급 기능 필요
- 일관된 에러 처리

✓ 성능

✓ **Fetch**: 네이티브 API로 더 가벼움

✓ **Axios**: 추가 기능으로 인해 약간 무거움

결론: 간단한 프로젝트는 fetch, 복잡한 프로젝트는 axios가 편리

Chapter 24

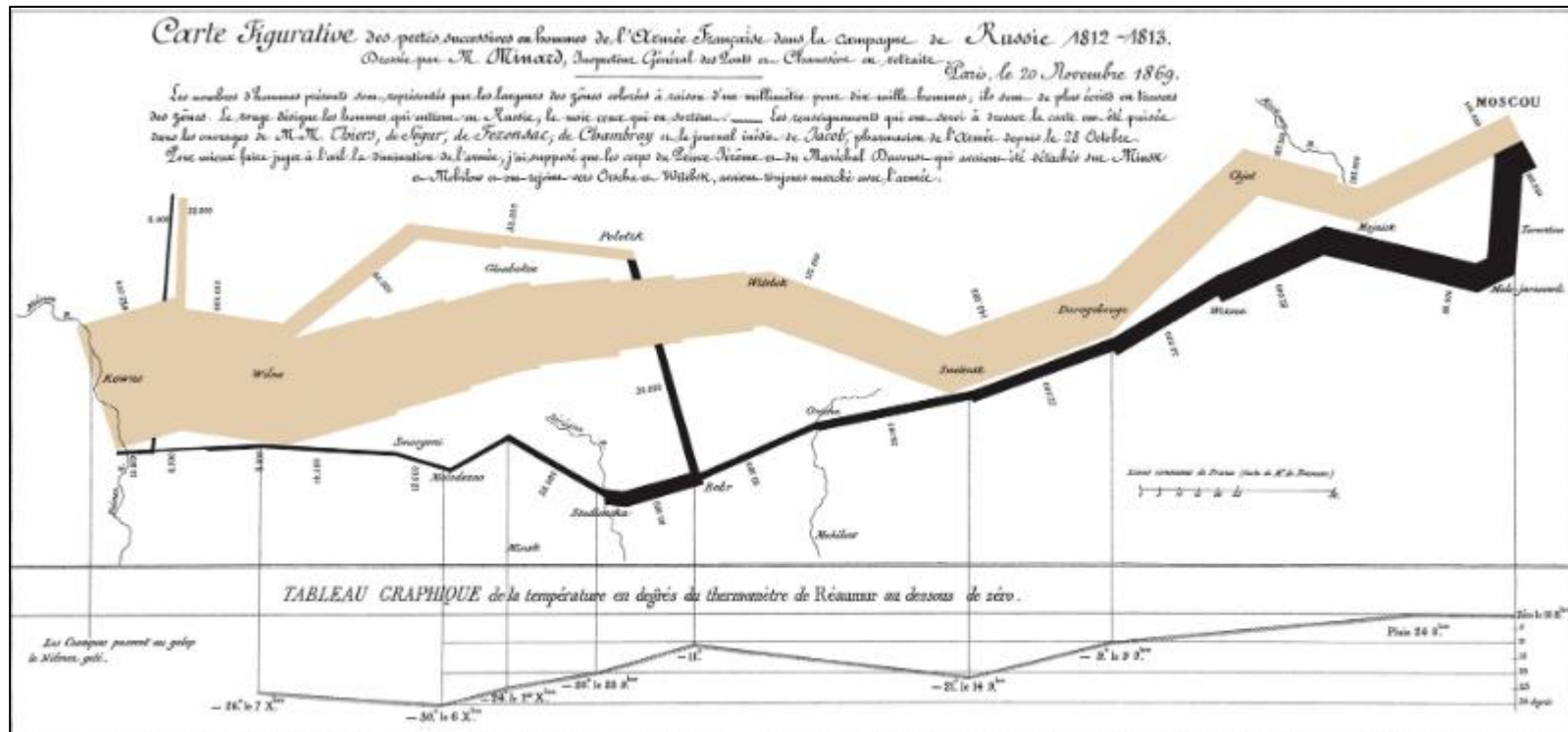
Echarts로 데이터 시각화 하기

Vue.js를 활용한 프론트 앤드 개발

데이터 시각화?

✓ 복잡하고 다양한 데이터를 사람들이 이해하기 쉬운 형태인 그래프, 차트, 지도, 대시보드 등 시각적 요소로 표현

우수 사례



링크

Echarts?

- ✓ Baidu에서 개발한 오픈소스 데이터 시각화 라이브러리
 - 막대그래프, 선그래프, 파이차트, 히트맵 등 다양한 시각화 유형을 지원
 - 인터랙티브하고 고성능의 차트를 웹에서 구현
- ✓ Vue와 함께 사용할 경우,
 - 컴포넌트 기반으로 차트를 재사용하고, 반응형 데이터 바인딩까지 가능
 - 실시간 대시보드나 데이터 기반 SPA 구축에 매우 적합

Echarts 설치 & main.js 설정

- ✓ npm install echarts
- ✓ npm install vue-echarts

```
import './assets/main.css';

import { createApp } from 'vue';
import App from './App.vue';

import * as echarts from 'echarts';
import VueECharts from 'vue-echarts';

const app = createApp(App);

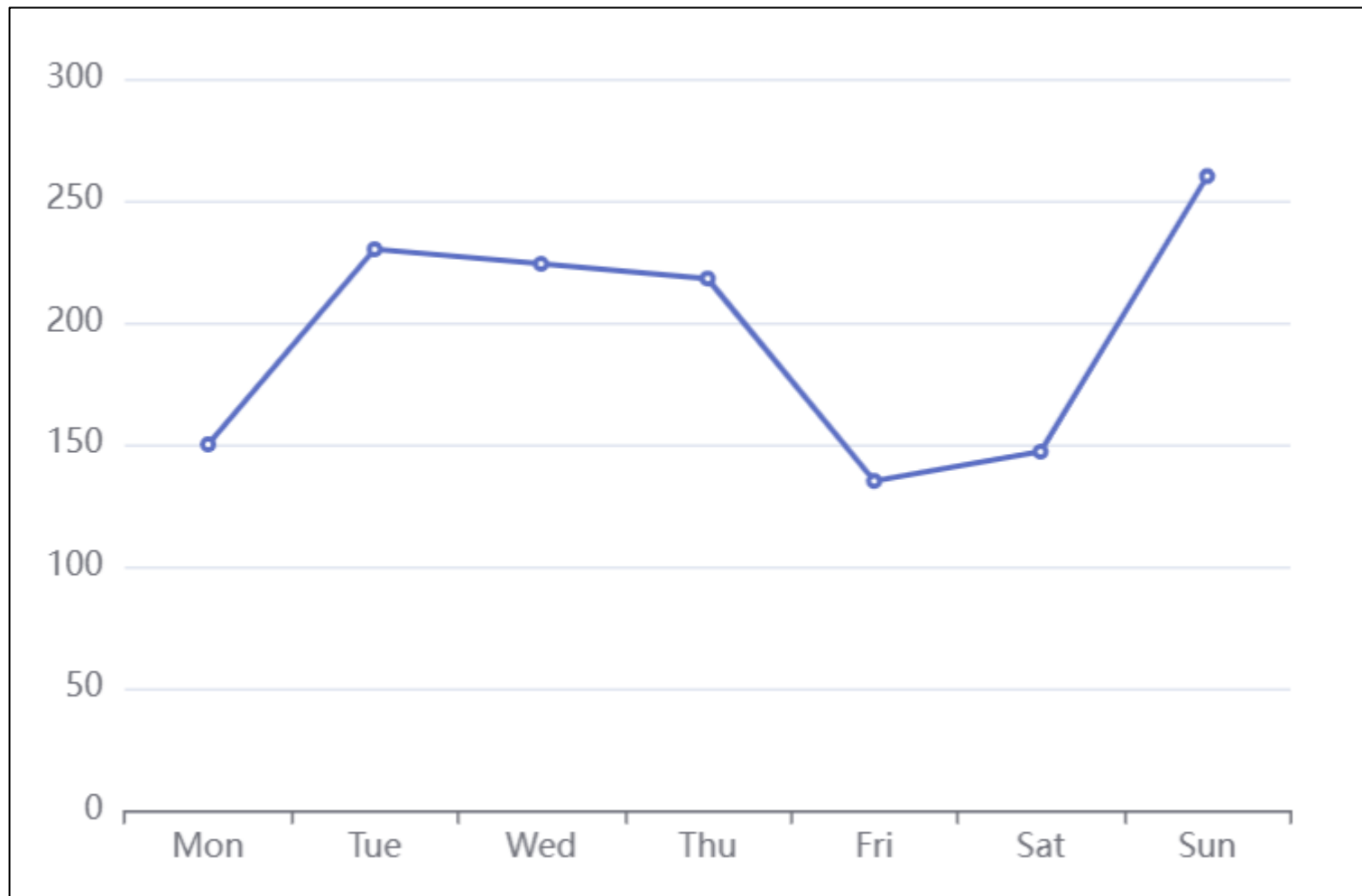
app.component('v-chart', VueECharts);

app.mount('#app');
```

```
npm install echarts
```

```
npm install vue-echarts
```


Echarts 사용



링크

코드

Echarts 옵션 이해

- ✓ option
- ✓ series
- ✓ tooltip

- 차트 반응형(Responsive) 처리
- 상호작용(tooltip, click, legend 선택 등)
- 필수 구성 요소: title, tooltip, legend, xAxis, yAxis, series)
- API와 연동해 실시간 데이터 시각화
- 반응형 화면 처리 (resize, 모바일 대응)
- 자주 사용하는 차트 옵션